

Snowflake Master Course — Zero to Hero

- [Snowflake Master Course — Zero to Hero](#)
 - [Part 1: Foundations & Core Features \(Chapters 1–6\)](#)
 - [Chapter 1: Introduction to Snowflake & Cloud Data Warehousing](#)
 - [Chapter 2: Snowflake Architecture Deep Dive](#)
 - [Chapter 3: Getting Started — Environment Setup](#)
 - [Chapter 4: Database Objects & Data Modeling](#)
 - [Chapter 5: Loading Data into Snowflake](#)
 - [Chapter 6: SQL in Snowflake](#)
 - [Chapter 7: Virtual Warehouses & Performance Optimization](#)
 - [Chapter 8: Security & Access Control](#)
 - [Chapter 9: Time Travel, Cloning & Data Protection](#)
 - [Chapter 10: Data Sharing & Collaboration](#)
 - [Chapter 11: Snowpark — Code in the Warehouse](#)
 - [Chapter 12: Streamlit in Snowflake](#)
 - [Part 2 Recap](#)
 - [Table of Contents](#)
- [Chapter 13: Snowflake Cortex AI](#)
 - [13.1 Introduction to Cortex AI](#)
 - [13.2 LLM Functions](#)
 - [13.3 Cortex Search — Semantic Search](#)
 - [13.4 Document AI — Structured Extraction from PDFs](#)
 - [13.5 Cortex Analyst — Natural Language to SQL](#)
 - [13.6 Cortex ML Functions — No-Code Machine Learning](#)
 - [Chapter 13 Summary](#)

- [Chapter 14: Data Governance with Snowflake Horizon](#)
 - [14.1 Snowflake Horizon — Unified Governance](#)
 - [14.2 Object Tagging](#)
 - [14.3 Dynamic Data Masking](#)
 - [14.4 Row Access Policies](#)
 - [14.5 Access History and Column-Level Lineage](#)
 - [Chapter 14 Summary](#)
- [Chapter 15: Cost Management and Optimization](#)
 - [15.1 The Snowflake Pricing Model](#)
 - [15.2 Resource Monitors](#)
 - [15.3 Cost Analysis Queries](#)
 - [15.4 Cost Optimization Strategies](#)
 - [Chapter 15 Summary](#)
- [Chapter 16: Enterprise Architecture and DevOps](#)
 - [16.1 Multi-Account Strategy](#)
 - [16.2 Infrastructure as Code with Terraform](#)
 - [16.3 dbt with Snowflake](#)
 - [16.4 Schema Version Control with schemachange](#)
 - [16.5 GitHub Actions CI/CD Pipeline](#)
 - [Chapter 16 Summary](#)
- [Chapter 17: Monitoring and Observability](#)
 - [17.1 The Two Monitoring Schemas](#)
 - [17.2 Essential Monitoring Queries](#)
 - [17.3 Snowflake Alerts](#)
 - [17.4 Event Tables — Structured Application Logging](#)
 - [17.5 Snowsight Monitoring Dashboards](#)
 - [Chapter 17 Summary](#)
- [Chapter 18: Advanced Topics and Enterprise Patterns](#)
 - [18.1 The Medallion Architecture in Snowflake](#)
 - [18.2 Iceberg Tables — Open Lakehouse Format](#)
 - [18.3 Hybrid Tables — Operational Analytics \(UNISTORE\)](#)
 - [18.4 Snowflake Native Apps](#)
 - [18.5 Performance at Enterprise Scale](#)

- [18.6 Enterprise Naming Conventions and Standards](#)
- [18.7 The Complete Snowflake Architecture Reference](#)
- [Chapter 18 Summary and Course Conclusion](#)

Snowflake Master Course — Zero to Hero

Part 1: Foundations & Core Features (Chapters 1–6)

Chapter 1: Introduction to Snowflake & Cloud Data Warehousing

Learning Objectives

By the end of this chapter you will be able to: explain why Snowflake was built, articulate its key differentiators versus competitors, identify the correct Snowflake edition for a given use case, and describe the full platform ecosystem.

1.1 The Data Warehouse Problem

For decades, enterprise analytics ran on on-premises Massively Parallel Processing (MPP) appliances — Teradata, Netezza, Greenplum, Vertica. These systems were revolutionary for their time: they could scan billions of rows in seconds using armies of commodity CPUs working in parallel. But they came with a brutal set of constraints.

The hardware provisioning trap. Buying an MPP appliance meant committing to a fixed amount of compute and storage, purchased together as a unit. If your queries needed more CPU, you bought more nodes — and those nodes also came with disk you didn't need. If your data grew faster than your compute, you were stuck. Procurement cycles ran 6–18 months. By the time new hardware arrived, requirements had changed again.

The scaling ceiling. On-premises systems couldn't elastically scale. A report that ran fine at midnight failed at 9 a.m. when fifty analysts were simultaneously querying the same cluster. The only solution was to buy more hardware — or tell people to wait.

The maintenance tax. DBAs spent significant time on tasks that added no business value: vacuuming, re-clustering, compressing, patching, monitoring hardware health, managing RAID arrays. Skilled data engineers were occupied keeping the lights on rather than building data products.

The multi-workload conflict. ETL pipelines competed with ad-hoc queries. Data scientists running ML jobs would grind the warehouse to a halt for reporting users. There was no clean way to isolate workloads without physically separate clusters.

The cloud changed everything. Amazon S3 offered virtually unlimited, durable, cheap object storage. EC2 offered on-demand compute that could be provisioned in minutes and shut down when idle. A new class of data warehouse — born in the cloud, not ported to it — became possible.

1.2 What Is Snowflake?

Snowflake was founded in 2012 by Benoit Dageville and Thierry Cruanes, two former Oracle database engineers, along with Marcin Żukowski, co-creator of Vectorwise. Their thesis was simple: build a cloud-native data platform from scratch, leveraging cloud object storage for limitless storage and cloud compute for elastic, independently scalable processing.

The company launched its product in 2014, raised \$263 million before going public, and in September 2020 completed what was — at the time — the largest software IPO in history, raising \$3.4 billion at a valuation of \$33 billion.

Snowflake is not a database. It is not a port of an on-premises system. It is a **Data Cloud** — a platform built entirely for cloud infrastructure that encompasses:

- A fully managed, ANSI-SQL cloud data warehouse
- A data lake (structured and semi-structured data at scale)
- A data engineering platform (streaming, pipelines, orchestration)
- A data science and ML platform (Snowpark, Snowflake ML)
- A data application platform (Native Apps, Streamlit in Snowflake)
- A data collaboration and monetization network (Marketplace, Data Sharing)
- An AI/ML platform (Cortex AI, Document AI, Cortex Analyst)

Critically, Snowflake runs identically on **AWS, Azure, and GCP**. You write the same SQL, use the same APIs, and get the same features regardless of cloud provider. Cross-cloud data sharing and replication are first-class features.

1.3 Key Differentiators

The following table compares Snowflake to its primary competitors across the dimensions that matter most to enterprise data teams:

Dimension	Snowflake	AWS Redshift	Google BigQuery	Databricks
Scaling model	Independent storage & compute; multi-cluster	Resize cluster; separate RA3 nodes	Serverless (auto-scales)	Cluster-based; auto-scaling clusters
Management overhead	Near-zero (fully managed)	Moderate (vacuum, sorts, WLM tuning)	Very low (serverless)	Moderate (cluster config, Spark tuning)
Semi-structured data	Native VARIANT (JSON, Avro, Parquet, XML)	SUPER type (limited)	JSON via STRUCT/ARRAY	Native via DataFrames & Delta
Pricing model	Credits (compute) + storage; pay per second	Hourly node pricing + S3	Per-query (TB scanned) + storage	DBU credits + cloud infra
Open format support	Iceberg Tables (read/write); Delta sharing	Good (external tables)	BigLake (external)	Native Delta Lake
ML/AI native	Cortex AI, Snowpark ML, Snowflake ML Functions	SageMaker integration	Vertex AI integration	MLflow, native LLM serving
Multi-cloud	Yes (AWS, Azure, GCP)	AWS only	GCP only	All clouds
Data sharing	Zero-copy, live, cross-account and cross-cloud	Limited	Analytics Hub	Delta Sharing

The separation of storage and compute is the defining architectural choice. In Snowflake, storage and compute scale independently — you can grow your dataset to petabytes without paying for more CPU, and you can spin up a massive compute cluster for a one-time job and shut it down 30 minutes later. This is simply not possible in traditional MPP systems.

No index management. Snowflake uses micro-partitions (covered in Chapter 2) and automatic metadata-driven pruning. You never create an index, choose a sort key, or define a distribution key. This eliminates an entire category of DBA work and makes Snowflake accessible to teams without dedicated database administrators.

Native semi-structured data. The `VARIANT` data type stores any JSON, Avro, XML, or Parquet document natively — up to 16MB — with full SQL querying support using dot notation, bracket notation, and `LATERAL FLATTEN`. No pre-defined schemas for semi-structured sources, no ETL to flatten JSON before loading.

1.4 Snowflake Editions

Snowflake offers four editions, each a superset of the previous:

Edition	Key Features	Target User
Standard	Full SQL DWH, Time Travel (1 day), Fail-Safe, Snowpipe, all connectors, all languages	Teams starting out; cost-sensitive workloads
Enterprise	Multi-cluster warehouses, Time Travel up to 90 days, Materialized Views, Column-level security, Annual/pre-paid pricing	Enterprise data teams needing workload isolation and governance
Business Critical	HIPAA compliance, PCI DSS, SOC 2, FedRAMP (AWS), enhanced encryption, Private Link, Tri-Secret Secure, database failover	Regulated industries: healthcare, finance, government
Virtual Private Snowflake (VPS)	Dedicated metadata store (no multi-tenancy), customer-managed Snowflake environment, highest isolation	Intelligence agencies, ultra-sensitive workloads

Pro Tip: Most enterprise teams should start with Enterprise edition. The jump to Business Critical is justified when you handle PHI/PII under HIPAA, need PCI DSS compliance for payment data, or your legal/security team requires Private Link.

1.5 Deployment Options

Snowflake runs on all three major cloud providers. Choose based on where your data already lives:

- **AWS:** `us-east-1`, `us-west-2`, `eu-west-1`, `ap-southeast-1`, and many more
- **Azure:** `eastus2`, `westeurope`, `australiaeast`, etc.
- **GCP:** `us-central1`, `europa-west4`, etc.

All regions offer the same feature set. Cross-region and cross-cloud replication enables disaster recovery and global distribution.

1.6 Core Use Cases

- 1. Cloud Data Warehouse:** Replace on-premises Teradata/Netezza/Oracle EDW. Store structured data from ERP, CRM, and transactional systems. Power BI/Tableau/Looker dashboards.
 - 2. Data Lake / Lakehouse:** Store raw semi-structured data (JSON clickstreams, IoT sensor data, API responses) alongside structured data. Query directly with SQL — no ETL required to flatten first.
 - 3. Data Engineering:** Build pipelines using Streams + Tasks, Dynamic Tables, Snowpipe for continuous ingestion, and Snowpark for complex transformations.
 - 4. Data Science & ML:** Use Snowpark Python to run pandas/scikit-learn/XGBoost inside Snowflake without moving data. Train and serve models via Snowflake ML and the Model Registry.
 - 5. Data Sharing & Monetization:** Share live datasets with partners, customers, or other business units instantly with zero-copy Data Sharing. Publish datasets on the Snowflake Marketplace for revenue.
 - 6. Data Applications:** Build operational applications using Hybrid Tables (row + columnar), Native Apps, and Streamlit in Snowflake.
-

1.7 The Snowflake Ecosystem



Chapter 1 Summary

Key Takeaway	Detail
Snowflake was built for the cloud	Not ported — designed from scratch to leverage object storage + elastic compute
Separation of storage and compute	Scale each independently; pay only for what you use
Multi-cloud native	Identical features and SQL on AWS, Azure, and GCP
More than a database	DWH + Data Lake + Engineering + Science + Apps + AI
Edition matters	Standard for start; Enterprise for most; Business Critical for regulated industries

What's Next: In Chapter 2 we'll open up the hood and examine exactly how Snowflake's three-layer architecture works — from micro-partitions on object storage to the cloud services brain that orchestrates it all.

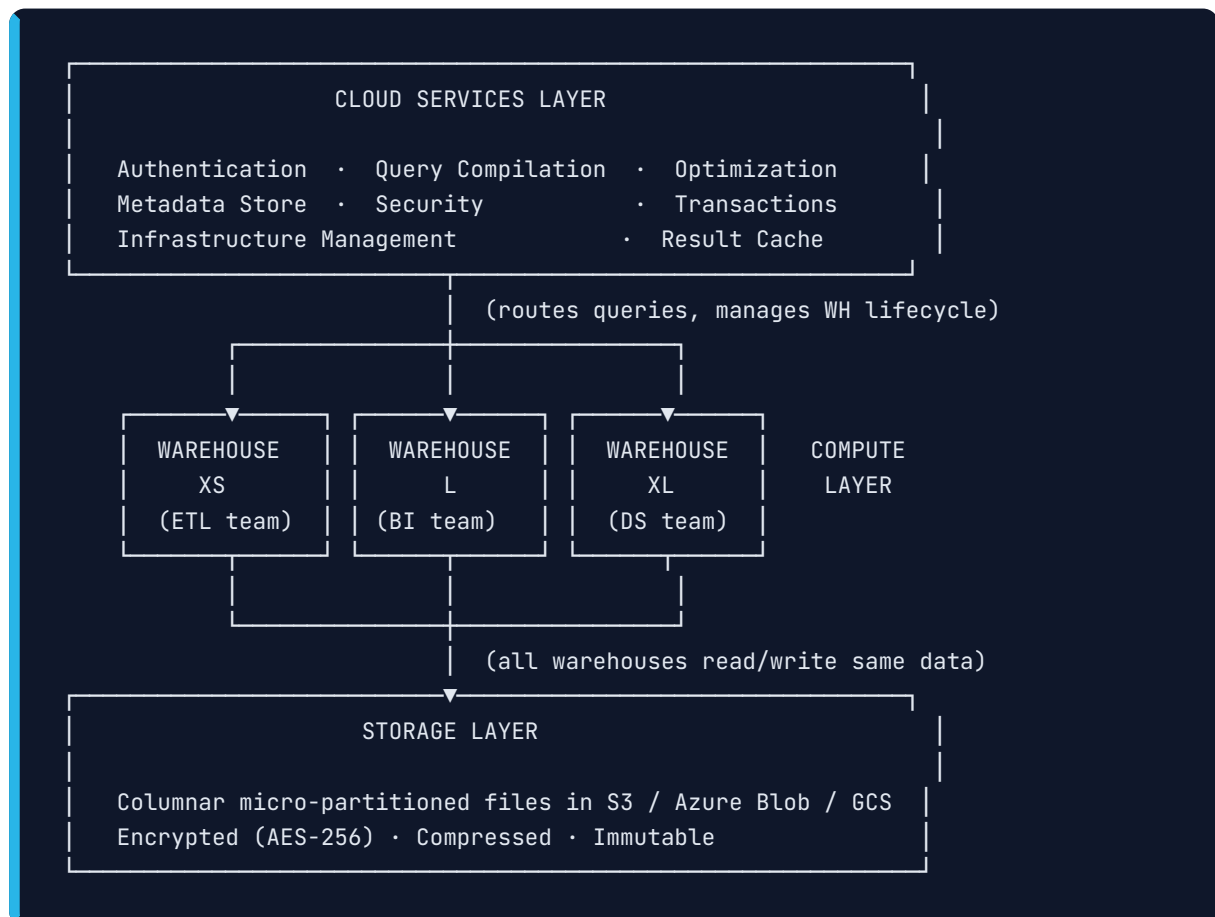
Chapter 2: Snowflake Architecture Deep Dive

Learning Objectives

Understand the three layers of Snowflake's architecture, how micro-partitions store and index data, how virtual warehouses execute queries in parallel, how the three caching layers work, and how a query travels end-to-end from client submission to result delivery.

2.1 The Multi-Cluster Shared Data Architecture

Snowflake's architecture has three completely decoupled layers:



The critical insight: **multiple compute clusters (warehouses) can simultaneously read and write the same storage.** An ETL warehouse and a BI warehouse query the same tables without any data movement or conflict. This is the “shared data” in the architecture name.

2.2 The Storage Layer

All data in Snowflake is stored in **cloud-provider object storage** — Amazon S3, Azure Blob Storage, or Google Cloud Storage. Snowflake manages this storage internally; you interact with it only through SQL. You never see raw files.

Data is stored in a proprietary **columnar file format** (conceptually similar to Parquet). Key properties:

- **Columnar**: values for each column are stored together, enabling efficient column-level compression and scan
 - **Compressed**: Snowflake automatically selects the best compression algorithm per column (LZO, Zstandard, etc.). Typical compression ratios are 3x–7x
 - **Encrypted**: AES-256 encryption at rest, always. You cannot disable it
 - **Immutable**: files are never modified in place. DML (INSERT/UPDATE/DELETE) writes new files and marks old ones for deletion
 - **Managed**: Snowflake handles all file management, garbage collection, and compaction
-

2.3 Micro-Partitions: The Engine of Performance

Every Snowflake table is divided into **micro-partitions** — the fundamental storage unit:

- **Size**: 50MB to 500MB of uncompressed data per micro-partition
- **Automatic**: Snowflake creates them automatically as data is loaded; you never define them
- **Contiguous on disk**: data within a partition is ordered by insertion order by default
- **Columnar internally**: each micro-partition stores data column-by-column for compression efficiency

What makes micro-partitions powerful is the **metadata** Snowflake stores for each one:

Micro-Partition #1042 (stored in object storage)

Metadata (stored in Cloud Services – ultra-fast lookup):

Column	Min Value	Max Value	Null Count
order_date	2024-01-01	2024-01-07	0
region	'EAST'	'WEST'	0
amount	10.00	9,999.00	12
status	'CANCELLED'	'SHIPPED'	0

Row count: 180,000 • Distinct values: stored per column

Partition Pruning is how Snowflake uses this metadata to skip irrelevant data entirely:

```
SELECT SUM(amount) FROM orders WHERE order_date = '2024-03-15';
```

Before touching any actual data, the Cloud Services layer scans the metadata for every micro-partition and asks: “Can this partition contain rows where `order_date` = '2024-03-15'?” If a partition’s metadata shows `order_date` ranges from 2024-01-01 to 2024-01-07, it is pruned — skipped entirely. Only partitions whose date ranges overlap 2024-03-15 are fetched and scanned. On a 10TB table, this might mean reading 50MB instead of 10TB.

Pro Tip: Partition pruning is why your filter columns matter. Columns that data is naturally sorted by (usually date/timestamp for append-heavy tables) get excellent pruning for free. Columns with no natural order (like `customer_id` on a table loaded randomly) get poor pruning without a clustering key (Chapter 7).

2.4 The Compute Layer — Virtual Warehouses

A **Virtual Warehouse** is an on-demand MPP cluster of cloud virtual machines. Key characteristics:

- **Independent:** each warehouse is isolated; multiple warehouses can run simultaneously against the same data

- **Ephemeral:** spin up in seconds, suspend when idle (auto-suspend), resume instantly on next query (auto-resume)
- **Sized:** from XS (1 node) to 6XL (512 nodes equivalent). Each step doubles compute capacity and credit consumption
- **Cached:** each warehouse has local SSD storage for caching recently accessed micro-partitions

Warehouse Size	Credits/hr	Relative Compute	Good For
X-Small (XS)	1	1×	Dev/test, light ad-hoc queries
Small (S)	2	2×	Small BI workloads
Medium (M)	4	4×	General-purpose analytics
Large (L)	8	8×	Data loading, heavier queries
X-Large (XL)	16	16×	Complex analytics, 100M+ rows
2X-Large (2XL)	32	32×	Large-scale transformations
3X-Large (3XL)	64	64×	Very large ETL jobs
4X-Large (4XL)	128	128×	Massive parallel loads
5X-Large (5XL)	256	256×	Extreme-scale workloads
6X-Large (6XL)	512	512×	Maximum compute available

Warning: Bigger is not always better. Queries that are not CPU-bound (e.g., a simple SELECT with a highly selective filter) run in the same time on an XS as on an XL — but cost 16× more. Always profile before upsizing. Upsizing helps most for: large scans, complex joins, heavy aggregations, and ML training.

2.5 The Cloud Services Layer

The Cloud Services layer is the “brain” of Snowflake. Unlike the compute layer which you provision explicitly, Cloud Services is a multi-tenant shared service that runs continuously — you are not charged for it unless it exceeds 10% of your daily warehouse compute credits (which almost never happens in normal usage).

Cloud Services handles:

Function	What It Does
Authentication	Validates credentials (password, key-pair, OAuth, SAML), enforces MFA
Authorization	Checks RBAC privileges before allowing any operation
Infrastructure Management	Provisions/deprovisions warehouse nodes, handles failover
Query Parsing	Parses SQL syntax, validates object references
Query Optimization	Generates an efficient physical execution plan using micro-partition metadata
Metadata Management	Stores and serves all micro-partition metadata, object definitions, statistics
Transaction Management	ACID semantics via snapshot isolation
Result Cache	Stores query results for up to 24 hours

2.6 End-to-End Query Execution

Here is exactly what happens when you run a query:

```
Step 1: Client submits SQL
↓
Step 2: Cloud Services authenticates & authorizes
↓
Step 3: SQL parser generates logical query plan
↓
Step 4: Optimizer uses micro-partition metadata to:
  a) Identify relevant partitions (prune the rest)
  b) Estimate cardinality, choose join order
  c) Generate physical execution plan
↓
Step 5: CHECK RESULT CACHE
  → Hit? Return result immediately. Zero compute. Zero cost.
  → Miss? Continue.
↓
Step 6: Route to virtual warehouse
↓
Step 7: Warehouse checks LOCAL SSD CACHE
  → Hit? Use cached micro-partitions.
  → Miss? Fetch from object storage to local SSD.
↓
Step 8: MPP execution across all warehouse nodes
  (each node processes a subset of micro-partitions in parallel)
↓
Step 9: Results aggregated, sent back to Cloud Services
↓
Step 10: Result stored in Result Cache (24-hour TTL)
↓
Step 11: Result returned to client
```

2.7 ACID Transactions

Snowflake provides full ACID transaction semantics using **snapshot isolation**:

- Each transaction sees a consistent snapshot of the database at the moment the transaction started
- No dirty reads, no phantom reads
- Writers don't block readers; readers don't block writers
- Multi-statement transactions are supported

```

BEGIN;
  UPDATE accounts SET balance = balance - 500 WHERE account_id = 'ACC-001';
  UPDATE accounts SET balance = balance + 500 WHERE account_id = 'ACC-002';
  INSERT INTO transaction_log VALUES ('TXN-9999', 500, CURRENT_TIMESTAMP());
COMMIT;

-- Or rollback on error
BEGIN;
  DELETE FROM staging_orders WHERE load_date = CURRENT_DATE();
  -- Something went wrong...
ROLLBACK;

```

Chapter 2 Summary

Key Takeaway	Detail
Three decoupled layers	Storage, Compute, Cloud Services — each scales independently
Micro-partitions are automatic	50–500MB columnar files with metadata enabling partition pruning
Multiple warehouses, one dataset	ETL, BI, and DS teams can work simultaneously without conflict
Three-layer cache hierarchy	Result Cache → Metadata Cache → Warehouse (SSD) Cache
ACID via snapshot isolation	Writers and readers don't block each other

What's Next: Chapter 3 gets hands-on — setting up your Snowflake environment, connecting via multiple clients, and writing your first queries.

Chapter 3: Getting Started — Environment Setup

Learning Objectives

Create a Snowflake account, navigate Snowsight, use SnowSQL CLI, connect via Python connector and SQLAlchemy, and implement secure credential management patterns.

3.1 Creating a Trial Account

1. Go to `snowflake.com` → “Start for free”
2. Choose your cloud provider and region (pick the region closest to you, or where your data is)
3. Choose “Enterprise” edition (30-day trial, full features)
4. Verify your email

Your account URL will be: `https://<orgname>-<accountname>.snowflakecomputing.com`

The account identifier format (used in connection strings): `<orgname>-<accountname>`

You’ll receive ACCOUNTADMIN credentials. Treat these carefully — ACCOUNTADMIN has unrestricted access to everything.

3.2 Navigating Snowsight

Snowsight is Snowflake’s modern web interface. Key areas:

Section	What's Here
Worksheets	SQL editor with autocomplete, multi-tab, results pane
Dashboards	Charts and tiles built from queries
Notebooks	Python/SQL/Markdown cells (like Jupyter, in Snowflake)
Streamlit	Build and manage Streamlit apps
Data → Databases	Object browser: databases, schemas, tables, views
Data → Marketplace	Browse 2,000+ free and paid datasets
Data → Private Sharing	Manage data shares (provider and consumer)
Projects → Notebooks	Python/SQL notebooks
Activity	Query history, copy history, task history
Admin → Warehouses	Create and manage warehouses
Admin → Users & Roles	User management, role hierarchy
Admin → Resource Monitors	Set credit budgets
Admin → Billing	Usage and cost reports

Pro Tip: In worksheets, use `Cmd+Enter` (Mac) or `Ctrl+Enter` (Windows) to run the current statement. `Cmd+Shift+Enter` runs all statements in the sheet.

3.3 SnowSQL CLI

SnowSQL is the official command-line client. Install it from snowflake.com/developers/downloads.

Configuration file at `~/.snowsql/config` :

```
[connections.my_connection]
accountname = myorg-myaccount
username = myuser
password = mypassword          # or use authenticator = externalbrowser for SSO
dbname = ANALYTICS_DB
schemaname = STAGING
warehousename = TRANSFORM_WH
role = SYSADMIN
```

Common usage patterns:

```
# Interactive session
snowsql -c my_connection

# Execute SQL file non-interactively
snowsql -c my_connection -f ./migrations/V1.0.0__initial_schema.sql

# Inline query
snowsql -c my_connection -q "SELECT CURRENT_VERSION(), CURRENT_USER(),
CURRENT_ROLE()"

# Output as CSV (useful for exports)
snowsql -c my_connection \
  -q "SELECT * FROM marts.fct_orders LIMIT 1000" \
  -o output_format=csv \
  -o header=true \
  > orders_export.csv

# Run with specific role override
snowsql -c my_connection --role ACCOUNTADMIN -q "SHOW USERS"

# Execute and exit on first error
snowsql -c my_connection -f my_script.sql --stop-on-first-error
```

Useful SnowSQL meta-commands (run inside the interactive session):

```
!set output_format=table  -- or csv, json, tsv
!set timing=true          -- show query execution time
!queries                  -- show recent query history
!exit                     -- exit the session
```

3.4 VS Code Extension

Install the **Snowflake** extension by Snowflake Inc. from the VS Code marketplace. It provides:

- Syntax highlighting for Snowflake SQL
- Object explorer (databases, schemas, tables, columns, views)
- Autocomplete for SQL keywords and object names
- Run queries directly and view results in the editor
- Connection management (supports multiple accounts)

After installing, connect by clicking the Snowflake icon in the sidebar → "Add Connection" → enter your account identifier and credentials.

3.5 Python Connector

The official Python connector is the most common way to interact with Snowflake programmatically.

```
pip install snowflake-connector-python
pip install "snowflake-connector-python[pandas]" # adds pandas support
```

Basic connection and query:

```

import snowflake.connector
import os

# Use environment variables — never hardcode credentials
conn = snowflake.connector.connect(
    account=os.environ['SNOWFLAKE_ACCOUNT'],      # e.g., 'myorg-myaccount'
    user=os.environ['SNOWFLAKE_USER'],
    password=os.environ['SNOWFLAKE_PASSWORD'],
    warehouse='ANALYTICS_WH',
    database='ANALYTICS_DB',
    schema='MARTS',
    role='DATA_ANALYST_ROLE',
    session_parameters={
        'QUERY_TAG': 'my_python_script',
        'TIMEZONE': 'America/New_York'
    }
)

cur = conn.cursor()

# Execute and fetch one row
cur.execute("SELECT CURRENT_VERSION(), CURRENT_USER(), CURRENT_ROLE()")
row = cur.fetchone()
print(f"Version: {row[0]}, User: {row[1]}, Role: {row[2]}")

# Fetch all rows as list of tuples
cur.execute("SELECT product_id, product_name, price FROM dim_products LIMIT 10")
rows = cur.fetchall()
for row in rows:
    print(row)

# Fetch as pandas DataFrame (requires [pandas] extra)
cur.execute("SELECT * FROM fct_orders WHERE order_date ≥ '2024-01-01' LIMIT
            10000")
df = cur.fetch_pandas_all()
print(df.describe())

# Parameterized queries (always use params, never f-strings for user input!)
cur.execute(
    "SELECT * FROM customers WHERE country_code = %s AND segment = %s",
    ('US', 'Gold')
)

cur.close()
conn.close()

```

Context manager pattern (recommended):

```

import snowflake.connector
from contextlib import contextmanager

@contextmanager
def get_snowflake_connection():
    conn = snowflake.connector.connect(
        account=os.environ['SNOWFLAKE_ACCOUNT'],
        user=os.environ['SNOWFLAKE_USER'],
        password=os.environ['SNOWFLAKE_PASSWORD'],
        warehouse='ANALYTICS_WH',
        database='ANALYTICS_DB',
        schema='MARTS'
    )
    try:
        yield conn
    finally:
        conn.close()

# Usage
with get_snowflake_connection() as conn:
    with conn.cursor() as cur:
        cur.execute("SELECT COUNT(*) FROM fct_orders")
        print(cur.fetchone()[0])

```

Write a pandas DataFrame to Snowflake:

```

from snowflake.connector.pandas_tools import write_pandas
import pandas as pd

df = pd.DataFrame({
    'customer_id': [1001, 1002, 1003],
    'email': ['alice@example.com', 'bob@example.com', 'carol@example.com'],
    'segment': ['Gold', 'Silver', 'Platinum']
})

with get_snowflake_connection() as conn:
    success, nchunks, nrows, _ = write_pandas(
        conn=conn,
        df=df,
        table_name='NEW_CUSTOMERS',
        schema='STAGING',
        auto_create_table=True,  # creates table if it doesn't exist
        overwrite=False
    )
    print(f"Loaded {nrows} rows in {nchunks} chunks")

```

3.6 SQLAlchemy Integration

For ORM-style access or using pandas `to_sql`:

```
from sqlalchemy import create_engine
import pandas as pd

engine = create_engine(
    "snowflake://{user}:{password}@{account}/{database}/{schema}"
    "?warehouse={warehouse}&role={role}".format(
        user=os.environ['SNOWFLAKE_USER'],
        password=os.environ['SNOWFLAKE_PASSWORD'],
        account=os.environ['SNOWFLAKE_ACCOUNT'],
        database='ANALYTICS_DB',
        schema='MARTS',
        warehouse='ANALYTICS_WH',
        role='DATA_ANALYST_ROLE'
    )
)

# Read
df = pd.read_sql("SELECT * FROM dim_customers LIMIT 1000", engine)

# Write
df.to_sql('my_new_table', engine, if_exists='replace', index=False, chunksize=1000)
```

3.7 Key-Pair Authentication (Service Accounts)

For production service accounts, password auth is discouraged. Use RSA key-pair auth instead:

```
# Generate 2048-bit RSA key pair
openssl genrsa 2048 | openssl pkcs8 -topk8 -inform PEM -out rsa_key.p8 -nocrypt
openssl rsa -in rsa_key.p8 -pubout -out rsa_key.pub

# View your public key (you'll need this for the SQL command)
cat rsa_key.pub
```

```
-- Register public key with the user (run as SECURITYADMIN or USERADMIN)
ALTER USER svc_dbt_user SET RSA_PUBLIC_KEY='MIIBIjANBgkq...'; -- paste your
public key content
```

```

from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives import serialization
import snowflake.connector

# Load private key
with open("/secure/path/rsa_key.p8", "rb") as key_file:
    private_key = serialization.load_pem_private_key(
        key_file.read(),
        password=None,
        backend=default_backend()
    )

pkb = private_key.private_bytes(
    encoding=serialization.Encoding.DER,
    format=serialization.PrivateFormat.TraditionalOpenSSL,
    encryption_algorithm=serialization.NoEncryption()
)

conn = snowflake.connector.connect(
    account=os.environ['SNOWFLAKE_ACCOUNT'],
    user='svc_dbt_user',
    private_key=pkb,
    warehouse='TRANSFORM_WH',
    database='ANALYTICS_DB',
    role='DBT_ROLE'
)

```

Chapter 3 Summary

Key Takeaway	Detail
Multiple connection methods	Web UI (Snowsight), SnowSQL CLI, Python connector, VS Code extension
Never hardcode credentials	Use environment variables or a secrets manager
Key-pair auth for service accounts	More secure than password; required for some enterprise setups
Parameterized queries	Always use <code>%s</code> placeholders; never f-string user input into SQL
Context managers	Use <code>with</code> blocks to ensure connections are properly closed

What's Next: Chapter 4 dives into the full object model — every type of object you can create in Snowflake, from tables to Dynamic Tables to Streams and Tasks.

Chapter 4: Database Objects & Data Modeling

Learning Objectives

Understand the complete Snowflake object hierarchy, create and manage all major object types, design appropriate table types for different use cases, and understand Dynamic Tables and Streams for pipeline construction.

4.1 The Object Hierarchy

```

Organization
├── Account(s)
│   ├── Database
│   │   └── Schema
│   │       ├── Table (Permanent / Temporary / Transient / External / Hybrid /
Iceberg)
│   │       ├── View (Standard / Secure / Materialized)
│   │       ├── Dynamic Table
│   │       ├── Stage (Internal Named / External)
│   │       ├── File Format
│   │       ├── Sequence
│   │       ├── Stream
│   │       ├── Task
│   │       ├── Pipe
│   │       ├── Function (UDF / UDTF)
│   │       ├── Procedure
│   │       ├── Alert
│   │       └── Event Table
│   ├── Warehouse
│   ├── Role
│   ├── User
│   ├── Network Policy
│   ├── Resource Monitor
│   ├── Integration (Storage / Notification / Security)
│   └── Share

```

4.2 Databases and Schemas

```
-- Create databases
CREATE DATABASE IF NOT EXISTS ANALYTICS_DB
  DATA_RETENTION_TIME_IN_DAYS = 30
  COMMENT = 'Main analytics database';

CREATE DATABASE IF NOT EXISTS DEV_ANALYTICS_DB
  DATA_RETENTION_TIME_IN_DAYS = 7
  COMMENT = 'Development environment - clone of prod';

-- Create schemas (logical namespaces within a database)
CREATE SCHEMA IF NOT EXISTS ANALYTICS_DB.RAW
  DATA_RETENTION_TIME_IN_DAYS = 7
  COMMENT = 'Raw ingested data - untransformed';

CREATE SCHEMA IF NOT EXISTS ANALYTICS_DB.STAGING
  DATA_RETENTION_TIME_IN_DAYS = 14
  COMMENT = 'Cleaned and validated data';

CREATE SCHEMA IF NOT EXISTS ANALYTICS_DB.MARTS
  DATA_RETENTION_TIME_IN_DAYS = 30
  COMMENT = 'Business-ready dimensional model';

CREATE SCHEMA IF NOT EXISTS ANALYTICS_DB.ML_FEATURES
  DATA_RETENTION_TIME_IN_DAYS = 14;

CREATE SCHEMA IF NOT EXISTS ANALYTICS_DB.GOVERNANCE
  COMMENT = 'Governance tables: tag mapping, masking policy config';

-- Set context
USE DATABASE ANALYTICS_DB;
USE SCHEMA MARTS;
USE WAREHOUSE ANALYTICS_WH;
USE ROLE DATA_ENGINEER_ROLE;

-- Explore
SHOW DATABASES;
SHOW SCHEMAS IN DATABASE ANALYTICS_DB;
SHOW TABLES IN SCHEMA ANALYTICS_DB.MARTS;
```

4.3 Table Types

Permanent Tables — full Time Travel + Fail-Safe, the default:

```

CREATE TABLE customers (
  customer_id    NUMBER(38,0)    NOT NULL,
  email          VARCHAR(255)    NOT NULL,
  full_name      VARCHAR(200),
  phone          VARCHAR(20),
  country_code   CHAR(2),
  segment        VARCHAR(20),
  created_at     TIMESTAMP_NTZ    DEFAULT CURRENT_TIMESTAMP(),
  updated_at     TIMESTAMP_NTZ    DEFAULT CURRENT_TIMESTAMP(),
  is_active      BOOLEAN          DEFAULT TRUE,
  attributes     VARIANT,
  CONSTRAINT pk_customers PRIMARY KEY (customer_id)
);

-- Informational constraints (Snowflake does not enforce FK constraints, but they
-- document intent)
ALTER TABLE orders ADD CONSTRAINT fk_orders_customers
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
  NOT ENFORCED; -- always required — Snowflake constraints are for
                -- documentation only

```

Warning: Snowflake primary key, foreign key, and unique constraints are **NOT enforced**. They are informational only — useful for query optimization hints and for tools like dbt that read metadata. Do not rely on them for data quality; enforce uniqueness in your ETL logic.

Transient Tables — Time Travel only, no Fail-Safe, ~50% less storage cost:

```

-- Use for intermediate staging data you don't need to recover after the Time
-- Travel window
CREATE TRANSIENT TABLE staging.order_staging (
  order_id      VARCHAR(36),
  raw_json      VARIANT,
  loaded_at     TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

-- Transient schema (all tables in schema become transient)
CREATE TRANSIENT SCHEMA ANALYTICS_DB.SCRATCH;

```

Temporary Tables — session-scoped, auto-dropped at session end, no Time Travel:

```
-- Use for intermediate results within a script or session
CREATE TEMPORARY TABLE session_customer_metrics AS
SELECT
    customer_id,
    COUNT(*) AS order_count,
    SUM(amount) AS lifetime_value
FROM orders
GROUP BY customer_id;

-- This table disappears when your session ends
```

Choosing the right table type:

Type	Time Travel	Fail-Safe	Storage Cost	Use When
Permanent	Yes (1-90 days)	Yes (7 days)	Full	Production tables, historical data
Transient	Yes (0-1 day)	No	~50% less	Staging, ELT intermediates, large scratch data
Temporary	No	No	Minimal	Session-level temp results, CTEs too complex for inline

4.4 Data Types

```

-- Numeric
NUMBER(38,0)      -- Integer (aliases: INT, INTEGER, BIGINT, SMALLINT)
NUMBER(18,4)      -- Fixed-point decimal (alias: DECIMAL, NUMERIC)
FLOAT             -- IEEE 64-bit floating point (aliases: REAL, DOUBLE)

-- String
VARCHAR(16777216) -- Variable-length, max 16MB (aliases: STRING, TEXT,
  NVARCHAR)
CHAR(10)          -- Fixed-length

-- Date/Time (ALWAYS prefer TIMESTAMP_NTZ for most use cases)
DATE              -- Calendar date only: 2024-01-15
TIME              -- Time only: 14:30:00.000
TIMESTAMP_NTZ    -- Timestamp without timezone (stored as-is, recommended)
TIMESTAMP_LTZ    -- Timestamp with local timezone (uses account timezone)
TIMESTAMP_TZ     -- Timestamp with stored timezone offset

-- Boolean
BOOLEAN          -- TRUE / FALSE / NULL

-- Semi-structured (the killer feature)
VARIANT           -- Any JSON, XML, Avro value up to 16MB
OBJECT            -- JSON object with string keys
ARRAY             -- Ordered list of VARIANT values

-- Binary
BINARY / VARBINARY -- Raw bytes

```

Pro Tip: Use `TIMESTAMP_NTZ` for almost everything. Store all timestamps in UTC and handle timezone conversions in your BI tool or presentation layer. `TIMESTAMP_LTZ` can cause surprising behavior when your account timezone changes.

4.5 Views

```

-- Standard view: query is visible to all users who can access it
CREATE OR REPLACE VIEW marts.v_active_customers AS
SELECT customer_id, email, full_name, country_code, segment, created_at
FROM customers
WHERE is_active = TRUE;

-- Secure view: query definition is hidden from non-owners
-- Use for views that expose sensitive data with masking logic
CREATE OR REPLACE SECURE VIEW marts.v_customer_summary AS
SELECT
    customer_id,
    REGEXP_REPLACE(email, '.*@', '****@') AS masked_email,
    full_name,
    country_code,
    segment
FROM customers;

-- Materialized view: pre-computed result, auto-refreshed (Enterprise+)
-- Best for: expensive aggregations that power dashboards
CREATE OR REPLACE MATERIALIZED VIEW marts.mv_monthly_revenue AS
SELECT
    DATE_TRUNC('MONTH', order_date) AS revenue_month,
    region,
    SUM(amount) AS total_revenue,
    COUNT(DISTINCT order_id) AS order_count,
    COUNT(DISTINCT customer_id) AS unique_customers
FROM orders
WHERE status = 'COMPLETED'
GROUP BY 1, 2;
-- Snowflake maintains this as data in the base table changes

```

4.6 Stages

Stages are named references to storage locations where data files live before loading:

```
-- Internal named stage (Snowflake-managed storage)
CREATE STAGE analytics_db.public.internal_stage
  DIRECTORY = (ENABLE = TRUE)
  COMMENT = 'General purpose internal stage for file uploads';

-- Upload file (from SnowSQL or Python)
-- PUT file:///local/path/orders.csv @internal_stage/orders/ AUTO_COMPRESS=TRUE;
LIST @internal_stage;

-- External S3 stage (using storage integration – recommended over credentials)
CREATE STORAGE INTEGRATION s3_integration
  TYPE = EXTERNAL_STAGE
  STORAGE_PROVIDER = 'S3'
  ENABLED = TRUE
  STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::123456789012:role/snowflake-s3-role'
  STORAGE_ALLOWED_LOCATIONS = ('s3://my-data-lake/', 's3://my-archive/');

-- Get the IAM values to configure trust in AWS
DESC INTEGRATION s3_integration;

CREATE STAGE analytics_db.raw.s3_orders_stage
  URL = 's3://my-data-lake/orders/'
  STORAGE_INTEGRATION = s3_integration
  FILE_FORMAT = (TYPE = PARQUET)
  COMMENT = 'Raw orders from the data lake';
```

4.7 Dynamic Tables

Dynamic Tables are the modern replacement for complex Streams + Tasks pipelines. You declare what the table should contain (a query), and Snowflake handles when to refresh it automatically:

```

-- Silver: clean raw orders (auto-refreshes when raw.orders changes)
CREATE OR REPLACE DYNAMIC TABLE staging.silver_orders
  TARGET_LAG = '5 minutes'      -- keep data at most 5 minutes stale
  WAREHOUSE = TRANSFORM_WH
  COMMENT = 'Cleaned orders from raw layer'
AS
SELECT
  raw_payload:order_id::VARCHAR(36)      AS order_id,
  raw_payload:customer_id::NUMBER        AS customer_id,
  TO_DATE(raw_payload:order_date::VARCHAR) AS order_date,
  UPPER(TRIM(raw_payload:status::VARCHAR)) AS status,
  ROUND(raw_payload:amount::FLOAT, 2)     AS amount,
  raw_payload:region::VARCHAR             AS region,
  _loaded_at
FROM raw.orders
WHERE raw_payload:order_id IS NOT NULL
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY raw_payload:order_id::VARCHAR
  ORDER BY _loaded_at DESC
) = 1; -- deduplicate

-- Gold: aggregate (depends on silver, auto-chains)
CREATE OR REPLACE DYNAMIC TABLE marts.gold_revenue_by_day
  TARGET_LAG = '1 hour'
  WAREHOUSE = ANALYTICS_WH
AS
SELECT
  order_date,
  region,
  COUNT(DISTINCT order_id)      AS orders,
  COUNT(DISTINCT customer_id)   AS customers,
  SUM(amount)                   AS revenue
FROM staging.silver_orders
WHERE status = 'COMPLETED'
GROUP BY 1, 2;

-- Monitor
SELECT name, target_lag, scheduling_state, last_completed_refresh,
       last_suspended_on
FROM information_schema.dynamic_tables;

-- Manual refresh
ALTER DYNAMIC TABLE marts.gold_revenue_by_day REFRESH;

-- Suspend/resume
ALTER DYNAMIC TABLE staging.silver_orders SUSPEND;
ALTER DYNAMIC TABLE staging.silver_orders RESUME;

```

4.8 Streams — Change Data Capture

Streams track DML changes (inserts, updates, deletes) to a table without modifying the source:

```
-- Create stream on orders table
CREATE STREAM raw.orders_stream ON TABLE raw.orders
  APPEND_ONLY = FALSE; -- captures INSERT + UPDATE + DELETE

-- Append-only stream (more efficient; for tables that only ever get new rows)
CREATE STREAM raw.events_stream ON TABLE raw.events
  APPEND_ONLY = TRUE;

-- Query a stream — three metadata columns are added
SELECT
  order_id,
  customer_id,
  amount,
  status,
  METADATA$ACTION,      -- 'INSERT' or 'DELETE'
  METADATA$ISUPDATE,    -- TRUE when this row is part of an UPDATE (one DELETE
                        + one INSERT)
  METADATA$ROW_ID       -- internal row identifier
FROM raw.orders_stream;

-- Consume stream in a DML — advances the offset
INSERT INTO staging.silver_orders
SELECT order_id, customer_id, amount, status
FROM raw.orders_stream
WHERE METADATA$ACTION = 'INSERT' AND METADATA$ISUPDATE = FALSE;

-- Check if stream has data (used in Tasks)
SELECT SYSTEM$STREAM_HAS_DATA('raw.orders_stream');
```

4.9 Tasks

Tasks schedule SQL statements or Snowpark stored procedures:

```

-- Simple scheduled task
CREATE TASK staging.refresh_daily_summary
  WAREHOUSE = TRANSFORM_WH
  SCHEDULE = 'USING CRON 0 6 * * * America/New_York' -- daily at 6am ET
AS
INSERT INTO staging.daily_summary
SELECT DATE_TRUNC('DAY', order_date) AS day, SUM(amount), COUNT(*)
FROM raw.orders
WHERE order_date = CURRENT_DATE() - 1;

-- Task DAG: parent triggers child
CREATE TASK staging.check_for_new_orders
  WAREHOUSE = TRANSFORM_WH
  SCHEDULE = '2 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('raw.orders_stream') -- only run if stream has
    data
AS SELECT 1; -- no-op; just triggers children

CREATE TASK staging.process_new_orders
  WAREHOUSE = TRANSFORM_WH
  AFTER staging.check_for_new_orders -- runs after parent completes
AS
MERGE INTO staging.silver_orders t
USING raw.orders_stream s ON t.order_id = s.order_id
WHEN MATCHED AND s.METADATA$ACTION = 'INSERT' THEN
  UPDATE SET status = s.status, amount = s.amount
WHEN NOT MATCHED AND s.METADATA$ACTION = 'INSERT' THEN
  INSERT (order_id, customer_id, amount, status) VALUES (s.order_id,
    s.customer_id, s.amount, s.status);

-- MUST resume tasks explicitly (they start suspended)
ALTER TASK staging.process_new_orders RESUME;
ALTER TASK staging.check_for_new_orders RESUME;

-- Check task run history
SELECT name, state, scheduled_time, completed_time, error_message
FROM TABLE(information_schema.task_history(
  SCHEDULED_TIME_RANGE_START => DATEADD(HOUR, -24, CURRENT_TIMESTAMP()),
  TASK_NAME => 'PROCESS_NEW_ORDERS'
))
ORDER BY scheduled_time DESC;

```

Chapter 4 Summary

Key Takeaway	Detail
Choose table type deliberately	Permanent (prod), Transient (staging), Temporary (session)
Constraints are informational	Snowflake does not enforce PK/FK/UNIQUE — enforce in ETL
VARIANT handles any JSON	Native semi-structured support without schema-on-write
Dynamic Tables replace Stream+Task for most pipelines	Declare the query; Snowflake handles refresh scheduling
Streams track changes	Lightweight CDC without touching source tables

What's Next: Chapter 5 covers loading data — bulk loading with COPY INTO, continuous ingestion with Snowpipe, and streaming via the Kafka connector.

Chapter 5: Loading Data into Snowflake

Learning Objectives

Load data using COPY INTO from internal and external stages, handle semi-structured data formats, set up Snowpipe for continuous ingestion, use the Kafka connector, and handle load errors gracefully.

5.1 Data Loading Pattern Overview

Pattern	Latency	Frequency	Best For
COPY INTO	Minutes–hours	Scheduled batch	Large historical loads, nightly ETL
Snowpipe	Seconds–minutes	Event-driven	Files dropped to S3/ Azure/GCS continuously
Kafka Connector	Seconds	Continuous	Kafka topic consumers
Streaming Ingest API	Near-real-time	Continuous	High-frequency event streams
External Tables	Query-time	N/A	Query data lake without loading

5.2 Bulk Loading with COPY INTO

Step 1: Create a file format:

```
CREATE FILE FORMAT csv_ff
  TYPE = CSV
  FIELD_DELIMITER = ','
  RECORD_DELIMITER = '\n'
  SKIP_HEADER = 1
  NULL_IF = ('NULL', 'null', '', 'N/A')
  EMPTY_FIELD_AS_NULL = TRUE
  TRIM_SPACE = TRUE
  COMPRESSION = AUTO;

CREATE FILE FORMAT json_ff
  TYPE = JSON
  STRIP_OUTER_ARRAY = TRUE
  STRIP_NULL_VALUES = FALSE
  COMPRESSION = AUTO;

CREATE FILE FORMAT parquet_ff
  TYPE = PARQUET
  SNAPPY_COMPRESSION = TRUE;
```

Step 2: Stage your files and load:

```

-- Upload from local machine (run in SnowSQL)
-- PUT file:///Users/me/data/customers_2024.csv @internal_stage/customers/
  AUTO_COMPRESS=TRUE;

-- Load from internal stage
COPY INTO raw.customers
FROM @internal_stage/customers/
FILE_FORMAT = (FORMAT_NAME = 'csv_ff')
ON_ERROR = 'CONTINUE'
PURGE = FALSE;

-- Load from external S3 stage with pattern matching
COPY INTO raw.orders
FROM @s3_orders_stage/2024/01/
FILE_FORMAT = (FORMAT_NAME = 'parquet_ff')
PATTERN = '.*orders_2024_01.*\.parquet'
MATCH_BY_COLUMN_NAME = CASE_INSENSITIVE; -- auto-map Parquet columns to table
columns

-- Transform during load (column selection and type casting)
COPY INTO raw.events (event_id, event_type, user_id, occurred_at, properties)
FROM (
  SELECT
    $1::VARCHAR(36), -- column 1 as event_id
    LOWER($2::VARCHAR), -- lowercase event_type
    $3::NUMBER, -- user_id
    TO_TIMESTAMP_NTZ($4, 'YYYY-MM-DD"THH24:MI:SS'), -- parse ISO timestamp
    PARSE_JSON($5) -- parse JSON string to VARIANT
  FROM @internal_stage/events/
)
FILE_FORMAT = (TYPE = CSV SKIP_HEADER = 1)
ON_ERROR = 'SKIP_FILE';

```

ON_ERROR options: - `ABORT_STATEMENT` : roll back entire load on first error (default) - `CONTINUE` : skip bad rows, continue loading - `SKIP_FILE` : skip files with any error; load clean files - `SKIP_FILE_<n>` : skip file if more than `n` errors - `SKIP_FILE_<n>%` : skip file if more than `n%` of rows have errors

Validate without loading:

```
-- Dry run: see what errors exist without committing any rows
COPY INTO raw.customers
FROM @internal_stage/customers/
FILE_FORMAT = (FORMAT_NAME = 'csv_ff')
VALIDATION_MODE = RETURN_ERRORS;

-- Returns all rows that WOULD have errored
COPY INTO raw.customers
FROM @internal_stage/customers/
FILE_FORMAT = (FORMAT_NAME = 'csv_ff')
VALIDATION_MODE = RETURN_ALL_ERRORS;
```

Verify loads:

```
SELECT
    file_name,
    row_count,
    rows_loaded,
    rows_parsed - rows_loaded AS rows_rejected,
    first_error,
    status,
    last_load_time
FROM TABLE(information_schema.copy_history(
    TABLE_NAME => 'CUSTOMERS',
    START_TIME => DATEADD(HOUR, -24, CURRENT_TIMESTAMP())
))
ORDER BY last_load_time DESC;
```

5.3 Loading Semi-Structured Data

```
-- Load raw JSON (array of objects)
CREATE TABLE raw.events (
  data          VARIANT,
  loaded_at     TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

COPY INTO raw.events (data)
FROM @internal_stage/events/
FILE_FORMAT = (TYPE = JSON STRIP_OUTER_ARRAY = TRUE);

-- Query with dot notation
SELECT
  data:event_id::VARCHAR      AS event_id,
  data:event_type::VARCHAR    AS event_type,
  data:user_id::NUMBER        AS user_id,
  data:timestamp::TIMESTAMP_NTZ AS event_time,
  data:properties:page::VARCHAR AS page
FROM raw.events
WHERE data:event_type = 'page_view'
LIMIT 100;

-- Flatten nested arrays
SELECT
  data:order_id::NUMBER      AS order_id,
  item.value:product_id::VARCHAR AS product_id,
  item.value:quantity::NUMBER AS quantity,
  item.value:price::FLOAT    AS price
FROM raw.orders_json,
     LATERAL FLATTEN(INPUT => data:line_items) item;
```

5.4 Snowpipe — Continuous Ingestion

Snowpipe enables automatic loading when files arrive in a stage:

```
-- Create pipe referencing COPY INTO statement
CREATE PIPE raw.orders_pipe
  AUTO_INGEST = TRUE      -- uses SQS notification (AWS) or Event Grid (Azure)
  COMMENT = 'Auto-ingest orders from S3 as files arrive'
AS
COPY INTO raw.orders
FROM @s3_orders_stage/
FILE_FORMAT = (FORMAT_NAME = 'parquet_ff')
MATCH_BY_COLUMN_NAME = CASE_INSENSITIVE;

-- Get the SQS ARN for configuring S3 bucket notifications
SHOW PIPES;
-- Copy the 'notification_channel' value → configure S3 Event Notification → SQS

-- Check pipe status
SELECT SYSTEM$PIPE_STATUS('raw.orders_pipe');

-- Monitor load activity
SELECT *
FROM TABLE(information_schema.pipe_usage_history(
  DATE_RANGE_START ⇒ DATEADD(DAY, -1, CURRENT_TIMESTAMP()),
  PIPE_NAME ⇒ 'RAW.ORDERS_PIPE'
));

-- Manually refresh (re-queue files already in stage)
ALTER PIPE raw.orders_pipe REFRESH PREFIX='2024/01/15/';

-- Pause/resume
ALTER PIPE raw.orders_pipe SET PIPE_EXECUTION_PAUSED = TRUE;
ALTER PIPE raw.orders_pipe SET PIPE_EXECUTION_PAUSED = FALSE;
```

5.5 Kafka Connector Configuration

```
# kafka-connect-snowflake.properties
name=snowflake-orders-connector
connector.class=com.snowflake.kafka.connector.SnowflakeSinkConnector
tasks.max=4
topics=orders.raw,customers.raw,events.raw
snowflake.url.name=myorg-myaccount.snowflakecomputing.com
snowflake.user.name=kafka_svc_user
snowflake.private.key=<base64-encoded-pkcs8-private-key>
snowflake.database.name=ANALYTICS_DB
snowflake.schema.name=RAW
snowflake.topic2table.map=orders.raw:raw_orders,customers.raw:raw_customers,events.raw:raw_events
buffer.count.records=10000
buffer.flush.time=60
buffer.size.bytes=5000000
snowflake.ingestion.method=SNOWPIPE_STREAMING
key.converter=org.apache.kafka.connect.storage.StringConverter
value.converter=org.apache.kafka.connect.json.JsonConverter
value.converter.schemas.enable=false
```

Chapter 5 Summary

Key Takeaway	Detail
COPY INTO for batch	Stage files first (internal or external), then COPY — supports CSV, JSON, Parquet, Avro, ORC
Snowpipe for continuous	Event-notification-driven; zero maintenance after setup
Always validate first	<code>VALIDATION_MODE = RETURN_ERRORS</code> before committing large loads
Semi-structured is native	Load JSON as VARIANT; query with dot notation; no pre-flattening needed
Kafka connector is production-grade	Supports SNOWPIPE_STREAMING for near-real-time ingestion

What's Next: Chapter 6 goes deep on Snowflake SQL — from window functions and QUALIFY to semi-structured queries, MERGE, recursive CTEs, and pattern matching.

Chapter 6: SQL in Snowflake

Learning Objectives

Write advanced Snowflake SQL including window functions, QUALIFY, semi-structured queries, PIVOT/UNPIVOT, MERGE, recursive CTEs, MATCH_RECOGNIZE, and ASOF JOIN.

6.1 Standard SQL + Snowflake Extensions

Snowflake supports full ANSI SQL plus powerful extensions including QUALIFY, ASOF JOIN, MATCH_RECOGNIZE, FLATTEN, and more.

6.2 Window Functions

```

-- ROW_NUMBER: unique sequential rank per partition
SELECT
  customer_id, order_id, order_date, amount,
  ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) AS
    recency_rank
FROM orders;

-- RANK (with gaps) vs DENSE_RANK (no gaps)
SELECT
  product_name, revenue,
  RANK() OVER (ORDER BY revenue DESC) AS rank_with_gaps,
  DENSE_RANK() OVER (ORDER BY revenue DESC) AS rank_no_gaps
FROM product_revenue;

-- LAG/LEAD: access adjacent rows
SELECT
  order_date, daily_revenue,
  LAG(daily_revenue, 1) OVER (ORDER BY order_date) AS prev_day,
  LEAD(daily_revenue, 1) OVER (ORDER BY order_date) AS next_day,
  daily_revenue - LAG(daily_revenue, 1) OVER (ORDER BY order_date) AS
    day_over_day
FROM daily_revenue_summary;

-- Running total + 7-day moving average
SELECT
  order_date,
  daily_revenue,
  SUM(daily_revenue) OVER (
    ORDER BY order_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS cumulative_revenue,
  ROUND(AVG(daily_revenue) OVER (
    ORDER BY order_date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
  ), 2) AS rolling_7d_avg
FROM daily_revenue_summary;

-- NTILE: divide into N equal buckets
SELECT
  customer_id, lifetime_value,
  NTILE(4) OVER (ORDER BY lifetime_value DESC) AS revenue_quartile
FROM customer_ltv;

-- FIRST_VALUE / LAST_VALUE
SELECT
  customer_id, order_id, amount,
  FIRST_VALUE(amount) OVER (
    PARTITION BY customer_id ORDER BY order_date
  ) AS first_order_amount,
  LAST_VALUE(amount) OVER (
    PARTITION BY customer_id ORDER BY order_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
  ) AS last_order_amount
FROM orders;

```

```
    ) AS latest_order_amount
FROM orders;
```

6.3 QUALIFY — Filtering Window Function Results

QUALIFY is a Snowflake-specific clause that filters on window function results without a subquery:

```
-- De-duplicate: keep only the most recent record per order
-- Old way (subquery):
SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY updated_at DESC)
        AS rn
    FROM raw.orders
) WHERE rn = 1;

-- New way with QUALIFY (cleaner, often faster):
SELECT order_id, customer_id, amount, status, updated_at
FROM raw.orders
QUALIFY ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY updated_at DESC) = 1;

-- Top 3 products per category
SELECT category, product_name, revenue
FROM product_sales
QUALIFY RANK() OVER (PARTITION BY category ORDER BY revenue DESC) ≤ 3;

-- Customers with more than median lifetime value
SELECT customer_id, lifetime_value
FROM customer_ltv
QUALIFY lifetime_value > PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY
    lifetime_value
    OVER ());
```

6.4 Querying Semi-Structured Data

```

-- Sample VARIANT column: raw_data contains {"event_type": "purchase", "user":
  {"id": 42, "tier": "gold"}, "items": [...]}

-- Dot notation (preferred)
SELECT raw_data:event_type::VARCHAR          AS event_type,
       raw_data:user.id::NUMBER              AS user_id,
       raw_data:user.tier::VARCHAR           AS user_tier
FROM raw.events;

-- Bracket notation (useful for keys with special characters or dynamic key
  names)
SELECT raw_data['event_type']::VARCHAR        AS event_type,
       raw_data['user']['id']::NUMBER        AS user_id
FROM raw.events;

-- TYPEOF: inspect the runtime type of a VARIANT value
SELECT TYPEOF(raw_data:amount),              -- 'integer', 'real', 'text', 'boolean',
       'null', 'array', 'object'
       TYPEOF(raw_data:items)
FROM raw.orders_json
LIMIT 5;

-- TRY_CAST: safe type conversion (returns NULL instead of error)
SELECT TRY_CAST(raw_data:amount::VARCHAR AS FLOAT) AS safe_amount
FROM raw.events;

-- IS_OBJECT, IS_ARRAY, IS_NULL_VALUE checks
SELECT * FROM raw.events
WHERE IS_ARRAY(raw_data:items) = TRUE;

-- FLATTEN: unnest an array into rows
SELECT
  e.raw_data:order_id::NUMBER    AS order_id,
  f.index                       AS item_index,
  f.value:product_id::VARCHAR    AS product_id,
  f.value:name::VARCHAR          AS product_name,
  f.value:quantity::NUMBER       AS quantity,
  f.value:unit_price::FLOAT      AS unit_price
FROM raw.events e,
     LATERAL FLATTEN(INPUT => e.raw_data:items) f
WHERE e.raw_data:event_type = 'order_placed';

-- Multi-level flatten
SELECT
  session_id,
  page.value:url::VARCHAR        AS page_url,
  click.value:element::VARCHAR   AS clicked_element
FROM raw.sessions,
     LATERAL FLATTEN(INPUT => data:pages) page,
     LATERAL FLATTEN(INPUT => page.value:clicks) click;

```

6.5 PIVOT and UNPIVOT

```

-- PIVOT: turn rows into columns
-- Source: (product_category, year, revenue) → (product_category, rev_2022,
-- rev_2023, rev_2024)
SELECT *
FROM (
    SELECT product_category, YEAR(order_date) AS yr, amount
    FROM orders
    WHERE status = 'COMPLETED'
)
PIVOT (SUM(amount) FOR yr IN (2022, 2023, 2024))
AS pvt (product_category, rev_2022, rev_2023, rev_2024);

-- UNPIVOT: turn columns into rows
-- Source table has: product, q1_rev, q2_rev, q3_rev, q4_rev
SELECT product, quarter, revenue
FROM quarterly_product_revenue
UNPIVOT (revenue FOR quarter IN (q1_rev, q2_rev, q3_rev, q4_rev));

```

6.6 MERGE — Upsert Pattern

```

-- Full upsert with delete support
MERGE INTO marts.dim_customers AS tgt
USING staging.stg_customers AS src
    ON tgt.customer_id = src.customer_id
WHEN MATCHED AND src.updated_at > tgt.updated_at THEN
    UPDATE SET
        email      = src.email,
        full_name   = src.full_name,
        segment     = src.segment,
        updated_at  = src.updated_at
WHEN MATCHED AND src.is_deleted = TRUE THEN
    DELETE
WHEN NOT MATCHED AND src.is_deleted = FALSE THEN
    INSERT (customer_id, email, full_name, segment, created_at, updated_at)
    VALUES (src.customer_id, src.email, src.full_name, src.segment,
        CURRENT_TIMESTAMP(), CURRENT_TIMESTAMP());

```

6.7 Recursive CTEs

```
-- Traverse an employee org chart
WITH RECURSIVE org AS (
  -- Anchor: CEO (no manager)
  SELECT employee_id, name, manager_id, 0 AS depth, name AS path
  FROM employees
  WHERE manager_id IS NULL

  UNION ALL

  -- Recursive: direct reports
  SELECT e.employee_id, e.name, e.manager_id, o.depth + 1,
         o.path || ' > ' || e.name
  FROM employees e
  JOIN org o ON e.manager_id = o.employee_id
)
SELECT employee_id, name, depth, path
FROM org
ORDER BY path;
```

6.8 Advanced SQL Patterns

```

-- ASOF JOIN: join to the nearest prior record (great for time-series)
SELECT
  t.trade_id, t.symbol, t.trade_time, t.shares,
  p.bid_price,
  t.shares * p.bid_price AS trade_value
FROM trades t
ASOF JOIN tick_prices p
  MATCH_CONDITION (t.trade_time ≥ p.tick_time)
  ON t.symbol = p.symbol;

-- MATCH_RECOGNIZE: detect sequences of events (SQL pattern matching)
SELECT *
FROM web_sessions
MATCH_RECOGNIZE (
  PARTITION BY user_id
  ORDER BY event_time
  MEASURES
    FIRST(event_time) AS funnel_start,
    LAST(event_time) AS funnel_end
  PATTERN (visit add_cart+ checkout purchase)
  DEFINE
    visit      AS event_type = 'page_view',
    add_cart   AS event_type = 'add_to_cart',
    checkout   AS event_type = 'checkout_start',
    purchase   AS event_type = 'purchase_complete'
);

-- Multi-table INSERT (fan-out from one source to multiple targets)
INSERT ALL
  WHEN event_type = 'purchase' THEN INTO events_purchases
  WHEN event_type = 'page_view' THEN INTO events_pageviews
  WHEN event_type = 'click' THEN INTO events_clicks
  ELSE INTO events_other
SELECT event_type, user_id, event_time, properties
FROM staging.raw_events_stream;

```

Chapter 6 Summary

Key Takeaway	Detail
QUALIFY replaces subqueries	Filter on window function results cleanly and efficiently
VARIANT is fully queryable	Dot notation, FLATTEN, TYPEOF, TRY_CAST for any JSON structure
MERGE handles upserts and deletes	Single statement for complex CDC application logic
ASOF JOIN for time-series	Join events to the nearest price/rate/state without complex correlated subqueries
Recursive CTEs for hierarchies	Traverse org charts, product hierarchies, bill-of-materials

What's Next: Chapter 7 covers performance — warehouse sizing, the three-layer cache system, clustering keys, query profiling, and how to make your queries run faster.

End of Part 1 — Continue with Part 2 for Chapters 7–12 # Snowflake Master Course — Part 2: Performance, Security & Advanced Features

Part 2 covers Chapters 7–12. By the end of this section you will understand how to tune warehouse performance, lock down data with enterprise-grade security, protect and restore data through Time Travel and cloning, share live datasets across organizational boundaries, write Python code that executes inside Snowflake via Snowpark, and build production-ready data apps with Streamlit in Snowflake.

Chapter 7: Virtual Warehouses & Performance Optimization

7.1 Virtual Warehouse Deep Dive

A Snowflake virtual warehouse is a named, resizable cluster of compute nodes (EC2 on AWS, Standard VMs on Azure, Compute Engine on GCP) that executes SQL queries, loads data, and runs Snowpark code. Warehouses are entirely separate from storage — you can have zero warehouses running while your data sits safely in cloud object storage.

Every warehouse size doubles the number of nodes and the per-hour credit consumption:

Size	Credits/ hr	Approximate Nodes	Primary Use Case
X-Small (XS)	1	1	Development, light ad-hoc queries
Small (S)	2	2	Small BI dashboards, simple reports
Medium (M)	4	4	General-purpose analytics
Large (L)	8	8	Complex analytics, moderate data loading
X-Large (XL)	16	16	Heavy analytical workloads
2X-Large (2XL)	32	32	Large-scale dbt transformations
3X-Large (3XL)	64	64	Very large ETL jobs
4X-Large (4XL)	128	128	Massive parallel processing
5X-Large (5XL)	256	256	Extreme batch workloads
6X-Large (6XL)	512	512	Maximum compute (rare, specialized)

Key insight: making a warehouse larger does **not** automatically make every query faster. Simple queries that touch small amounts of data run equally fast on XS and XL. Larger sizes help when queries are highly parallelizable (large joins, heavy aggregations, wide table scans) or when you are loading/transforming large volumes of data.

```

-- Create a well-configured analytics warehouse
CREATE WAREHOUSE analytics_wh
  WAREHOUSE_SIZE      = 'LARGE'
  AUTO_SUSPEND        = 60          -- suspend after 60 seconds of
    inactivity
  AUTO_RESUME         = TRUE        -- automatically resume when a query
    arrives
  MIN_CLUSTER_COUNT   = 1          -- minimum clusters (Enterprise+
    feature)
  MAX_CLUSTER_COUNT   = 3          -- scale out to 3 clusters under
    concurrent load
  SCALING_POLICY       = 'ECONOMY'  -- ECONOMY or STANDARD (see 7.2)
  INITIALLY_SUSPENDED = TRUE        -- don't start billing immediately
  COMMENT             = 'Analytics team primary warehouse';

-- Resize on the fly – no data movement, takes effect immediately
ALTER WAREHOUSE analytics_wh SET WAREHOUSE_SIZE = 'X-LARGE';

-- Suspend and resume manually
ALTER WAREHOUSE analytics_wh SUSPEND;
ALTER WAREHOUSE analytics_wh RESUME;

-- List all warehouses with current state
SHOW WAREHOUSES;

-- Monitor warehouse credit consumption over the last 30 days
SELECT
  warehouse_name,
  SUM(credits_used)          AS total_credits,
  SUM(credits_used_compute) AS compute_credits,
  SUM(credits_used_cloud_services) AS cloud_services_credits,
  COUNT(*)                  AS sessions,
  MIN(start_time)           AS earliest_use,
  MAX(end_time)             AS latest_use
FROM snowflake.account_usage.warehouse_metering_history
WHERE start_time ≥ DATEADD(DAY, -30, CURRENT_TIMESTAMP())
GROUP BY warehouse_name
ORDER BY total_credits DESC;

-- Credits consumed in the last 24 hours by hour
SELECT
  DATE_TRUNC('HOUR', start_time) AS hour,
  warehouse_name,
  SUM(credits_used)              AS credits
FROM snowflake.account_usage.warehouse_metering_history
WHERE start_time ≥ DATEADD(DAY, -1, CURRENT_TIMESTAMP())
GROUP BY 1, 2
ORDER BY 1, 3 DESC;

```

7.2 Multi-Cluster Warehouses (Enterprise Edition and Above)

A standard warehouse is a single cluster. A multi-cluster warehouse automatically adds additional clusters when query concurrency exceeds what one cluster can handle, then removes those clusters when demand subsides. This is scaling **out** (more clusters for concurrent users) rather than scaling **up** (bigger nodes for individual query performance).

```
-- Create a reporting warehouse that scales out for concurrent dashboard users
CREATE WAREHOUSE reporting_wh
  WAREHOUSE_SIZE      = 'MEDIUM'
  MIN_CLUSTER_COUNT   = 1
  MAX_CLUSTER_COUNT   = 5
  SCALING_POLICY      = 'STANDARD';

-- STANDARD policy: add a new cluster immediately when any query is queued
-- ECONOMY policy: add a new cluster only when the system estimates it will save
--                   more credits than it costs (waits ~6 minutes before adding)

-- Best practice: use STANDARD for time-sensitive user-facing workloads
-- Use ECONOMY for batch jobs where a few minutes of delay is acceptable

-- Dedicated ETL warehouse (single cluster, no concurrency concern)
CREATE WAREHOUSE etl_wh
  WAREHOUSE_SIZE      = 'LARGE'
  AUTO_SUSPEND        = 120
  AUTO_RESUME         = TRUE
  MIN_CLUSTER_COUNT   = 1
  MAX_CLUSTER_COUNT   = 1;

-- Dedicated data science warehouse
CREATE WAREHOUSE ds_wh
  WAREHOUSE_SIZE      = 'X-LARGE'
  AUTO_SUSPEND        = 300 -- data scientists often run notebooks with pauses
  AUTO_RESUME         = TRUE;
```

Workload isolation best practice: separate distinct workload types across different warehouses. A heavy ETL job on one warehouse will not impact BI users on a different warehouse because each warehouse has its own independent compute resources.

7.3 The Three-Layer Cache System

Snowflake has three caching layers, each serving a different purpose. Understanding them is critical for query optimization because they determine whether you pay for compute or get results for free.

Layer 1: Result Cache (Cloud Services Layer, 24-hour TTL)

When you run a query, Snowflake stores the result set in the Cloud Services layer. If the exact same query is re-submitted by any user with the same role, against data that has not changed, the result is returned instantly from cache — **zero warehouse compute cost**.

The result cache: - Persists across warehouse suspensions and resumes - Is shared across sessions (same role) - Is invalidated automatically when the underlying data changes (new micro-partitions are written) - Does not apply if the query uses non-deterministic functions like `CURRENT_TIMESTAMP()` or `RANDOM()`

```
-- Disable result cache for this session (useful for benchmarking query
-- performance)
ALTER SESSION SET USE_CACHED_RESULT = FALSE;

-- Re-enable
ALTER SESSION SET USE_CACHED_RESULT = TRUE;

-- Check if recent queries used the result cache
-- percentage_scanned_from_cache = 100% means fully served from result or
-- warehouse cache
SELECT
  query_id,
  LEFT(query_text, 100) AS query_preview,
  execution_status,
  total_elapsed_time / 1000 AS elapsed_seconds,
  percentage_scanned_from_cache,
  partitions_scanned,
  partitions_total
FROM snowflake.account_usage.query_history
WHERE start_time ≥ DATEADD(HOUR, -1, CURRENT_TIMESTAMP())
ORDER BY start_time DESC
LIMIT 50;
```

Layer 2: Metadata Cache (Cloud Services Layer)

Snowflake maintains metadata about every micro-partition: min/max values per column, null count, distinct count, and bloom filters. This metadata is stored in the Cloud Services layer and is consulted before any warehouse compute touches actual data.

This enables **partition pruning**: if your WHERE clause filters on

`order_date = '2024-01-15'` and a micro-partition's metadata shows that the min order_date is `2024-03-01`, Snowflake skips that partition entirely without reading it.

`SELECT COUNT(*) FROM large_table` executes in milliseconds because the answer is stored in metadata — no warehouse needed.

Layer 3: Warehouse Cache (SSD on Warehouse Nodes)

When a warehouse reads micro-partitions from cloud storage, it caches them on the local SSD of the warehouse nodes. Subsequent queries that touch the same micro-partitions read from the fast SSD rather than cloud storage — significantly faster.

The warehouse cache: - Is local to a specific warehouse - Survives across multiple queries on the same warehouse - Is lost when the warehouse suspends (the SSD cache is cleared on resume) - This is one reason why setting AUTO_SUSPEND too aggressively (e.g., 5 seconds) hurts performance for repetitive workloads

```
-- Observe cache hit rate for a specific warehouse over the past week
SELECT
  DATE_TRUNC('DAY', start_time)                AS day,
  warehouse_name,
  COUNT(*)                                     AS queries,
  ROUND(AVG(percentage_scanned_from_cache), 1) AS avg_cache_pct,
  SUM(bytes_scanned) / POWER(1024, 3)          AS gb_scanned,
  SUM(bytes_scanned * (percentage_scanned_from_cache/100))
  / POWER(1024, 3)                            AS gb_from_cache
FROM snowflake.account_usage.query_history
WHERE start_time ≥ DATEADD(DAY, -7, CURRENT_TIMESTAMP())
AND warehouse_name IS NOT NULL
GROUP BY 1, 2
ORDER BY 1 DESC, 3 DESC;
```

7.4 Partition Pruning — The Foundation of Query Performance

Snowflake divides every table into micro-partitions of roughly 50–500 MB of uncompressed data (typically 16 MB compressed). Each micro-partition is stored as an immutable columnar file in cloud object storage. Metadata about each micro-partition (min/max values per column, bloom filters) is stored in the metadata cache.

When you filter a query, Snowflake checks the metadata to determine which micro-partitions **could** contain matching rows, and skips all others. This is partition pruning, and it is the single most impactful performance technique in Snowflake.

```

-- Example: this query benefits from pruning because order_date has natural
-- ordering
-- (data was inserted approximately in date order)
SELECT
    customer_id,
    order_id,
    amount,
    status
FROM orders
WHERE order_date = '2024-01-15';

-- Check how much pruning occurred for your last few queries
SELECT
    query_id,
    LEFT(query_text, 80) AS
    query_preview,
    partitions_total,
    partitions_scanned,
    partitions_total - partitions_scanned AS
    partitions_pruned,
    ROUND(partitions_scanned / NULLIF(partitions_total, 0) * 100, 2) AS
    pct_scanned,
    total_elapsed_time / 1000 AS
    elapsed_seconds
FROM snowflake.account_usage.query_history
WHERE query_text ILIKE '%orders%'
AND start_time ≥ DATEADD(HOUR, -2, CURRENT_TIMESTAMP())
ORDER BY start_time DESC
LIMIT 20;

-- Queries with poor pruning (scanning more than 50% of all partitions)
-- These are candidates for clustering key evaluation
SELECT
    query_id,
    LEFT(query_text, 100) AS
    query_preview,
    partitions_scanned,
    partitions_total,
    ROUND(partitions_scanned / NULLIF(partitions_total, 0) * 100, 1) AS
    pct_scanned,
    total_elapsed_time / 1000 AS
    elapsed_seconds
FROM snowflake.account_usage.query_history
WHERE partitions_total > 100
AND (partitions_scanned::FLOAT / NULLIF(partitions_total, 0)) > 0.5
AND start_time ≥ DATEADD(DAY, -7, CURRENT_TIMESTAMP())
ORDER BY partitions_scanned DESC
LIMIT 30;

```

7.5 Clustering Keys

Micro-partitions are organized in the order data was inserted. If you query heavily on a column that does not align with insertion order — for example, a `country_code` column in a table that was loaded in date order — Snowflake cannot prune effectively because every micro-partition may contain every country.

Clustering keys instruct Snowflake's **Automatic Clustering** service to continuously rearrange micro-partitions so that rows with similar values in the clustering key columns are co-located. This is a background process that incurs credit cost but dramatically improves query performance for the targeted access pattern.

```

-- Evaluate whether a table needs clustering
-- SYSTEM$CLUSTERING_INFORMATION returns a JSON object with statistics
SELECT SYSTEM$CLUSTERING_INFORMATION('orders', '(order_date)');

-- Key fields in the response:
-- "average_depth": ideally < 1.2 (how many micro-partitions a single value spans)
-- "average_overlaps": ideally < 2 (average overlapping partitions per value)
-- Higher values = worse clustering = more partitions scanned per query

-- Add a clustering key to a large table
ALTER TABLE orders CLUSTER BY (order_date);

-- Composite clustering key: most common filter columns first
ALTER TABLE events CLUSTER BY (TO_DATE(event_timestamp), event_type);

-- Expression-based clustering key
ALTER TABLE transactions CLUSTER BY (YEAR(transaction_date), MONTH(transaction_date));

-- Recluster a table immediately (instead of waiting for background process)
ALTER TABLE orders RECLUSTER;

-- Suspend or resume automatic clustering
ALTER TABLE orders SUSPEND RECLUSTER;
ALTER TABLE orders RESUME RECLUSTER;

-- Check clustering state after recluster
SELECT SYSTEM$CLUSTERING_INFORMATION('events', '(TO_DATE(event_timestamp), event_type)');

-- Monitor automatic clustering credit consumption
SELECT
    DATE(start_time) AS cluster_date,
    database_name,
    schema_name,
    table_name,
    SUM(credits_used) AS credits_used,
    SUM(num_bytes_reclustered)
    / POWER(1024, 3) AS gb_reclustered
FROM snowflake.account_usage.automatic_clustering_history
WHERE start_time ≥ DATEADD(DAY, -30, CURRENT_TIMESTAMP())
GROUP BY 1, 2, 3, 4
ORDER BY 1 DESC, 5 DESC;

```

When to use clustering keys: - Table has > 1 TB of data - Queries consistently filter on a column that has poor natural ordering relative to insert order - The column has high cardinality but queries filter on a narrow range (dates, region, status) - Pruning ratio is consistently above 30–40% of all partitions

When NOT to use clustering keys: - Table is small (< 1 TB) — automatic micro-partition elimination is already effective - Table is frequently bulk-replaced (COPY INTO with TRUNCATECOLUMNS, full refreshes) - No consistent filter pattern exists

7.6 Search Optimization Service

The Search Optimization Service (Enterprise+) is designed for **point lookup queries** on large tables — queries that filter by exact match on a high-cardinality column (email address, customer UUID, order ID) where partition pruning offers no help because the values are scattered across all micro-partitions.

Search Optimization builds a persistent, server-managed search access path (bloom filters and other structures) that lets Snowflake find exactly which micro-partitions contain a specific value without scanning all partitions.

```
-- Add search optimization to an entire table
ALTER TABLE customers ADD SEARCH OPTIMIZATION;

-- More targeted: optimize specific columns and access patterns
ALTER TABLE customers ADD SEARCH OPTIMIZATION ON EQUALITY(email, customer_uuid);
ALTER TABLE customers ADD SEARCH OPTIMIZATION ON EQUALITY(email),
    SUBSTRING(full_name);

-- EQUALITY: fast = and IN lookups
-- SUBSTRING: fast LIKE '%substring%' queries (very powerful for text search)
-- GEO: fast geospatial point lookups

-- Check optimization status and cost estimate
SELECT *
FROM information_schema.search_optimization_history
WHERE table_name = 'CUSTOMERS'
ORDER BY created DESC;

-- Verify which columns have search optimization active
SHOW TERSE SEARCH OPTIMIZATION ON customers;

-- Remove search optimization (stops incurring cost)
ALTER TABLE customers DROP SEARCH OPTIMIZATION;
```

7.7 Query Acceleration Service

The Query Acceleration Service (QAS) automatically offloads eligible portions of large, complex analytical queries to additional serverless compute resources — without requiring you to resize the warehouse. It is particularly effective for queries with large scans, aggregations, and filtering on large result sets.

```

-- Enable QAS on a warehouse
ALTER WAREHOUSE analytics_wh SET ENABLE_QUERY_ACCELERATION = TRUE;

-- Set maximum additional scale factor (how much extra compute can be added)
-- A factor of 8 means up to 8x the warehouse's base compute can be added
ALTER WAREHOUSE analytics_wh SET QUERY_ACCELERATION_MAX_SCALE_FACTOR = 8;

-- Identify queries that would benefit from QAS
SELECT
  query_id,
  LEFT(query_text, 100) AS query_preview,
  warehouse_name,
  total_elapsed_time / 1000 AS elapsed_seconds,
  eligible_query_acceleration_time / 1000 AS eligible_seconds,
  ROUND(eligible_query_acceleration_time
    / NULLIF(total_elapsed_time, 0) * 100, 1) AS pct_eligible
FROM snowflake.account_usage.query_acceleration_eligible
WHERE start_time ≥ DATEADD(DAY, -7, CURRENT_TIMESTAMP())
ORDER BY eligible_query_acceleration_time DESC
LIMIT 20;

-- Check actual QAS credit consumption
SELECT
  DATE_TRUNC('DAY', start_time) AS day,
  warehouse_name,
  SUM(credits_used) AS gas_credits
FROM snowflake.account_usage.query_acceleration_history
WHERE start_time ≥ DATEADD(DAY, -30, CURRENT_TIMESTAMP())
GROUP BY 1, 2
ORDER BY 1 DESC;

```

7.8 Query Profiling with Snowsight

Snowsight's Query Profile is your primary debugging tool for slow queries. Access it via:

Activity → Query History → click a query_id → Query Profile tab.

The profile displays an execution plan as a node graph. Each node represents a processing step. Key metrics to examine:

Spilling to local/remote storage: When a warehouse runs out of in-memory space for intermediate results (large sorts, hash joins, aggregations), it spills to local SSD (slow) or remote cloud storage (very slow). Spilling usually means the warehouse is too small for the query.

High network data transfer: Large data transfers between nodes indicate skewed data distribution or suboptimal join ordering.

Pruning percentage: Low pruning (high `partitions_scanned / partitions_total`) indicates missing or ineffective clustering.

```

-- Find queries that spilled to remote storage (most severe performance issue)
SELECT
  query_id,
  LEFT(query_text, 100) AS query_preview,
  warehouse_name,
  warehouse_size,
  total_elapsed_time / 1000 AS elapsed_seconds,
  bytes_spilled_to_local_storage / POWER(1024, 3) AS gb_spilled_local,
  bytes_spilled_to_remote_storage / POWER(1024, 3) AS gb_spilled_remote
FROM snowflake.account_usage.query_history
WHERE bytes_spilled_to_remote_storage > 0
AND start_time ≥ DATEADD(DAY, -7, CURRENT_TIMESTAMP())
ORDER BY bytes_spilled_to_remote_storage DESC
LIMIT 20;

-- Find long-running queries (over 60 seconds) from the past week
SELECT
  query_id,
  LEFT(query_text, 100) AS query_preview,
  warehouse_name,
  warehouse_size,
  total_elapsed_time / 1000 AS elapsed_seconds,
  bytes_scanned / POWER(1024, 3) AS gb_scanned,
  execution_status
FROM snowflake.account_usage.query_history
WHERE total_elapsed_time > 60000
AND start_time ≥ DATEADD(DAY, -7, CURRENT_TIMESTAMP())
ORDER BY total_elapsed_time DESC
LIMIT 30;

-- Queries with poor partition pruning (scanning over 70% of partitions)
SELECT
  query_id,
  LEFT(query_text, 100) AS query_preview,
  partitions_scanned,
  partitions_total,
  ROUND(partitions_scanned / NULLIF(partitions_total, 0) * 100, 1) AS pct_scanned,
  total_elapsed_time / 1000 AS elapsed_seconds
FROM snowflake.account_usage.query_history
WHERE partitions_total > 200
AND partitions_scanned::FLOAT / NULLIF(partitions_total, 0) > 0.7
AND start_time ≥ DATEADD(DAY, -7, CURRENT_TIMESTAMP())
ORDER BY partitions_scanned DESC
LIMIT 20;

-- Most expensive queries by total credit consumption
-- (credits ≈ elapsed_time × warehouse_size / seconds_per_hour)
SELECT
  query_id,
  LEFT(query_text, 100) AS query_preview,

```

```

warehouse_name,
warehouse_size,
total_elapsed_time / 1000                                AS elapsed_seconds,
CASE warehouse_size
  WHEN 'X-Small' THEN 1
  WHEN 'Small'   THEN 2
  WHEN 'Medium'  THEN 4
  WHEN 'Large'   THEN 8
  WHEN 'X-Large' THEN 16
  ELSE 32
END * (total_elapsed_time / 3600000.0)                  AS approx_credits_consumed
FROM snowflake.account_usage.query_history
WHERE start_time ≥ DATEADD(DAY, -7, CURRENT_TIMESTAMP())
ORDER BY approx_credits_consumed DESC
LIMIT 20;

```

7.9 Performance Optimization Best Practices

Warehouse sizing: - Start with Medium for general analytics; resize up only after observing spilling or sustained high queue times - Use a separate, larger warehouse for ETL/dbt transformations than for BI queries - Set AUTO_SUSPEND to 60–120 seconds for user-facing warehouses, 300+ seconds for batch jobs where warm-up latency matters

Query writing: - Filter on clustered columns (date, region, status) as the primary filter condition - Avoid `SELECT *` in production — select only required columns to reduce bytes scanned - Use `QUALIFY` instead of subqueries for window function filtering:

```

-- Inefficient: subquery
SELECT * FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DE
    SC) AS rn
  FROM orders
) WHERE rn = 1;

-- Efficient: QUALIFY (Snowflake-native)
SELECT *
FROM orders
QUALIFY ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) =
  1;

-- Use transient tables for intermediate results in multi-step transformations
-- (no Fail-Safe overhead, cheaper storage)
CREATE TRANSIENT TABLE staging.orders_enriched AS
SELECT
  o.*,
  c.country_code,
  c.customer_segment
FROM staging.orders_raw o
JOIN staging.customers c ON o.customer_id = c.customer_id;

-- Avoid functions on filter columns (prevents pruning)
-- Bad: function applied to column prevents min/max pruning
SELECT * FROM orders WHERE YEAR(order_date) = 2024;

-- Good: range filter preserves pruning
SELECT * FROM orders WHERE order_date BETWEEN '2024-01-01' AND '2024-12-31';

-- Use COPY INTO with parallelism for bulk loading
COPY INTO staging.orders_raw
FROM @raw_data_stage/orders/
FILE_FORMAT = (TYPE = PARQUET)
MATCH_BY_COLUMN_NAME = CASE_INSENSITIVE
PURGE = FALSE;

```

Chapter 7 Summary

Virtual warehouses are independent compute clusters that scale instantly without affecting storage. The three-layer cache system (result cache, metadata cache, warehouse SSD cache) means that well-structured, repeated queries can return at zero or near-zero compute cost. Partition pruning is the single most impactful performance lever — engineer your data loading, clustering keys, and query filter patterns to maximize it. Use the `snowflake.account_usage.query_history` view to identify and diagnose poor-performing queries systematically rather than guessing.

Key Takeaways: - Separate ETL, BI, and data science workloads into dedicated warehouses - The result cache (24-hour TTL) eliminates compute cost for repeated identical queries - Clustering keys re-sort micro-partitions for columns with poor natural ordering - Spilling to remote storage is the clearest signal that a warehouse needs to be larger - Search Optimization Service targets point-lookup queries on high-cardinality columns

What's Next: Chapter 8 covers Snowflake's enterprise security model — RBAC, dynamic data masking, row access policies, network policies, SSO, and OAuth.

Chapter 8: Security & Access Control

8.1 Snowflake's Multi-Layer Security Model

Snowflake's security model operates at six distinct layers, each independently configurable:

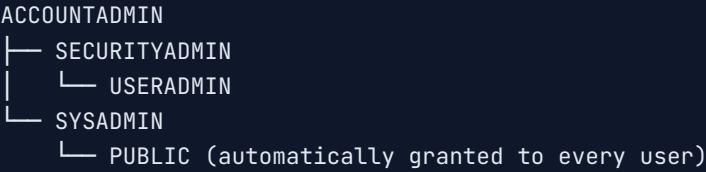
Layer	Mechanism
Network	IP allowlisting, Private Link (AWS PrivateLink / Azure Private Link / GCP Private Service Connect)
Authentication	Password, MFA (TOTP), SSO (SAML 2.0), key-pair, OAuth 2.0
Authorization	Role-Based Access Control (RBAC) with object-level privileges
Column Security	Dynamic Data Masking, Tokenization
Row Security	Row Access Policies
Encryption	AES-256 at rest (Tri-Secret Secure option), TLS 1.2+ in transit
Audit	Access history, login history, query history (all in <code>snowflake.account_usage</code>)

Snowflake is SOC 2 Type II, SOC 1 Type II, PCI DSS, HIPAA, ISO 27001, FedRAMP Moderate, HITRUST, and GDPR compliant out of the box. You do not manage encryption keys or security patches — Snowflake handles the infrastructure.

8.2 Role-Based Access Control (RBAC)

Every privilege in Snowflake is granted to a role, and roles are granted to users or other roles. Users switch between roles within a session. Roles form a directed acyclic hierarchy — a role inherits all privileges of any role granted to it.

System-defined role hierarchy:



System Role	Purpose	Who Should Have It
ACCOUNTADMIN	Top-level admin: billing, account settings, resource monitors	2–3 named individuals maximum
SYSADMIN	Creates and owns databases, warehouses, schemas, tables	Senior engineers, platform team
SECURITYADMIN	Creates users, roles, and network policies	Security/IAM team
USERADMIN	Creates users and roles (cannot grant object privileges)	Delegated provisioning
PUBLIC	Minimal; auto-granted to every user	Everyone

Building a least-privilege role hierarchy:

```

-- Run as SECURITYADMIN --

-- Create functional roles
CREATE ROLE IF NOT EXISTS data_engineer_role;
CREATE ROLE IF NOT EXISTS data_analyst_role;
CREATE ROLE IF NOT EXISTS data_scientist_role;
CREATE ROLE IF NOT EXISTS dbt_role;
CREATE ROLE IF NOT EXISTS data_viewer_role; -- read-only, most restrictive

-- Build role hierarchy: engineers inherit analyst privileges
GRANT ROLE data_analyst_role TO ROLE data_engineer_role;
GRANT ROLE data_engineer_role TO ROLE SYSADMIN;
GRANT ROLE data_scientist_role TO ROLE SYSADMIN;
GRANT ROLE dbt_role TO ROLE SYSADMIN;

-- Create users
CREATE USER alice
  LOGIN_NAME      = 'alice@company.com'
  DISPLAY_NAME    = 'Alice Smith'
  DEFAULT_ROLE    = data_engineer_role
  DEFAULT_WAREHOUSE = transform_wh
  DEFAULT_NAMESPACE = analytics_db.staging
  MUST_CHANGE_PASSWORD = TRUE;

CREATE USER bob
  LOGIN_NAME      = 'bob@company.com'
  DISPLAY_NAME    = 'Bob Jones'
  DEFAULT_ROLE    = data_analyst_role
  DEFAULT_WAREHOUSE = analytics_wh
  DEFAULT_NAMESPACE = analytics_db.marts
  MUST_CHANGE_PASSWORD = TRUE;

-- Assign roles to users
GRANT ROLE data_engineer_role TO USER alice;
GRANT ROLE data_analyst_role TO USER bob;
GRANT ROLE data_scientist_role TO USER carol;

-- Run as SYSADMIN for object privileges --

-- Analyst: read-only access to marts layer
GRANT USAGE ON DATABASE analytics_db TO ROLE data_analyst_role;
GRANT USAGE ON ALL SCHEMAS IN DATABASE analytics_db TO ROLE data_analyst_role;
GRANT SELECT ON ALL TABLES IN SCHEMA analytics_db.marts TO ROLE
  data_analyst_role;
GRANT SELECT ON ALL VIEWS IN SCHEMA analytics_db.marts TO ROLE data_analyst_role;
-- FUTURE grants ensure newly created tables are automatically accessible
GRANT SELECT ON FUTURE TABLES IN SCHEMA analytics_db.marts TO ROLE
  data_analyst_role;
GRANT SELECT ON FUTURE VIEWS IN SCHEMA analytics_db.marts TO ROLE
  data_analyst_role;

-- Engineer: read from raw, full write to staging
GRANT USAGE ON DATABASE analytics_db TO ROLE data_engineer_role;

```

```

GRANT USAGE ON ALL SCHEMAS IN DATABASE analytics_db TO ROLE data_engineer_role;
GRANT SELECT ON ALL TABLES IN SCHEMA analytics_db.raw TO ROLE data_engineer_role;
GRANT FUTURE TABLES ON SCHEMA analytics_db.raw TO ROLE data_engineer_role;
GRANT ALL PRIVILEGES ON SCHEMA analytics_db.staging TO ROLE data_engineer_role;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA analytics_db.staging TO ROLE
    data_engineer_role;
GRANT ALL PRIVILEGES ON FUTURE TABLES IN SCHEMA analytics_db.staging TO ROLE
    data_engineer_role;

-- Warehouse usage
GRANT USAGE ON WAREHOUSE analytics_wh TO ROLE data_analyst_role;
GRANT USAGE ON WAREHOUSE transform_wh TO ROLE data_engineer_role;
GRANT USAGE ON WAREHOUSE ds_wh TO ROLE data_scientist_role;

-- dbt service role: full write access to staging and marts
GRANT USAGE ON DATABASE analytics_db TO ROLE dbt_role;
GRANT ALL PRIVILEGES ON SCHEMA analytics_db.staging TO ROLE dbt_role;
GRANT ALL PRIVILEGES ON SCHEMA analytics_db.marts TO ROLE dbt_role;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA analytics_db.staging TO ROLE
    dbt_role;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA analytics_db.marts TO ROLE dbt_role;
GRANT ALL PRIVILEGES ON FUTURE TABLES IN SCHEMA analytics_db.staging TO ROLE
    dbt_role;
GRANT ALL PRIVILEGES ON FUTURE TABLES IN SCHEMA analytics_db.marts TO ROLE
    dbt_role;
GRANT ALL PRIVILEGES ON ALL VIEWS IN SCHEMA analytics_db.marts TO ROLE dbt_role;
GRANT ALL PRIVILEGES ON FUTURE VIEWS IN SCHEMA analytics_db.marts TO ROLE
    dbt_role;
GRANT USAGE ON WAREHOUSE transform_wh TO ROLE dbt_role;

-- Audit: inspect privileges
SHOW GRANTS TO ROLE data_analyst_role;
SHOW GRANTS ON DATABASE analytics_db;
SHOW GRANTS TO USER alice;
SHOW GRANTS OF ROLE data_engineer_role;

```

8.3 Column-Level Security — Dynamic Data Masking

Dynamic Data Masking (DDM) applies a masking policy to a column. When a user queries that column, the policy function runs and returns either the real value or a masked value depending on the user's current role. The policy is applied at query time — the underlying data is never modified.

```

-- Email masking: full value for engineers, partial mask for analysts, fully
redacted for all others
CREATE OR REPLACE MASKING POLICY email_mask AS (val VARCHAR)
  RETURNS VARCHAR →
  CASE
    WHEN CURRENT_ROLE() IN ('DATA_ENGINEER_ROLE', 'SECURITYADMIN', 'ACCOUNTADMIN')
      THEN val -- see full email
    WHEN CURRENT_ROLE() = 'DATA_ANALYST_ROLE'
      THEN REGEXP_REPLACE(val, '^[^@]+', '****') -- mask local part:
      ****@domain.com
    ELSE '***REDACTED***'
  END;

-- Phone masking: show only last 4 digits to analysts
CREATE OR REPLACE MASKING POLICY phone_mask AS (val VARCHAR)
  RETURNS VARCHAR →
  CASE
    WHEN CURRENT_ROLE() IN ('DATA_ENGINEER_ROLE', 'SECURITYADMIN', 'ACCOUNTADMIN')
      THEN val
    WHEN CURRENT_ROLE() = 'DATA_ANALYST_ROLE'
      THEN CONCAT('***-***-', RIGHT(val, 4))
    ELSE '***-***-****'
  END;

-- SSN masking: only ACCOUNTADMIN sees full value
CREATE OR REPLACE MASKING POLICY ssn_mask AS (val VARCHAR)
  RETURNS VARCHAR →
  CASE
    WHEN CURRENT_ROLE() = 'ACCOUNTADMIN' THEN val
    ELSE '***-**- ' || RIGHT(val, 4)
  END;

-- Numeric salary masking: round to nearest $10,000 for analysts
CREATE OR REPLACE MASKING POLICY salary_mask AS (val NUMBER)
  RETURNS NUMBER →
  CASE
    WHEN CURRENT_ROLE() IN ('DATA_ENGINEER_ROLE', 'ACCOUNTADMIN') THEN val
    ELSE ROUND(val, -4) -- rounds to nearest 10,000
  END;

-- Apply masking policies to columns
ALTER TABLE customers MODIFY COLUMN email SET MASKING POLICY email_mask;
ALTER TABLE customers MODIFY COLUMN phone SET MASKING POLICY phone_mask;
ALTER TABLE employees MODIFY COLUMN ssn SET MASKING POLICY ssn_mask;
ALTER TABLE employees MODIFY COLUMN salary SET MASKING POLICY salary_mask;

-- Remove a masking policy from a column
ALTER TABLE customers MODIFY COLUMN email UNSET MASKING POLICY;

-- View which tables/columns have masking policies applied
SELECT *

```

```
FROM information_schema.policy_references
WHERE policy_name = 'EMAIL_MASK';

-- List all masking policies in a schema
SHOW MASKING POLICIES IN SCHEMA analytics_db.marts;
```

Conditional masking — masking based on values in a reference table:

```
-- Masking based on a user → data classification mapping table
CREATE OR REPLACE MASKING POLICY dynamic_pii_mask AS (val VARCHAR)
  RETURNS VARCHAR →
  CASE
    WHEN EXISTS (
      SELECT 1 FROM analytics_db.security.user_data_access
      WHERE username = CURRENT_USER()
      AND can_see_pii = TRUE
    ) THEN val
    ELSE '***'
  END;
```

8.4 Row-Level Security — Row Access Policies

A Row Access Policy is a function that returns `TRUE` or `FALSE` for each row. Rows where the function returns `FALSE` are invisible to the querying user — they do not appear in results and do not count in aggregations.

```

-- Simple region-based row access: users only see their region
-- Requires a mapping table that defines which regions each user can access
CREATE TABLE analytics_db.security.user_region_mapping (
  username          VARCHAR,
  allowed_region    VARCHAR,
  effective_date    DATE
);

-- Populate the mapping table
INSERT INTO analytics_db.security.user_region_mapping VALUES
  ('alice@company.com', 'NORTH_AMERICA', '2024-01-01'),
  ('alice@company.com', 'EUROPE', '2024-01-01'),
  ('bob@company.com', 'NORTH_AMERICA', '2024-01-01');

-- Create the row access policy
CREATE OR REPLACE ROW ACCESS POLICY analytics_db.security.region_access_policy
  AS (region VARCHAR)
  RETURNS BOOLEAN →
  CASE
    -- Global access: SYSADMIN and ACCOUNTADMIN see all rows
    WHEN CURRENT_ROLE() IN ('SYSADMIN', 'ACCOUNTADMIN', 'DATA_ENGINEER_ROLE')
    THEN TRUE
    -- Role-based access via mapping table
    WHEN EXISTS (
      SELECT 1
      FROM analytics_db.security.user_region_mapping
      WHERE username = CURRENT_USER()
      AND allowed_region = region
      AND effective_date ≤ CURRENT_DATE()
    ) THEN TRUE
    ELSE FALSE
  END;

-- Apply to a table (the column name must match the policy parameter name in
  type)
ALTER TABLE analytics_db.marts.fct_orders
  ADD ROW ACCESS POLICY analytics_db.security.region_access_policy ON (region);

-- Multiple columns can be used (pass multiple arguments to policy)
CREATE OR REPLACE ROW ACCESS POLICY analytics_db.security.dept_country_policy
  AS (department_id NUMBER, country_code VARCHAR)
  RETURNS BOOLEAN →
  CURRENT_ROLE() IN ('SYSADMIN', 'ACCOUNTADMIN')
  OR EXISTS (
    SELECT 1 FROM analytics_db.security.user_access_matrix
    WHERE username = CURRENT_USER()
    AND (allowed_dept_id = department_id OR allowed_dept_id IS NULL)
    AND (allowed_country = country_code OR allowed_country = 'ALL')
  );

-- Apply multi-column policy
ALTER TABLE analytics_db.marts.fct_employee_performance
  ADD ROW ACCESS POLICY analytics_db.security.dept_country_policy

```

```
ON (department_id, country_code);

-- Remove a row access policy
ALTER TABLE analytics_db.marts.fct_orders
  DROP ROW ACCESS POLICY analytics_db.security.region_access_policy;

-- View all row access policy assignments
SELECT *
FROM information_schema.policy_references
WHERE policy_kind = 'ROW_ACCESS_POLICY';
```

8.5 Network Policies

Network policies restrict which IP addresses or CIDR ranges can connect to a Snowflake account or specific user.

```

-- Allow only corporate IP ranges, block a specific known-bad IP
CREATE NETWORK POLICY corporate_network_policy
  ALLOWED_IP_LIST = ('10.0.0.0/8', '192.168.1.0/24', '203.0.113.50/32')
  BLOCKED_IP_LIST = ('203.0.113.100')
  COMMENT = 'Allow corporate office and VPN only';

-- Apply to the entire account (affects all users without user-level override)
ALTER ACCOUNT SET NETWORK_POLICY = corporate_network_policy;

-- Apply to a specific service account only (overrides account-level policy for
  this user)
ALTER USER dbt_service_user SET NETWORK_POLICY = corporate_network_policy;

-- Remove network policy from a user
ALTER USER dbt_service_user UNSET NETWORK_POLICY;

-- View current account-level network policy
SHOW PARAMETERS LIKE 'NETWORK_POLICY' IN ACCOUNT;

-- Modern approach: Network Rules (more granular, supports Private Link)
CREATE NETWORK RULE allow_office_ips
  TYPE = IPV4
  MODE = INGRESS
  VALUE_LIST = ('10.0.0.0/8', '203.0.113.0/24')
  COMMENT = 'Corporate office and VPN CIDR blocks';

CREATE NETWORK RULE block_public_egress
  TYPE = HOST_PORT
  MODE = EGRESS
  VALUE_LIST = ('0.0.0.0/0:80', '0.0.0.0/0:443')
  COMMENT = 'Used in egress restriction scenarios';

CREATE NETWORK POLICY v2_corporate_policy
  ALLOWED_NETWORK_RULE_LIST = ('allow_office_ips')
  COMMENT = 'Network policy using rules';

```

8.6 Multi-Factor Authentication and SSO

MFA adds a time-based one-time password (TOTP) requirement to password authentication.

```

-- Enroll a user in MFA (user must complete enrollment via Snowsight or CLI)
ALTER USER alice SET MINS_TO_BYPASS_MFA = 0; -- never bypass MFA

-- Check MFA enrollment status
SELECT name, login_name, has_mfa
FROM snowflake.account_usage.users
WHERE deleted_on IS NULL;

-- Force all non-service users to use MFA (ACCOUNTADMIN only)
-- Note: this is enforced at the policy level via the Snowflake Trust Center
-- in Enterprise accounts. In Standard, enforce per-user via ALTER USER.

```

SSO via SAML 2.0 — integrate with Okta, Azure AD, Google Workspace, Ping Identity:

```

-- Create a SAML 2.0 security integration (Okta example)
CREATE SECURITY INTEGRATION okta_saml_integration
  TYPE = SAML2
  ENABLED = TRUE
  SAML2_ISSUER = 'http://www.okta.com/exkABC123'
  SAML2_SSO_URL = 'https://company.okta.com/app/snowflake/
    exkABC123/sso/saml'
  SAML2_PROVIDER = 'OKTA'
  SAML2_X509_CERT = 'MIIDpD...<certificate-content>...=='
  SAML2_SP_INITIATED_LOGIN_PAGE_LABEL = 'Sign in with Okta'
  SAML2_ENABLE_SP_INITIATED = TRUE
  SAML2_SIGN_REQUEST = TRUE;

-- Retrieve the Snowflake SP metadata URL (provide to your IdP)
SELECT SYSTEM$GET_SNOWFLAKE_PLATFORM_INFO();

-- Describe the integration
DESCRIBE INTEGRATION okta_saml_integration;

```

Key-pair authentication for service accounts and automated tools (no password):

```

# Generate private key (RSA 2048-bit)
openssl genrsa 2048 | openssl pkcs8 -topk8 -inform PEM -out rsa_key.p8 -nocrypt

# Generate public key from private key
openssl rsa -in rsa_key.p8 -pubout -out rsa_key.pub

# Get the public key content (strip headers)
cat rsa_key.pub | grep -v "PUBLIC KEY" | tr -d '\n'

```

```

-- Assign the public key to a service account user
ALTER USER dbt_service_user SET RSA_PUBLIC_KEY = 'MIIBIjANBgqhki...';

-- Verify key fingerprint
DESC USER dbt_service_user;

-- Look for RSA_PUBLIC_KEY_FP in the output

```

8.7 OAuth for BI Tools

OAuth enables BI tools and applications to authenticate using short-lived tokens rather than storing Snowflake credentials.

```

-- Snowflake OAuth integration for Tableau Desktop
CREATE SECURITY INTEGRATION tableau_oauth
  TYPE = OAUTH
  OAUTH_CLIENT = TABLEAU_DESKTOP
  ENABLED = TRUE;

-- Snowflake OAuth for a custom application
CREATE SECURITY INTEGRATION custom_app_oauth
  TYPE = OAUTH
  OAUTH_CLIENT = CUSTOM
  OAUTH_CLIENT_TYPE = CONFIDENTIAL
  OAUTH_REDIRECT_URI = 'https://myapp.company.com/oauth/callback'
  OAUTH_ISSUE_REFRESH_TOKENS = TRUE
  OAUTH_REFRESH_TOKEN_VALIDITY = 86400 -- 24 hours
  ENABLED = TRUE;

-- Get OAuth endpoints
DESCRIBE INTEGRATION custom_app_oauth;
-- Returns: OAUTH_AUTHORIZATION_ENDPOINT, OAUTH_TOKEN_ENDPOINT, CLIENT_ID

-- External OAuth: accept tokens from your own Identity Provider (e.g., Azure AD)
CREATE SECURITY INTEGRATION azure_ad_external_oauth
  TYPE = EXTERNAL_OAUTH
  ENABLED = TRUE
  EXTERNAL_OAUTH_TYPE = AZURE
  EXTERNAL_OAUTH_ISSUER = 'https://sts.windows.net/
    <tenant-id>/'
  EXTERNAL_OAUTH_JWS_KEYS_URL = 'https://
    login.microsoftonline.com/<tenant-id>/discovery/v2.0/keys'
  EXTERNAL_OAUTH_AUDIENCE_LIST = ('api://<app-id>')
  EXTERNAL_OAUTH_TOKEN_USER_MAPPING_CLAIM = 'upn'
  EXTERNAL_OAUTH_SNOWFLAKE_USER_MAPPING_ATTRIBUTE = 'login_name'
  EXTERNAL_OAUTH_ANY_ROLE_MODE = 'ENABLE';

```

8.8 Secrets Management

Snowflake Secrets allow you to store credentials, API keys, and connection strings securely within Snowflake, where they can be referenced in stored procedures, UDFs, and external access integrations without ever being exposed in plaintext code.

```

-- Create a generic secret (e.g., API key)
CREATE SECRET analytics_db.public.openai_api_key
  TYPE = GENERIC_STRING
  SECRET_STRING = '{"api_key": "sk-proj-abc123xyz", "endpoint": "https://api.openai.com/v1"}'
  COMMENT = 'OpenAI API credentials for enrichment pipeline';

-- Create a username/password secret
CREATE SECRET analytics_db.public.postgres_creds
  TYPE = PASSWORD
  USERNAME = 'etl_user'
  PASSWORD = 'secure_password_here';

-- Create an OAuth2 client credentials secret
CREATE SECRET analytics_db.public.salesforce_oauth
  TYPE = OAUTH2
  API_AUTHENTICATION = salesforce_api_integration -- references a security integration
  OAUTH_SCOPES = ('api', 'refresh_token');

-- Reference a secret in a stored procedure
CREATE OR REPLACE PROCEDURE call_enrichment_api(customer_id VARCHAR)
  RETURNS VARCHAR
  LANGUAGE PYTHON
  RUNTIME_VERSION = '3.11'
  PACKAGES = ('snowflake-snowpark-python', 'requests')
  HANDLER = 'run'
  EXTERNAL_ACCESS_INTEGRATIONS = (openai_external_access)
  SECRETS = ('api_creds' = analytics_db.public.openai_api_key)
AS $$
import requests
import json

def run(session, customer_id: str) → str:
    import _snowflake
    creds_str = _snowflake.get_generic_secret_string('api_creds')
    creds = json.loads(creds_str)

    response = requests.post(
        f"{creds['endpoint']}/chat/completions",
        headers={
            'Authorization': f"Bearer {creds['api_key']}",
            'Content-Type': 'application/json'
        },
        json={
            "model": "gpt-4o",
            "messages": [{"role": "user", "content": f"Classify customer {customer_id}"}]
        }
    )
    return response.json()['choices'][0]['message']['content']
$$;

```

Chapter 8 Summary

Snowflake's security model is defense-in-depth: network isolation prevents unauthorized connections, RBAC grants least-privilege access to objects, dynamic data masking protects sensitive columns at query time, row access policies limit which rows are visible, and all activity is logged in `snowflake.account_usage`. The key design principle is that security is declarative — policies are attached to objects and enforced automatically, not embedded in application code.

Key Takeaways: - Grant privileges to roles, not users; assign roles to users - Use FUTURE grants to ensure new objects inherit correct permissions automatically - Dynamic Data Masking is transparent to users — masked data looks like real data (just obfuscated) - Row Access Policies are invisible — users see a smaller result set without knowing rows were excluded - Use key-pair authentication for all service accounts (no shared passwords)

What's Next: Chapter 9 covers Time Travel and Zero-Copy Cloning — Snowflake's mechanisms for recovering from data errors and creating instant development environments.

Chapter 9: Time Travel, Cloning & Data Protection

9.1 Time Travel — Querying Historical Data

Time Travel allows you to query, restore, and clone data as of any point within the retention period, as if you had a time machine for your database. It works by retaining old micro-partition versions even after they are logically deleted or overwritten.

Retention period by edition:

Snowflake Edition	Maximum Retention	Default
Standard	1 day	1 day
Enterprise and above	90 days	1 day

Storage consumed during the retention period is charged at Snowflake's standard storage rate (approximately \$23/TB/month). Setting very long retention periods on large, frequently-updated tables has meaningful cost implications.

```

-- Configure retention at different object levels
-- (more specific overrides less specific)
ALTER ACCOUNT      SET DATA_RETENTION_TIME_IN_DAYS = 7;          -- account default
ALTER DATABASE     analytics_db SET DATA_RETENTION_TIME_IN_DAYS = 30;
ALTER SCHEMA       analytics_db.marts SET DATA_RETENTION_TIME_IN_DAYS = 14;
ALTER TABLE       analytics_db.marts.fct_orders SET DATA_RETENTION_TIME_IN_DAYS = 3
0;

-- Disable Time Travel for a table (retention = 0)
-- Useful for extremely large, frequently-updated tables where cost outweighs
  benefit
ALTER TABLE analytics_db.staging.orders_raw SET DATA_RETENTION_TIME_IN_DAYS = 0;

-- Query data AS OF a specific timestamp
SELECT order_id, customer_id, amount, status
FROM analytics_db.marts.fct_orders
AT (TIMESTAMP => '2024-01-15 09:00:00.000'::TIMESTAMP_NTZ);

-- Query relative to current time (OFFSET is in seconds; negative = past)
SELECT * FROM analytics_db.marts.fct_orders
AT (OFFSET => -3600);      -- as it was 1 hour ago

-- Query BEFORE a specific query ran (useful if you know the query_id that caused
  damage)
-- This queries the state as it was just BEFORE that query executed
SELECT * FROM analytics_db.marts.fct_orders
BEFORE (STATEMENT => '01b3e4d5-0001-7890-abcd-ef0123456789');

-- Find rows that existed 2 hours ago but no longer exist (deleted rows)
SELECT order_id, customer_id, amount, status
FROM analytics_db.marts.fct_orders
  AT (TIMESTAMP => DATEADD(HOUR, -2, CURRENT_TIMESTAMP()))
MINUS
SELECT order_id, customer_id, amount, status
FROM analytics_db.marts.fct_orders;

-- Find rows that were modified in the last 2 hours
-- (exist in both versions but have different values)
SELECT
  h.order_id,
  h.amount      AS amount_2hrs_ago,
  c.amount      AS current_amount,
  c.amount - h.amount AS delta,
  h.status      AS status_2hrs_ago,
  c.status      AS current_status
FROM analytics_db.marts.fct_orders
  AT (TIMESTAMP => DATEADD(HOUR, -2, CURRENT_TIMESTAMP())) h
JOIN analytics_db.marts.fct_orders c USING (order_id)
WHERE h.amount ≠ c.amount OR h.status ≠ c.status;

```

9.2 Restoring Dropped Objects with UNDROP

UNDROP restores a dropped table, schema, or database to the exact state it was in when it was dropped. The restored object retains its original name, columns, data, and privileges.

```
-- Scenario: a developer accidentally drops the customers table
DROP TABLE analytics_db.marts.dim_customers;

-- Restore it (must be within the retention period)
UNDROP TABLE analytics_db.marts.dim_customers;

-- Scenario: an entire schema was dropped
DROP SCHEMA analytics_db.staging;
UNDROP SCHEMA analytics_db.staging;
-- All tables, views, stages, and file formats within the schema are restored

-- Scenario: a database was dropped
DROP DATABASE analytics_db;
UNDROP DATABASE analytics_db;

-- If a new object with the same name was created after the drop,
-- UNDROP will fail. Rename the new object first, then UNDROP.
ALTER TABLE analytics_db.marts.dim_customers RENAME TO
    analytics_db.marts.dim_customers_new;
UNDROP TABLE analytics_db.marts.dim_customers;
-- Now you have both: dim_customers (original) and dim_customers_new
-- (replacement)

-- Recover specific deleted rows into a recovery table
CREATE TABLE analytics_db.marts.dim_customers_recovered AS
SELECT *
FROM analytics_db.marts.dim_customers
    AT (TIMESTAMP => '2024-01-14 08:00:00'::TIMESTAMP_NTZ)
WHERE customer_id NOT IN (
    SELECT customer_id FROM analytics_db.marts.dim_customers
);

-- Merge recovered rows back into the live table
MERGE INTO analytics_db.marts.dim_customers AS target
USING analytics_db.marts.dim_customers_recovered AS source
    ON target.customer_id = source.customer_id
    WHEN NOT MATCHED THEN INSERT VALUES (source.*);
```

9.3 Fail-Safe

After the Time Travel retention period expires, Snowflake retains micro-partitions for an additional 7 days in **Fail-Safe**. This is an internal disaster recovery mechanism — it is not accessible to you as a user. Only Snowflake Support can recover data from Fail-Safe, and recovery is not guaranteed or instantaneous.

Important distinctions by table type:

Table Type	Time Travel	Fail-Safe	Notes
Permanent	Yes (0–90 days)	Yes (7 days)	Default table type
Transient	Yes (0–1 day max)	No	Cheaper storage, use for intermediates
Temporary	Yes (0–1 day max)	No	Session-scoped, auto-dropped

```

-- Create a transient table (no Fail-Safe, max 1 day Time Travel)
CREATE TRANSIENT TABLE staging.orders_intermediate (
  order_id    NUMBER,
  customer_id NUMBER,
  amount      FLOAT,
  processed_at TIMESTAMP_NTZ
);

-- Create a transient schema (all tables within are transient)
CREATE TRANSIENT SCHEMA analytics_db.scratch;

-- Create a temporary table (dropped at end of session)
CREATE TEMPORARY TABLE session_temp_results AS
SELECT * FROM analytics_db.marts.fct_orders WHERE order_date = CURRENT_DATE();
-- This table exists only for the duration of this session

```

Storage cost timeline visualization:

```

Data inserted at T=0
├─ T=0 to T=90 days → [Live Data + Time Travel storage overhead]
├─ T=90 to T=97 days → [Fail-Safe: Snowflake internal, you pay for storage]
└─ T=97+ days       → [Data gone, no further storage charge for old versions]

```

9.4 Zero-Copy Cloning

Zero-Copy Cloning creates an instant copy of any Snowflake object (table, schema, database, stage, file format, sequence, stream) without physically copying the underlying data. The clone and the source share the same micro-partitions in cloud storage. Storage is only charged when either object modifies data — at that point, only the new or modified micro-partitions are stored separately.

This makes cloning: - **Instant**: completes in seconds regardless of data size - **Free at creation**: no additional storage until data diverges - **Independently mutable**: changes to the clone do not affect the source, and vice versa

```
-- Clone a single table
CREATE TABLE analytics_db.marts.fct_orders_backup
  CLONE analytics_db.marts.fct_orders;

-- Clone at a specific point in Time Travel (historical snapshot)
CREATE TABLE analytics_db.marts.fct_orders_jan15_snapshot
  CLONE analytics_db.marts.fct_orders
  AT (TIMESTAMP ⇒ '2024-01-15 00:00:00'::TIMESTAMP_NTZ);

-- Clone a schema (all tables, views, stages, file formats, sequences within it)
CREATE SCHEMA analytics_db.staging_dev
  CLONE analytics_db.staging;

-- Clone an entire database (everything in every schema)
CREATE DATABASE analytics_dev
  CLONE analytics_prod;

-- Clone production database to a point-in-time snapshot for development
CREATE DATABASE analytics_dev
  CLONE analytics_prod
  AT (TIMESTAMP ⇒ DATEADD(DAY, -1, CURRENT_TIMESTAMP()));

-- Grant the dev team access to the cloned database
GRANT USAGE ON DATABASE analytics_dev TO ROLE data_engineer_role;
GRANT ALL PRIVILEGES ON ALL SCHEMAS IN DATABASE analytics_dev TO ROLE
  data_engineer_role;
GRANT ALL PRIVILEGES ON ALL TABLES IN DATABASE analytics_dev TO ROLE
  data_engineer_role;
```

Production use cases for cloning:

```

-- Use Case 1: Pre-transformation safety snapshot
-- Before running a risky backfill or data transformation
CREATE TABLE analytics_db.marts.fct_orders_pre_backfill
  CLONE analytics_db.marts.fct_orders;

-- Run the risky transformation
UPDATE analytics_db.marts.fct_orders
SET amount = amount * 1.1
WHERE order_date ≥ '2024-01-01' AND region = 'EUROPE';

-- Validate the transformation
SELECT
  'original' AS source,
  SUM(amount) AS total_amount,
  COUNT(*) AS row_count
FROM analytics_db.marts.fct_orders_pre_backfill
WHERE order_date ≥ '2024-01-01' AND region = 'EUROPE'
UNION ALL
SELECT
  'transformed',
  SUM(amount),
  COUNT(*)
FROM analytics_db.marts.fct_orders
WHERE order_date ≥ '2024-01-01' AND region = 'EUROPE';

-- If the transformation is wrong: restore from clone
CREATE OR REPLACE TABLE analytics_db.marts.fct_orders
  CLONE analytics_db.marts.fct_orders_pre_backfill;

-- Clean up backup clone
DROP TABLE analytics_db.marts.fct_orders_pre_backfill;

-- Use Case 2: Ephemeral dev environment from prod snapshot
-- Run every morning as part of a CI/CD pipeline
CREATE OR REPLACE DATABASE dev_db CLONE prod_db;
GRANT USAGE ON DATABASE dev_db TO ROLE data_engineer_role;
GRANT ALL PRIVILEGES ON ALL SCHEMAS IN DATABASE dev_db TO ROLE
  data_engineer_role;
GRANT ALL PRIVILEGES ON ALL TABLES IN DATABASE dev_db TO ROLE data_engineer_role;
-- Developers now have a full copy of prod data to work with
-- Cost is zero until they write data; the whole clone shares prod's storage

-- Use Case 3: Test a dbt migration
CREATE DATABASE dbt_test_db CLONE analytics_prod;
-- Run dbt against dbt_test_db
-- dbt run --target test_db --profiles-dir ./profiles
-- Verify results, then drop
DROP DATABASE dbt_test_db;

```

Chapter 9 Summary

Time Travel, Fail-Safe, and Zero-Copy Cloning together form Snowflake's data protection triad. Time Travel gives you user-accessible historical snapshots for querying and recovery. Fail-Safe provides a last-resort backstop managed by Snowflake Support. Zero-Copy Cloning enables instant, cost-free environment provisioning that would take hours with traditional databases. These features fundamentally change how you approach data operations — instead of writing complex backup scripts, you clone; instead of carefully rolling back transactions, you restore from Time Travel.

Key Takeaways: - Time Travel retention is configurable per object; set longer periods for production critical tables - Transient tables have no Fail-Safe — use them for staging/intermediate data to reduce storage cost - Zero-copy clones share micro-partitions until data diverges — creation is always instant - `UNDROP` restores tables, schemas, and databases within the Time Travel window - Always clone before risky transformations; the overhead is near zero

What's Next: Chapter 10 explores how to share live Snowflake data across account boundaries without moving or copying data.

Chapter 10: Data Sharing & Collaboration

10.1 The Snowflake Data Sharing Architecture

Traditional data sharing involves ETL pipelines, file exports, FTP transfers, or API integrations — all of which create data copies that go stale and require ongoing maintenance. Snowflake's Secure Data Sharing works differently: the provider grants access to their actual live data objects (tables, secure views, secure materialized views). The consumer queries the provider's data directly — there is no copy, no export, and no ETL. The data is always current.

The mechanism works because both provider and consumer accounts exist on the same cloud region's Snowflake metadata and storage layer. The provider grants read access through a Share object; the consumer creates a read-only database from that Share.

Prerequisites:

- Both provider and consumer must be in the same Snowflake region (e.g., both on AWS us-east-1)
- Cross-region sharing is possible via Replication but adds latency
- The consumer account must be a Snowflake account (or a Reader Account — see 10.3)

10.2 Creating and Consuming Secure Shares

```

-- === PROVIDER SIDE ===

-- Step 1: Create the share object
CREATE SHARE product_analytics_share
  COMMENT = 'Product catalog and sales aggregates for partner analytics';

-- Step 2: Grant database and schema usage (required for navigation)
GRANT USAGE ON DATABASE analytics_db TO SHARE product_analytics_share;
GRANT USAGE ON SCHEMA analytics_db.marts TO SHARE product_analytics_share;

-- Step 3: Grant SELECT on specific tables
GRANT SELECT ON TABLE analytics_db.marts.dim_products      TO SHARE
  product_analytics_share;
GRANT SELECT ON TABLE analytics_db.marts.dim_categories    TO SHARE
  product_analytics_share;
GRANT SELECT ON TABLE analytics_db.marts.fct_daily_revenue TO SHARE
  product_analytics_share;

-- Best practice: share Secure Views rather than raw tables
-- to control exactly what columns and rows partners can see
CREATE SECURE VIEW analytics_db.marts.v_partner_revenue AS
SELECT
  order_date,
  product_category,
  region,
  SUM(amount)      AS total_revenue,
  COUNT(order_id) AS order_count
FROM analytics_db.marts.fct_orders
WHERE status = 'COMPLETED'
GROUP BY 1, 2, 3;

-- Must be a SECURE view (not standard view) to share
GRANT SELECT ON VIEW analytics_db.marts.v_partner_revenue TO SHARE
  product_analytics_share;

-- Step 4: Add the consumer Snowflake account to the share
ALTER SHARE product_analytics_share
  ADD ACCOUNTS = 'partnerorg.partneraccount';

-- Add multiple accounts at once
ALTER SHARE product_analytics_share
  ADD ACCOUNTS = 'partnerorg.partneraccount', 'clientorg.clientaccount';

-- Remove an account from the share
ALTER SHARE product_analytics_share
  REMOVE ACCOUNTS = 'partnerorg.partneraccount';

-- Inspect the share
SHOW SHARES;
SHOW GRANTS TO SHARE product_analytics_share;
DESCRIBE SHARE product_analytics_share;

-- === CONSUMER SIDE ===

```

```

-- Step 1: View available inbound shares
SHOW SHARES;

-- Step 2: Create a database from the share (read-only, no warehouse needed for
          metadata)
CREATE DATABASE partner_product_data
  FROM SHARE providerorg.provideraccount.product_analytics_share
  COMMENT = 'Partner product catalog - read-only shared data';

-- Step 3: Grant access to the shared database to local roles
GRANT IMPORTED PRIVILEGES ON DATABASE partner_product_data TO ROLE
  data_analyst_role;

-- Step 4: Query shared data - always live, never stale
SELECT
  product_category,
  SUM(total_revenue) AS category_revenue,
  SUM(order_count) AS category_orders
FROM partner_product_data.marts.v_partner_revenue
WHERE order_date ≥ DATEADD(DAY, -30, CURRENT_DATE())
GROUP BY product_category
ORDER BY category_revenue DESC;

```

10.3 Snowflake Marketplace

The Snowflake Marketplace is a data exchange where organizations publish datasets for discovery by the broader Snowflake ecosystem. Some listings are free; others are paid (Snowflake handles billing). Providers can offer:

- **Free listings:** weather data, geospatial data, financial indices
- **Paid listings:** proprietary datasets, third-party enrichment data
- **Private listings:** share with specific accounts, not public

```

-- As a consumer: after clicking "Get Data" in Snowsight Marketplace,
-- the dataset appears as a database in your account
-- Example: Knoema economic dataset
SELECT *
FROM knoema_economy_data_atlas.datasets.world_gdp_indicator
WHERE "Country Name" = 'United States'
ORDER BY "Year" DESC
LIMIT 10;

-- Monitor your Marketplace activity
SELECT *
FROM snowflake.data_sharing_usage.listing_events_daily
WHERE event_date ≥ DATEADD(DAY, -30, CURRENT_DATE())
ORDER BY event_date DESC;

-- As a provider: create a listing via Snowsight (Data Products → Marketplace → +
-- Listing)
-- Or check existing listings
SHOW LISTINGS;

-- View listing access events
SELECT
    listing_name,
    event_type,
    consumer_account_name,
    event_date,
    total_queries
FROM snowflake.data_sharing_usage.listing_events_daily
WHERE event_date ≥ DATEADD(DAY, -30, CURRENT_DATE())
ORDER BY event_date DESC;

```

10.4 Reader Accounts

Reader Accounts allow you to share data with organizations that do not have their own Snowflake account. Snowflake creates a managed account; the provider pays the compute costs when the reader queries the data.

```
-- Create a reader account (managed account)
CREATE MANAGED ACCOUNT external_partner_reader
  ADMIN_NAME      = 'partner_admin'
  ADMIN_PASSWORD  = 'TemporaryPass123!' -- partner must change on first login
  TYPE            = READER
  COMMENT         = 'Reader account for Acme Corp partner integration';

-- Retrieve the login URL and account identifier for the reader account
SHOW MANAGED ACCOUNTS;
-- Note the cloud, region, locator, and URL from the output

-- Add the reader account to an existing share
ALTER SHARE product_analytics_share
  ADD ACCOUNTS = 'myorg.external_partner_reader';

-- Monitor reader account query consumption (you pay for this)
SELECT *
FROM snowflake.account_usage.warehouse_metering_history
WHERE account_name = 'EXTERNAL_PARTNER_READER';
```

10.5 Snowflake Data Clean Rooms

Data Clean Rooms enable privacy-preserving data collaboration — two or more parties can analyze combined datasets without either party seeing the other's raw data. Snowflake's Native App-based Clean Room enforces query restrictions, minimum aggregation thresholds, and approved query patterns defined by each contributing party.

```

-- Install the Snowflake Data Clean Room native app from Marketplace
-- Once installed, create a clean room as the provider

-- Provider defines: what data to contribute, which queries are approved
-- Consumer runs only the approved queries on the combined dataset

-- Example: an advertiser and a retailer want to measure ad campaign
-- effectiveness
-- Retailer contributes: customer purchase data
-- Advertiser contributes: ad impression and click data
-- Neither can see the other's raw rows; only approved aggregate queries are
-- allowed

-- Example approved query template (written by the clean room provider)
-- Returns overlap count between both parties' customer lists by segment
SELECT
    r.customer_segment,
    COUNT(DISTINCT a.advertiser_customer_id) AS reached_customers,
    COUNT(DISTINCT r.retailer_customer_id) AS converted_customers,
    SUM(r.purchase_amount) AS attributed_revenue
FROM retailer_customers r
JOIN advertiser_impressions a
    ON SHA2(r.email_normalized) = SHA2(a.email_normalized) -- hashed join, no
    raw email exposed
WHERE a.campaign_id = :campaign_id
GROUP BY r.customer_segment
HAVING COUNT(DISTINCT r.retailer_customer_id) ≥ 100;
    -- minimum threshold enforced by clean room
-- Results with fewer than 100 customers are suppressed to prevent re-
-- identification

```

10.6 Replication — Cross-Region and Cross-Cloud

Replication creates synchronized copies of databases across Snowflake regions or cloud providers. Use it for: - Business continuity / disaster recovery - Bringing data closer to users in other regions (lower query latency) - Cross-cloud strategy (primary on AWS, replica on Azure)

```

-- Enable replication on source account (run as ACCOUNTADMIN)
ALTER ACCOUNT ENABLE REPLICATION OF DATABASES TO ACCOUNTS
    aws_us_west_2.disaster_recovery_account;

-- On the primary account: mark the database for replication
ALTER DATABASE analytics_prod ENABLE REPLICATION;

-- On the secondary account: create the replica
CREATE DATABASE analytics_prod_replica
    AS REPLICA OF primaryorg.primaryaccount.analytics_prod;

-- On the secondary account: refresh from primary (can be scheduled)
ALTER DATABASE analytics_prod_replica REFRESH;

-- Set up automatic replication schedule
ALTER DATABASE analytics_prod_replica
    SET REPLICATION_SCHEDULE = '10 MINUTE'; -- refresh every 10 minutes

-- Check replication lag
SELECT
    phase_name,
    start_time,
    end_time,
    DATEDIFF('SECOND', start_time, end_time) AS duration_seconds
FROM TABLE(information_schema.database_replication_usage_history(
    DATE_RANGE_START => DATEADD(DAY, -1, CURRENT_TIMESTAMP()),
    DATABASE_NAME => 'ANALYTICS_PROD_REPLICA'
));

```

Chapter 10 Summary

Snowflake's sharing model eliminates the data pipeline overhead traditionally required to share datasets between organizations. Secure Data Sharing provides live, read-only access to actual production data with no copying. The Marketplace extends this to the global Snowflake ecosystem. Reader Accounts democratize access for organizations without Snowflake subscriptions. Data Clean Rooms enable privacy-safe multi-party collaboration. Together, these features position Snowflake as a data exchange platform, not just a data warehouse.

Key Takeaways: - Shares provide live data access — consumers always query the provider's most current data - Share Secure Views, not raw tables, to control what partners can see - Reader Accounts let you share with non-Snowflake customers; you pay their

compute costs - Data Clean Rooms enforce query restrictions programmatically — privacy by construction - Replication handles cross-region and cross-cloud redundancy with configurable lag

What's Next: Chapter 11 introduces Snowpark — writing Python, Java, and Scala code that runs inside Snowflake's compute engine, eliminating data movement for data engineering and machine learning workloads.

Chapter 11: Snowpark — Code in the Warehouse

11.1 What Is Snowpark and Why Does It Matter?

Traditional data engineering involves a painful extraction step: pull data from the warehouse into a local Python process (pandas DataFrame), transform it, then write it back. For large datasets this means expensive network transfer, memory pressure on your laptop or server, and coordination between two systems.

Snowpark eliminates this by letting you write code in Python, Java, or Scala that compiles to a query plan and executes **inside Snowflake's compute engine**. The data never leaves Snowflake. You write familiar Python syntax; Snowflake executes it at scale.

Snowpark enables:

- **DataFrames**: lazy, SQL-backed DataFrames that execute inside Snowflake
- **User-Defined Functions (UDFs)**: scalar and vectorized, written in Python/Java/Scala
- **User-Defined Table Functions (UDTFs)**: return multiple rows per invocation
- **Stored Procedures**: complex multi-step logic orchestrated in Python
- **Snowpark ML**: scikit-learn-compatible ML pipeline that trains and scores inside Snowflake

11.2 Environment Setup

```
# Install Snowpark for Python (Python 3.9-3.11 recommended)
pip install "snowflake-snowpark-python[pandas]"

# Additional packages for ML
pip install "snowflake-ml-python"

# For local development with Anaconda (recommended for package compatibility)
conda create -n snowpark_env python=3.11
conda activate snowpark_env
conda install -c https://repo.anaconda.com/pkgs/snowflake snowflake-snowpark-
python pandas
```

```
# connection.py — centralized connection configuration
from snowflake.snowpark import Session
import os

def get_session() → Session:
    """Create and return a Snowpark session using environment variables."""
    connection_params = {
        "account": os.environ["SNOWFLAKE_ACCOUNT"], # e.g., myorg-myaccount
        "user": os.environ["SNOWFLAKE_USER"],
        "password": os.environ["SNOWFLAKE_PASSWORD"],
        "role": os.environ.get("SNOWFLAKE_ROLE", "DATA_ENGINEER_ROLE"),
        "warehouse": os.environ.get("SNOWFLAKE_WAREHOUSE", "TRANSFORM_WH"),
        "database": os.environ.get("SNOWFLAKE_DATABASE", "ANALYTICS_DB"),
        "schema": os.environ.get("SNOWFLAKE_SCHEMA", "STAGING"),
    }
    session = Session.builder.configs(connection_params).create()
    return session

if __name__ == "__main__":
    session = get_session()
    version = session.sql("SELECT CURRENT_VERSION()").collect()[0][0]
    print(f"Connected to Snowflake {version}")
    session.close()
```

11.3 Snowpark DataFrames

Snowpark DataFrames are lazy — operations are assembled into a query plan and nothing executes until you call an action (`collect()`, `show()`, `write.save_as_table()`, `count()`). This mirrors the Spark and Spark DataFrame paradigm.

```

from snowflake.snowpark import Session
from snowflake.snowpark.functions import (
    col, lit, upper, lower, trim, when, coalesce,
    sum as sum_, avg, count, max as max_, min as min_,
    round as round_, concat, to_date, year, month,
    date_trunc, datediff, regexp_replace, split, get
)
from snowflake.snowpark.types import StringType, IntegerType, FloatType, DateType

session = get_session()

# Load a DataFrame from a Snowflake table
# Nothing executes yet – this is just metadata
customers_df = session.table("customers")

# Inspect schema without reading data
print(customers_df.schema)
# [StructField('CUSTOMER_ID', LongType()), StructField('EMAIL',
#     StringType()), ...]

# Preview data (triggers execution – fetches first N rows)
customers_df.show(5)

# === Filtering and selecting ===
active_customers = (
    customers_df
    .filter(col("is_active") == True)
    .filter(col("country_code").isin(["US", "CA", "GB"]))
    .filter(col("created_at") >= lit("2023-01-01").cast(DateType()))
    .select(
        col("customer_id"),
        col("email"),
        trim(upper(col("full_name"))).alias("full_name_normalized"),
        col("country_code"),
        col("created_at")
    )
    .with_column("email_domain", split(col("email"), lit("@"))[1])
    .with_column(
        "customer_tier",
        when(col("lifetime_spend") >= 10000, lit("PLATINUM"))
        .when(col("lifetime_spend") >= 5000, lit("GOLD"))
        .when(col("lifetime_spend") >= 1000, lit("SILVER"))
        .otherwise(lit("BRONZE"))
    )
)

# Show the execution plan (SQL that will be run)
active_customers.explain()

# Execute and collect results
results = active_customers.collect() # returns list of snowflake.snowpark.Row
print(f"Found {len(results)} active customers")
for row in results[:3]:

```

```

    print(row["CUSTOMER_ID"], row["EMAIL"], row["CUSTOMER_TIER"])

# === Aggregations ===
orders_df = session.table("fct_orders")

revenue_summary = (
    orders_df
    .filter(col("status") == lit("COMPLETED"))
    .group_by("region", date_trunc("MONTH", col("order_date")).alias("month"))
    .agg(
        sum("amount").alias("total_revenue"),
        count("order_id").alias("order_count"),
        avg("amount").alias("avg_order_value"),
        max("amount").alias("max_order_value"),
        count("customer_id").alias("unique_customers")
    )
    .sort(col("month").desc(), col("total_revenue").desc())
)
revenue_summary.show()

# === Joins ===
products_df = session.table("dim_products")

enriched_orders = (
    orders_df
    .join(
        customers_df,
        orders_df["customer_id"] == customers_df["customer_id"],
        join_type="left"
    )
    .join(
        products_df,
        orders_df["product_id"] == products_df["product_id"],
        join_type="inner"
    )
    .select(
        orders_df["order_id"],
        orders_df["order_date"],
        orders_df["amount"],
        orders_df["status"],
        customers_df["country_code"].alias("customer_country"),
        customers_df["customer_segment"],
        products_df["product_name"],
        products_df["category"]
    )
)

# === Writing results back to Snowflake ===

# Overwrite an existing table
revenue_summary.write.mode("overwrite").save_as_table("marts.monthly_revenue_summary")

# Append to an existing table

```

```
new_records_df.write.mode("append").save_as_table("staging.incremental_orders")

# Create if not exists (error if exists)
enriched_orders.write.mode("errorifexists").save_as_table("marts.enriched_orders")

# Write to an external stage as Parquet
active_customers.write.copy_into_location(
    "@analytics_db.public.exports_stage/active_customers/",
    file_format_type="parquet",
    overwrite=True,
    single=False,      # create multiple files for parallelism
    header=True
)

# Convert to pandas DataFrame (pulls data to local memory – use carefully on
# large datasets)
summary_pandas = revenue_summary.to_pandas()
print(summary_pandas.head())
```

11.4 User-Defined Functions (UDFs)

```

from snowflake.snowpark.functions import udf
from snowflake.snowpark.types import StringType, IntegerType, FloatType,
    BooleanType, VariantType

# === Scalar UDF: runs once per row ===
@udf(
    name="classify_email_domain",
    return_type=StringType(),
    input_types=[StringType()],
    is_permanent=False # session-scoped; use is_permanent=True for persistent
                        UDFs
)
def classify_email_domain(email: str) → str:
    """Classify email address as personal, business, or invalid."""
    if not email or '@' not in email:
        return 'invalid'
    domain = email.split('@')[1].lower()
    personal_domains = {
        'gmail.com', 'yahoo.com', 'hotmail.com', 'outlook.com',
        'icloud.com', 'aol.com', 'protonmail.com', 'me.com'
    }
    return 'personal' if domain in personal_domains else 'business'

# Use UDF in a DataFrame operation
customers_with_type = customers_df.with_column(
    "email_type",
    classify_email_domain(col("email"))
)
customers_with_type.show()

# === Permanent UDF: persists in Snowflake, callable from SQL ===
# Must specify a stage to upload the handler code
session.add_packages('snowflake-snowpark-python')

@udf(
    name="analytics_db.public.calculate_credit_score",
    return_type=IntegerType(),
    input_types=[FloatType(), IntegerType(), BooleanType()],
    is_permanent=True,
    replace=True,
    stage_location="@analytics_db.public.udf_code_stage/"
)
def calculate_credit_score(
    payment_history_pct: float,
    account_age_months: int,
    has_derogatory_marks: bool
) → int:
    """Calculate a simplified credit score 300-850."""
    base_score = 300
    # Payment history: up to 350 points
    base_score += int(payment_history_pct * 350)
    # Account age: up to 150 points (max at 120 months / 10 years)
    age_score = min(account_age_months / 120.0, 1.0) * 150

```

```
base_score += int(age_score)
# Derogatory marks: penalty
if has_derogatory_marks:
    base_score = int(base_score * 0.75)
return min(max(base_score, 300), 850) # clamp to 300-850

# Call from SQL directly after registering
# SELECT customer_id, calculate_credit_score(payment_pct, age_months, has_marks)
# FROM customers;
```

11.5 Vectorized (Pandas) UDFs

Vectorized UDFs receive entire batches of rows as pandas Series instead of one row at a time. They are dramatically faster than row-by-row UDFs for numerical operations because they use numpy/pandas vectorized operations under the hood.

```

import pandas as pd
from snowflake.snowpark.functions import pandas_udf
from snowflake.snowpark.types import PandasSeries, FloatType, StringType,
    IntegerType

@pandas_udf(
    return_type=FloatType(),
    input_types=[FloatType(), FloatType(), StringType()]
)
def calculate_shipping_cost(
    weight_kg: pd.Series,
    distance_km: pd.Series,
    shipping_class: pd.Series
) → pd.Series:
    """Calculate shipping cost using vectorized pandas operations."""
    base_rate = pd.Series([0.0] * len(weight_kg))
    base_rate = base_rate.where(shipping_class ≠ 'STANDARD', 0.05)
    base_rate = base_rate.where(shipping_class ≠ 'EXPRESS', 0.12)
    base_rate = base_rate.where(shipping_class ≠ 'OVERNIGHT', 0.25)
    return (weight_kg * 2.5) + (distance_km * base_rate) + 3.99

# Apply vectorized UDF — processes entire batches, very fast
orders_with_shipping = (
    session.table("orders")
    .with_column(
        "shipping_cost",
        calculate_shipping_cost(
            col("package_weight_kg"),
            col("shipping_distance_km"),
            col("shipping_class")
        )
    )
)
orders_with_shipping.show()

```

11.6 User-Defined Table Functions (UDTFs)

UDTFs return zero or more rows per input row — useful for parsing nested structures, generating date ranges, or exploding arrays.

```

from snowflake.snowpark.functions import udtf
from snowflake.snowpark.types import StructType, StructField, StringType,
    IntegerType, DateType
from typing import Iterator, Tuple
import datetime

@udtf(
    output_schema=StructType([
        StructField("date_value", DateType()),
        StructField("day_of_week", StringType()),
        StructField("is_weekend", IntegerType())
    ]),
    input_types=[DateType(), DateType()]
)
class DateRangeGenerator:
    """Generate all dates between start_date and end_date (inclusive)."""

    DAYS = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']

    def process(
        self,
        start_date: datetime.date,
        end_date: datetime.date
    ) → Iterator[Tuple[datetime.date, str, int]]:
        current = start_date
        delta = datetime.timedelta(days=1)
        while current ≤ end_date:
            day_name = self.DAYS[current.weekday()]
            is_weekend = 1 if current.weekday() ≥ 5 else 0
            yield (current, day_name, is_weekend)
            current += delta

# Use the UDTF to generate a date dimension
date_spine_df = session.table_function(
    DateRangeGenerator,
    lit("2024-01-01").cast(DateType()),
    lit("2024-12-31").cast(DateType())
)
date_spine_df.show(10)

```

11.7 Stored Procedures

Stored procedures contain multi-step orchestration logic — they can run DML, call other procedures, handle exceptions, and return results. They run inside Snowflake, so there is no network round-trip for each step.

```

from snowflake.snowpark import Session
from snowflake.snowpark.types import StringType, IntegerType

def run_incremental_load(session: Session, load_date: str) → str:
    """
    Incremental load procedure: loads orders for a specific date from
    raw stage into staging, validates, then promotes to marts.
    """

    try:
        # Step 1: Load raw data for the target date
        session.sql(f"""
            COPY INTO staging.orders_raw (order_id, customer_id, product_id,
            amount, status, order_date)
            FROM (
                SELECT
                    $1::NUMBER,
                    $2::NUMBER,
                    $3::NUMBER,
                    $4::FLOAT,
                    $5::VARCHAR,
                    $6::DATE
                FROM @analytics_db.public.raw_data_stage/orders/
            )
            FILE_FORMAT = (TYPE = CSV, SKIP_HEADER = 1)
            PATTERN = '.*{load_date.replace("-", "")}.*\\.csv'
        """).collect()

        # Step 2: Validate – check for nulls and invalid amounts
        validation = session.sql(f"""
            SELECT
                COUNT(*) AS total_rows,
                SUM(CASE WHEN order_id IS NULL THEN 1 ELSE 0 END) AS
            null_order_ids,
                SUM(CASE WHEN amount ≤ 0 THEN 1 ELSE 0 END) AS invalid_amounts
            FROM staging.orders_raw
            WHERE order_date = '{load_date}'
        """).collect()[0]

        if validation["NULL_ORDER_IDS"] > 0:
            raise ValueError(f"Found {validation['NULL_ORDER_IDS']} rows with
            null order_id")
        if validation["INVALID_AMOUNTS"] > 0:
            raise ValueError(f"Found {validation['INVALID_AMOUNTS']} rows with
            invalid amounts")

        # Step 3: Upsert into marts (MERGE)
        merge_result = session.sql(f"""
            MERGE INTO marts.fct_orders AS target
            USING (
                SELECT
                    o.order_id,
                    o.customer_id,
                    o.product_id,

```

```

        o.amount,
        o.status,
        o.order_date,
        c.country_code,
        c.customer_segment
    FROM staging.orders_raw o
    JOIN staging.customers c ON o.customer_id = c.customer_id
    WHERE o.order_date = '{load_date}'
) AS source ON target.order_id = source.order_id
WHEN MATCHED THEN UPDATE SET
    target.amount          = source.amount,
    target.status          = source.status,
    target.updated_at      = CURRENT_TIMESTAMP()
WHEN NOT MATCHED THEN INSERT (
    order_id, customer_id, product_id, amount, status,
    order_date, country_code, customer_segment, created_at,
    updated_at
) VALUES (
    source.order_id, source.customer_id, source.product_id,
    source.amount,
    source.status, source.order_date, source.country_code,
    source.customer_segment, CURRENT_TIMESTAMP(), CURRENT_TIMESTAMP()
)
""").collect()

rows_inserted = validation["TOTAL_ROWS"]
return f"SUCCESS: Loaded {rows_inserted} rows for {load_date}"

except Exception as e:
    return f"FAILED: {str(e)}"

# Register as a permanent stored procedure
session.sproc.register(
    func=run_incremental_load,
    name="analytics_db.public.run_incremental_load",
    return_type=StringType(),
    input_types=[StringType()],
    is_permanent=True,
    replace=True,
    stage_location="@analytics_db.public.sproc_code_stage/",
    packages=["snowflake-snowpark-python"],
    execute_as="caller"    # runs with the caller's privileges, not owner's
)

# Call from SQL:
# CALL run_incremental_load('2024-01-15');
```

11.8 Snowpark ML

Snowpark ML provides a scikit-learn-compatible API for training and deploying ML models entirely within Snowflake. No data extraction to an external ML platform is required.

```

from snowflake.ml.modeling.preprocessing import (
    StandardScaler, MinMaxScaler, OneHotEncoder, LabelEncoder
)
from snowflake.ml.modeling.ensemble import (
    RandomForestClassifier, GradientBoostingClassifier
)
from snowflake.ml.modeling.linear_model import LogisticRegression
from snowflake.ml.modeling.pipeline import Pipeline
from snowflake.ml.modeling.model_selection import train_test_split, GridSearchCV
from snowflake.ml.modeling.metrics import accuracy_score, f1_score, roc_auc_score
from snowflake.ml.registry import Registry

# Load feature table
features_df = session.table("ml_features.churn_features_v2")

# Split into train/test (executed inside Snowflake)
train_df, test_df = train_test_split(
    features_df,
    test_size=0.2,
    random_state=42,
    label_cols=["CHURNED"]
)

print(f"Training rows: {train_df.count()}, Test rows: {test_df.count()}")

# Define feature columns
categorical_cols = ["country_code", "plan_type", "acquisition_channel"]
numerical_cols = ["tenure_months", "monthly_spend", "support_tickets_30d",
                  "login_frequency_30d", "feature_adoption_score"]
label_col = ["CHURNED"]

# Build output column names for encoded features
cat_out_cols = [f"{c}_enc" for c in categorical_cols]
num_out_cols = [f"{c}_scaled" for c in numerical_cols]

# Define the ML pipeline
pipeline = Pipeline(steps=[
    ("ohe", OneHotEncoder(
        input_cols=categorical_cols,
        output_cols=cat_out_cols,
        handle_unknown="ignore",
        drop_input_cols=True
    )),
    ("scaler", StandardScaler(
        input_cols=numerical_cols,
        output_cols=num_out_cols,
        drop_input_cols=True
    )),
    ("classifier", GradientBoostingClassifier(
        input_cols=cat_out_cols + num_out_cols,
        label_cols=label_col,
        output_cols=["CHURN_PROBABILITY", "CHURNED_PREDICTION"],
    ))
])

```

```

        n_estimators=200,
        max_depth=5,
        learning_rate=0.1,
        random_state=42
    ))
])

# Train entirely inside Snowflake – no data leaves
pipeline.fit(train_df)

# Score the test set
predictions = pipeline.predict(test_df)
predictions.select(
    "CUSTOMER_ID", "CHURNED", "CHURNED_PREDICTION", "CHURN_PROBABILITY"
).show(10)

# Evaluate metrics
accuracy = accuracy_score(df=predictions, y_true_col_names="CHURNED",
                           y_pred_col_names="CHURNED_PREDICTION")
auc = roc_auc_score(df=predictions, y_true_col_names="CHURNED",
                    y_score_col_names="CHURN_PROBABILITY")
f1 = f1_score(df=predictions, y_true_col_names="CHURNED",
              y_pred_col_names="CHURNED_PREDICTION")

print(f"Accuracy: {accuracy:.4f} | AUC: {auc:.4f} | F1: {f1:.4f}")

# Register in Snowflake Model Registry
registry = Registry(session=session, database_name="ML_DB", schema_name="MODELS")

model_version = registry.log_model(
    model=pipeline,
    model_name="customer_churn_model",
    version_name="v2_gradient_boosting",
    metrics={
        "test_accuracy": float(accuracy),
        "test_auc": float(auc),
        "test_f1": float(f1)
    },
    tags={
        "team": "data_science",
        "project": "churn_reduction",
        "training_date": "2024-01-15"
    }
)

print(f"Model registered: {model_version.version_name}")

# Load the model from registry and score new customers
loaded_model = registry.get_model("customer_churn_model").version("v2_gradient_b
oosting")

new_customers_df = session.table("ml_features.new_customers_to_score")
scored = loaded_model.run(new_customers_df, function_name="predict")

# Write predictions back to Snowflake for downstream use

```

```
scored.select("CUSTOMER_ID", "CHURNED_PREDICTION", "CHURN_PROBABILITY") \
    .write.mode("overwrite") \
    .save_as_table("marts.churn_predictions_v2")
```

Chapter 11 Summary

Snowpark fundamentally changes the data engineering workflow by running your Python code inside Snowflake rather than pulling data out. DataFrames provide a familiar API backed by Snowflake SQL, with lazy evaluation ensuring efficient query plans. UDFs and UDTFs extend SQL with custom Python logic. Stored procedures orchestrate complex multi-step pipelines without network round-trips. Snowpark ML brings the entire model training and deployment lifecycle inside Snowflake, eliminating the data extraction step that historically forced ML teams to maintain separate infrastructure.

Key Takeaways: - Snowpark DataFrames are lazy — no execution until an action is called - Vectorized (pandas) UDFs are significantly faster than row-by-row UDFs for numerical operations - `execute_as='caller'` in stored procedures uses the caller's privileges (safer); `execute_as='owner'` uses the procedure owner's privileges (more powerful) - Snowpark ML models trained inside Snowflake can be versioned in the Model Registry and deployed as SQL-callable functions - Always use `is_permanent=True` + a stage location for UDFs and procedures that should survive session end

What's Next: Chapter 12 covers Streamlit in Snowflake — building production-ready data applications that run inside Snowflake with no external infrastructure.

Chapter 12: Streamlit in Snowflake

12.1 What Is Streamlit in Snowflake?

Streamlit is an open-source Python framework that converts Python scripts into interactive web applications with minimal code. Snowflake hosts Streamlit apps natively through **Streamlit in Snowflake (SiS)**, which means:

- **No infrastructure to manage:** no web server, no container, no deployment pipeline
- **Data stays in Snowflake:** apps query Snowflake directly via an active Snowpark session — data never leaves
- **Governed by Snowflake RBAC:** access is controlled by the same roles and privileges as your tables and views
- **Collaborative:** multiple users can access the same app simultaneously; each gets their own session
- **Versioned:** app code is stored in a Snowflake stage; you can roll back by updating the stage file

12.2 Creating Your First Streamlit App

Via Snowsight UI: Navigate to Projects → Streamlit → + Streamlit App. Give it a name, select a warehouse and database/schema, then paste your `app.py` code into the editor.

Via SQL:

```
-- First: upload your app.py to a stage
-- PUT file://./app.py @analytics_db.public.streamlit_stage/sales_dashboard/
  AUTO_COMPRESS=FALSE;

-- Create the Streamlit app
CREATE STREAMLIT analytics_db.public.sales_dashboard
  ROOT_LOCATION = '@analytics_db.public.streamlit_stage/sales_dashboard/'
  MAIN_FILE     = 'app.py'
  QUERY_WAREHOUSE = 'ANALYTICS_WH'
  TITLE        = 'Sales Analytics Dashboard'
  COMMENT      = 'Main BI dashboard for the analytics team';

-- Grant access
GRANT USAGE ON STREAMLIT analytics_db.public.sales_dashboard TO ROLE
  data_analyst_role;

-- List all Streamlit apps
SHOW STREAMLITS;

-- Drop a Streamlit app
DROP STREAMLIT analytics_db.public.old_dashboard;
```

12.3 Complete Sales Dashboard Application

```

# app.py — Production-ready Streamlit in Snowflake sales dashboard
import streamlit as st
import pandas as pd
import altair as alt
from datetime import datetime, timedelta
from snowflake.snowpark.context import get_active_session

# — Session —
# In Streamlit in Snowflake, the session is automatically created with the
# current user's Snowflake context. No credentials needed in the code.
session = get_active_session()

# — Page Configuration —
st.set_page_config(
    page_title="Sales Analytics Dashboard",
    page_icon="📊",
    layout="wide",
    initial_sidebar_state="expanded"
)

# Custom CSS for a cleaner look
st.markdown("""
<style>
    .metric-card {
        background-color: #f8f9fa;
        border-radius: 8px;
        padding: 16px;
        border-left: 4px solid #1f77b4;
    }
    .stMetric label { font-size: 0.85rem; color: #666; }
</style>
""", unsafe_allow_html=True)

st.title("Sales Analytics Dashboard")
st.caption(f"Data refreshed from Snowflake | Current user: {session.get_current_user()}")

# — Sidebar Filters —
with st.sidebar:
    st.header("Dashboard Filters")

    # Date range selector
    date_range_option = st.selectbox(
        "Date Range",
        ["Last 7 Days", "Last 30 Days", "Last 90 Days", "Year to Date", "Custom Range"],
        index=1
    )

    if date_range_option == "Custom Range":
        col1, col2 = st.columns(2)
        start_date = col1.date_input("Start", datetime.now() - timedelta(days=30))

```

```

end_date = col2.date_input("End", datetime.now())
date_filter = f"order_date BETWEEN '{start_date}' AND '{end_date}'"
else:
    offset_map = {
        "Last 7 Days": "DATEADD(DAY, -7, CURRENT_DATE())",
        "Last 30 Days": "DATEADD(DAY, -30, CURRENT_DATE())",
        "Last 90 Days": "DATEADD(DAY, -90, CURRENT_DATE())",
        "Year to Date": "DATE_TRUNC('YEAR', CURRENT_DATE())"
    }
    date_filter = f"order_date ≥ {offset_map[date_range_option]}"

st.divider()

# Region multiselect – populated dynamically from data
@st.cache_data(ttl=300) # cache for 5 minutes
def get_regions():
    return session.sql(
        "SELECT DISTINCT region FROM marts.fct_orders ORDER BY 1"
    ).to_pandas()["REGION"].tolist()

all_regions = get_regions()
selected_regions = st.multiselect("Regions", all_regions, default=all_regions)

if not selected_regions:
    st.warning("Please select at least one region.")
    st.stop()

region_list = "" + "','".join(selected_regions) + ""
region_filter = f"region IN ({region_list})"

st.divider()

# Status filter
selected_statuses = st.multiselect(
    "Order Status",
    ["COMPLETED", "PENDING", "REFUNDED", "CANCELLED"],
    default=["COMPLETED"]
)
status_list = "" + "','".join(selected_statuses) + ""
status_filter = f"status IN ({status_list})"

# Combined WHERE clause
where_clause = f"WHERE {date_filter} AND {region_filter} AND {status_filter}"

# — KPI Metrics —
@st.cache_data(ttl=60) # cache KPIs for 60 seconds
def get_kpis(where: str):
    return session.sql(f"""
        SELECT
            SUM(amount)                AS total_revenue,
            COUNT(DISTINCT order_id)    AS total_orders,
            COUNT(DISTINCT customer_id) AS unique_customers,
    """)

```

```

        AVG(amount)                                AS avg_order_value,
        SUM(CASE WHEN status = 'REFUNDED' THEN amount ELSE 0 END)
        / NULLIF(SUM(amount), 0) * 100              AS refund_rate_pct
    FROM marts.fct_orders
    {where}
    """).to_pandas()

kpi_df = get_kpis(where_clause)
row = kpi_df.iloc[0]

col1, col2, col3, col4, col5 = st.columns(5)
col1.metric("Total Revenue",      f"${row['TOTAL_REVENUE']:,}.0f}")
col2.metric("Total Orders",       f"{row['TOTAL_ORDERS']:,}")
col3.metric("Unique Customers",   f"{row['UNIQUE_CUSTOMERS']:,}")
col4.metric("Avg Order Value",    f"${row['AVG_ORDER_VALUE']:,}.2f}")
col5.metric("Refund Rate",        f"{row['REFUND_RATE_PCT']:.1f}%")

st.divider()

# — Charts: Revenue Trend + Region Breakdown —————
chart_left, chart_right = st.columns([3, 2])

with chart_left:
    st.subheader("Revenue Trend")

    @st.cache_data(ttl=60)
    def get_revenue_trend(where: str):
        return session.sql(f"""
            SELECT
                DATE_TRUNC('DAY', order_date) AS period,
                SUM(amount) AS revenue,
                COUNT(DISTINCT order_id) AS orders,
                COUNT(DISTINCT customer_id) AS customers
            FROM marts.fct_orders
            {where}
            GROUP BY 1
            ORDER BY 1
        """).to_pandas()

    trend_df = get_revenue_trend(where_clause)

    metric_choice = st.radio("Metric", ["Revenue", "Orders", "Customers"],
                              horizontal=True)
    y_col = {"Revenue": "REVENUE", "Orders": "ORDERS", "Customers": "CUSTOMERS"}[
        metric_choice]
    y_fmt = {"Revenue": "$,.0f", "Orders": ",d", "Customers": ",d"}[
        metric_choice]

    trend_chart = alt.Chart(trend_df).mark_area(
        interpolate="monotone",
        line={"color": "#1f77b4", "strokeWidth": 2},
        color=alt.Gradient(
            gradient="linear",
            stops=[

```

```

        alt.GradientStop(color="#1f77b4", offset=0),
        alt.GradientStop(color="rgba(31,119,180,0.05)", offset=1)
    ],
    x1=1, x2=1, y1=1, y2=0
)
).encode(
    x=alt.X("period:T", title="Date", axis=alt.Axis(format="%b %d")),
    y=alt.Y(f"{y_col}:Q", title=metric_choice),
    tooltip=[
        alt.Tooltip("period:T", title="Date", format="%B %d, %Y"),
        alt.Tooltip(f"{y_col}:Q", title=metric_choice, format=y_fmt)
    ]
).properties(height=320)

st.altair_chart(trend_chart, use_container_width=True)

with chart_right:
    st.subheader("By Region")

    @st.cache_data(ttl=60)
    def get_region_breakdown(where: str):
        return session.sql(f"""
            SELECT
                region,
                SUM(amount) AS revenue,
                COUNT(DISTINCT order_id) AS orders
            FROM marts.fct_orders
            {where}
            GROUP BY region
            ORDER BY revenue DESC
        """).to_pandas()

    region_df = get_region_breakdown(where_clause)

    bar_chart = alt.Chart(region_df).mark_bar(cornerRadiusTopLeft=4,
        cornerRadiusTopRight=4).encode(
        x=alt.X("REVENUE:Q", title="Revenue ($)"),
        y=alt.Y("REGION:N", sort="-x", title="Region"),
        color=alt.Color("REGION:N", legend=None, scheme="tableau10"),
        tooltip=[
            alt.Tooltip("REGION:N", title="Region"),
            alt.Tooltip("REVENUE:Q", title="Revenue", format="$, .0f"),
            alt.Tooltip("ORDERS:Q", title="Orders", format=",d")
        ]
    ).properties(height=320)

    st.altair_chart(bar_chart, use_container_width=True)

# ——— Top Products ———
st.subheader("Top 15 Products by Revenue")

@st.cache_data(ttl=60)
def get_top_products(where: str):
    return session.sql(f"""

```

```

SELECT
    p.product_name,
    p.category,
    COUNT(o.order_id)           AS orders,
    SUM(o.amount)               AS revenue,
    AVG(o.amount)               AS avg_order,
    SUM(o.amount) / SUM(SUM(o.amount)) OVER () * 100 AS pct_of_total
FROM marts.fct_orders o
JOIN marts.dim_products p USING (product_id)
{where}
GROUP BY 1, 2
ORDER BY revenue DESC
LIMIT 15
""").to_pandas()

top_df = get_top_products(where_clause)

# Format for display
display_df = top_df.copy()
display_df["REVENUE"] = display_df["REVENUE"].apply(lambda x: f"${x:,.2f}")
display_df["AVG_ORDER"] = display_df["AVG_ORDER"].apply(lambda x: f"${x:,.2f}")
display_df["PCT_OF_TOTAL"] = display_df["PCT_OF_TOTAL"].apply(lambda x: f"{x:.1f}%")
display_df.columns = ["Product", "Category", "Orders", "Revenue", "Avg Order",
                     "% of Total"]

st.dataframe(display_df, use_container_width=True, hide_index=True)

# — Customer Cohort Analysis —
with st.expander("Customer Segment Analysis"):
    @st.cache_data(ttl=300)
    def get_segment_analysis(where: str):
        return session.sql(f"""
            SELECT
                c.customer_segment,
                COUNT(DISTINCT o.customer_id) AS customers,
                COUNT(o.order_id)           AS orders,
                SUM(o.amount)               AS revenue,
                AVG(o.amount)               AS avg_order,
                SUM(o.amount) / COUNT(DISTINCT o.customer_id) AS
revenue_per_customer
            FROM marts.fct_orders o
            JOIN marts.dim_customers c USING (customer_id)
            {where}
            GROUP BY 1
            ORDER BY revenue DESC
        """).to_pandas()

segment_df = get_segment_analysis(where_clause)

seg_chart = alt.Chart(segment_df).mark_circle(opacity=0.8).encode(
    x=alt.X("AVG_ORDER:Q", title="Average Order Value ($)",
    y=alt.Y("REVENUE_PER_CUSTOMER:Q", title="Revenue Per Customer ($)",

```

```

size=alt.Size("CUSTOMERS:Q", title="Customer Count", scale=alt.Scale(range=[100, 2000])),
color=alt.Color("CUSTOMER_SEGMENT:N", title="Segment"),
tooltip=[
    alt.Tooltip("CUSTOMER_SEGMENT:N", title="Segment"),
    alt.Tooltip("CUSTOMERS:Q", title="Customers", format="d"),
    alt.Tooltip("REVENUE:Q", title="Revenue", format="$,.0f"),
    alt.Tooltip("AVG_ORDER:Q", title="Avg Order", format="$,.2f"),
    alt.Tooltip("REVENUE_PER_CUSTOMER:Q", title="Rev/Customer", format="$,.2f")
]
).properties(height=400, title="Customer Segments: Average Order vs Revenue per Customer")

st.altair_chart(seg_chart, use_container_width=True)

# — Raw Data Export —
with st.expander("Export Raw Data"):
    max_rows = st.slider("Maximum rows to export", 100, 10000, 1000, step=100)

    if st.button("Load Data for Export"):
        raw_df = session.sql(f"""
            SELECT
                o.order_id,
                o.order_date,
                o.amount,
                o.status,
                o.region,
                c.full_name      AS customer_name,
                c.customer_segment,
                c.country_code,
                p.product_name,
                p.category
            FROM marts.fct_orders o
            JOIN marts.dim_customers c USING (customer_id)
            JOIN marts.dim_products p USING (product_id)
            {where_clause}
            ORDER BY o.order_date DESC
            LIMIT {max_rows}
        """).to_pandas()

        st.dataframe(raw_df, use_container_width=True)

        csv_data = raw_df.to_csv(index=False).encode("utf-8")
        st.download_button(
            label=f"Download {len(raw_df):,} rows as CSV",
            data=csv_data,
            file_name=f"orders_export_{datetime.now().strftime('%Y%m%d_%H%M')}.csv",
            mime="text/csv"
        )

# — Footer —
st.divider()

```

```
st.caption(
    "Built with Streamlit in Snowflake | "
    f"Warehouse: {session.get_current_warehouse()} | "
    f"Database: {session.get_current_database()}"
)
```

12.4 Working with Multiple Pages

For larger applications, create multiple page files and reference them in the main app:

```
# app.py - main file, acts as navigation hub
import streamlit as st

pages = {
    "Overview": [
        st.Page("pages/sales_overview.py", title="Sales Overview", icon="📊"),
        st.Page("pages/revenue_trend.py", title="Revenue Trend", icon="📈"),
    ],
    "Operations": [
        st.Page("pages/inventory.py", title="Inventory", icon="📦"),
        st.Page("pages/logistics.py", title="Logistics", icon="🚚"),
    ],
    "Administration": [
        st.Page("pages/user_activity.py", title="User Activity", icon="👤"),
    ]
}

pg = st.navigation(pages)
pg.run()
```

12.5 Performance Optimization for SiS Apps

```
import streamlit as st
from snowflake.snowpark.context import get_active_session

session = get_active_session()

# Use @st.cache_data to cache query results
# ttl: time-to-live in seconds (or timedelta)
# show_spinner: whether to show a spinner while loading
@st.cache_data(ttl=300, show_spinner="Loading data...")
def load_summary_data(date_filter: str, region_filter: str) → pd.DataFrame:
    """Cache query results – avoids re-running Snowflake queries on every
    interaction."""
    return session.sql(f"""
        SELECT ...
        FROM marts.fct_orders
        WHERE {date_filter} AND {region_filter}
    """).to_pandas()

# Use st.cache_resource for objects that should be shared across sessions
# (not recommended for session objects, but useful for ML models)
@st.cache_resource
def load_ml_model():
    """Load model from registry once, share across all user sessions."""
    from snowflake.ml.registry import Registry
    registry = Registry(session=session)
    return registry.get_model("churn_model").version("v2")

# Avoid calling session.sql() in loops – batch your queries
# Bad: N separate Snowflake round trips
for product_id in product_ids:
    df = session.sql(f"SELECT * FROM products WHERE id = {product_id}").to_pandas()

# Good: single query with IN clause
ids_str = ",".join(str(i) for i in product_ids)
df = session.sql(f"SELECT * FROM products WHERE id IN ({ids_str})").to_pandas()

# Use Snowflake to do the heavy lifting – filter and aggregate in SQL, not pandas
# Bad: pull all data to pandas then filter
all_orders = session.table("fct_orders").to_pandas()
filtered = all_orders[all_orders["region"] == "US"] # filter 10M rows in pandas

# Good: filter in Snowflake, pull only what you need
filtered = session.sql("""
    SELECT * FROM marts.fct_orders WHERE region = 'US' LIMIT 50000
    """).to_pandas()
```

12.6 Deploying and Managing SiS Apps

```
-- Update an app by uploading a new version of app.py to the stage
-- PUT file://./app.py @analytics_db.public.streamlit_stage/sales_dashboard/
--     AUTO_COMPRESS=FALSE OVERWRITE=TRUE;

-- The app reloads automatically when users refresh their browser

-- Grant app access to roles
GRANT USAGE ON STREAMLIT analytics_db.public.sales_dashboard TO ROLE
    data_analyst_role;
GRANT USAGE ON STREAMLIT analytics_db.public.sales_dashboard TO ROLE
    data_scientist_role;

-- The app runs queries using the user's current role – row access policies
-- and masking policies apply transparently. A user who cannot query a table
-- directly also cannot query it through the Streamlit app.

-- Revoke access
REVOKE USAGE ON STREAMLIT analytics_db.public.sales_dashboard FROM ROLE
    data_viewer_role;

-- Inspect app configuration
SHOW STREAMLITS IN DATABASE analytics_db;
DESCRIBE STREAMLIT analytics_db.public.sales_dashboard;

-- Monitor Streamlit app usage
SELECT
    event_timestamp,
    user_name,
    event_type,
    event_data
FROM snowflake.account_usage.access_history
WHERE object_name = 'SALES_DASHBOARD'
AND object_type = 'STREAMLIT'
ORDER BY event_timestamp DESC
LIMIT 50;
```

12.7 Connecting External Data Sources

```
# Use External Access Integrations to call external APIs from SiS
# (requires ACCOUNTADMIN to set up the integration first)

import streamlit as st
import requests
from snowflake.snowpark.context import get_active_session
import _snowflake
import json

session = get_active_session()

st.title("Customer Enrichment App")

customer_id = st.text_input("Enter Customer ID")

if st.button("Enrich Customer Data") and customer_id:
    # Query internal Snowflake data
    customer_row = session.sql(f"""
        SELECT customer_id, email, country_code, lifetime_spend
        FROM marts.dim_customers
        WHERE customer_id = {customer_id}
    """).collect()

    if customer_row:
        customer = customer_row[0]
        st.json({
            "customer_id": customer["CUSTOMER_ID"],
            "email": customer["EMAIL"],
            "country_code": customer["COUNTRY_CODE"],
            "lifetime_spend": customer["LIFETIME_SPEND"]
        })

    # Call external enrichment API (using a secret for credentials)
    api_creds = json.loads(_snowflake.get_generic_secret_string('enrichment_
    api_creds'))
    response = requests.get(
        f"{api_creds['endpoint']}/enrich",
        params={"email": customer["EMAIL"]},
        headers={"Authorization": f"Bearer {api_creds['api_key']}"},
        timeout=10
    )
    if response.ok:
        st.subheader("Enriched Data")
        st.json(response.json())
    else:
        st.error(f"Customer {customer_id} not found.")
```

Chapter 12 Summary

Streamlit in Snowflake eliminates the gap between data analysis and data presentation. Because apps run inside Snowflake's infrastructure, there is no deployment pipeline, no separate server to manage, and no need to extract data to an external application layer. Snowflake's RBAC governs app access, and data policies (masking, row access) apply transparently when app users query data — the app cannot bypass security that its user's role cannot bypass directly.

Key Takeaways: - `get_active_session()` gives the app a Snowflake session automatically — no credentials in code - Use `@st.cache_data(ttl=N)` aggressively to reduce Snowflake query frequency for repeated interactions - Push computation to Snowflake SQL — never pull large DataFrames to pandas and filter in Python - RBAC and data policies apply to Streamlit app queries exactly as they do to direct SQL queries - Multiple page apps use `st.Page()` for organized navigation

What's Next: Part 3 of this course covers Snowflake Cortex AI, Data Governance with Horizon, Cost Management and Resource Monitors, CI/CD with dbt and Terraform, and Operational Monitoring with alerts and dashboards.

Part 2 Recap

You have now covered the six most operationally critical areas of Snowflake:

Chapter	Topic	Core Skill
7	Virtual Warehouses & Performance	Size, cache, prune, cluster
8	Security & Access Control	RBAC, masking, row policies, SSO
9	Time Travel & Cloning	Recovery, safety snapshots, dev environments
10	Data Sharing & Collaboration	Shares, Marketplace, Clean Rooms
11	Snowpark	Python DataFrames, UDFs, stored procedures, ML
12	Streamlit in Snowflake	Data applications, caching, governance

These capabilities work together in production systems. A typical architecture: - **Snowpark** loads and transforms raw data into structured marts - **RBAC + masking policies** protect PII at rest and in query results - **Clustering keys** ensure analytics queries run efficiently - **Time Travel** provides recovery from operational errors - **Secure Data Sharing** distributes curated data to partners without duplication - **Streamlit in Snowflake** surfaces insights to business users without exposing underlying data models

Proceed to **Part 3** for Cortex AI, governance, cost management, and DevOps. # Snowflake Master Course — Part 3: AI, Governance, DevOps & Enterprise Architecture

Course Structure - Part 1 (Chapters 1–6): Foundations — architecture, setup, data loading, core SQL - Part 2 (Chapters 7–12): Engineering — warehouses, security, Time Travel, data sharing, Snowpark, Streamlit - **Part 3 (Chapters 13–18): Advanced — Cortex AI, governance, cost management, DevOps, monitoring, enterprise patterns**

Table of Contents

- [Chapter 13: Snowflake Cortex AI](#)
 - [Chapter 14: Data Governance with Snowflake Horizon](#)
 - [Chapter 15: Cost Management and Optimization](#)
 - [Chapter 16: Enterprise Architecture and DevOps](#)
 - [Chapter 17: Monitoring and Observability](#)
 - [Chapter 18: Advanced Topics and Enterprise Patterns](#)
-

Chapter 13: Snowflake Cortex AI

13.1 Introduction to Cortex AI

Snowflake Cortex is a suite of AI and machine learning capabilities that run natively inside Snowflake. This is architecturally significant: your data never leaves Snowflake's security perimeter, there is no external model infrastructure to provision or manage, and billing is consumption-based (per token or per inference). You write SQL or Python, and Snowflake handles everything else.

Cortex sits at the intersection of two trends: the commoditization of large language models and the maturity of the cloud data warehouse. Rather than exporting data to an external ML platform, Cortex brings the models to where the data already lives.

Core Cortex Components

Component	Purpose
Cortex LLM Functions	SQL functions wrapping hosted LLMs: COMPLETE, SUMMARIZE, SENTIMENT, TRANSLATE, EXTRACT_ANSWER, CLASSIFY_TEXT
Cortex Search	Hybrid semantic + keyword search over unstructured data stored in Snowflake
Cortex Analyst	Natural language to SQL via a governed semantic model
Document AI	Extract structured fields from PDFs and images using a trained extraction model
Cortex ML Functions	No-code ML: forecasting, anomaly detection, classification, contribution analysis

Availability and Pricing

Cortex LLM functions are available on Business Critical and Enterprise editions. Pricing is per 1,000 tokens — both input and output tokens count. As of 2025, Snowflake hosts a range of models including proprietary (Snowflake Arctic) and open-weight models (Llama, Mistral, Mixtral, Gemma, Reka). Model availability varies by cloud region; check `SHOW AVAILABLE MODELS IN CORTEX` for your account.

13.2 LLM Functions

LLM functions are scalar SQL functions in the `SNOWFLAKE.CORTEX` schema. They are vectorized — you can call them on every row of a table in a single SQL statement.

COMPLETE — General-Purpose LLM

`COMPLETE` is the most flexible function. You pass a model name, a prompt (string or structured messages array), and optional configuration.

```
-- Simple string prompt: summarize customer feedback
SELECT
  review_id,
  SNOWFLAKE.CORTEX.COMPLETE(
    'llama3.1-70b',
    'You are a data analyst. Summarize this customer feedback in one sentence: ' || customer_feedback
  ) AS ai_summary
FROM customer_reviews
LIMIT 10;
```

For fine-grained control over model behavior, use the structured messages format with a system prompt and optional configuration object:

```
-- Structured messages format with temperature control
SELECT
  ticket_id,
  SNOWFLAKE.CORTEX.COMPLETE(
    'mistral-7b',
    [
      {
        'role': 'system',
        'content': 'You are an expert at classifying customer support
tickets. Always respond with a valid JSON object and nothing else. The
JSON must have keys: category, priority, and sentiment.'
      },
      {
        'role': 'user',
        'content': 'Classify this support ticket: ' || ticket_text
      }
    ],
    {'temperature': 0.1, 'max_tokens': 150}
  ) AS raw_response
FROM support_tickets
LIMIT 20;
```

Low temperature (0.0–0.2) produces more deterministic outputs, which is important when you need structured JSON responses. High temperature (0.7–1.0) produces more creative, varied outputs.

Parsing Structured JSON from LLM Responses

A common pattern is to prompt the LLM to return JSON and then parse it with

```
TRY_PARSE_JSON :
```

```

WITH classified AS (
  SELECT
    ticket_id,
    ticket_text,
    TRY_PARSE_JSON(
      SNOWFLAKE.CORTEX.COMPLETE(
        'snowflake-arctic',
        'Analyze this support ticket and return ONLY a JSON object with
        exactly these keys: '
        || '{"category": "billing|technical|feature_request|
        account_access|general", '
        || '"priority": "high|medium|low", '
        || '"sentiment": "positive|negative|neutral", '
        || '"requires_escalation": true|false}. '
        || 'Ticket text: ' || ticket_text
      )
    ) AS parsed
  FROM support_tickets
  WHERE LENGTH(ticket_text) > 10
)
SELECT
  ticket_id,
  ticket_text,
  parsed:category::VARCHAR AS category,
  parsed:priority::VARCHAR AS priority,
  parsed:sentiment::VARCHAR AS sentiment,
  parsed:requires_escalation::BOOLEAN AS requires_escalation
FROM classified
WHERE parsed IS NOT NULL -- TRY_PARSE_JSON returns NULL on invalid JSON
ORDER BY
  CASE parsed:priority::VARCHAR WHEN 'high' THEN 1 WHEN 'medium' THEN 2 ELSE 3
  END;

```

SUMMARIZE

SUMMARIZE is purpose-built for document summarization. It handles chunking internally and is optimized for long-form text:

```

SELECT
  document_id,
  document_title,
  SNOWFLAKE.CORTEX.SUMMARIZE(document_text) AS summary,
  LENGTH(document_text) AS original_chars,
  LENGTH(SNOWFLAKE.CORTEX.SUMMARIZE(document_text)) AS summary_chars
FROM legal_documents
WHERE LENGTH(document_text) > 1000
ORDER BY original_chars DESC;

```

SENTIMENT

`SENTIMENT` returns a float between -1.0 (very negative) and 1.0 (very positive). It is suitable for bulk sentiment scoring of reviews, tickets, and social media data:

```
SELECT
  review_id,
  product_id,
  review_date,
  review_text,
  SNOWFLAKE.CORTEX.SENTIMENT(review_text) AS sentiment_score,
  CASE
    WHEN SNOWFLAKE.CORTEX.SENTIMENT(review_text) > 0.3 THEN 'Positive'
    WHEN SNOWFLAKE.CORTEX.SENTIMENT(review_text) < -0.3 THEN 'Negative'
    ELSE 'Neutral'
  END AS sentiment_label
FROM product_reviews
WHERE review_date ≥ DATEADD(DAY, -30, CURRENT_DATE());
```

Performance note: Calling `SENTIMENT` twice on the same column computes it twice. Use a CTE or lateral join to compute once and reference multiple times.

```
-- Efficient: compute once in a CTE
WITH scored AS (
  SELECT
    review_id,
    product_id,
    review_text,
    SNOWFLAKE.CORTEX.SENTIMENT(review_text) AS score
  FROM product_reviews
)
SELECT
  product_id,
  COUNT(*) AS review_count,
  AVG(score) AS avg_sentiment,
  COUNT_IF(score > 0.3) AS positive_count,
  COUNT_IF(score BETWEEN -0.3 AND 0.3) AS neutral_count,
  COUNT_IF(score < -0.3) AS negative_count
FROM scored
GROUP BY product_id
ORDER BY avg_sentiment ASC; -- worst-rated products first
```

TRANSLATE

`TRANSLATE` translates text from a source language to a target language using ISO 639-1 language codes:

```
-- Translate French support tickets to English for routing
SELECT
    support_ticket_id,
    ticket_text AS original_text,
    SNOWFLAKE.CORTEX.TRANSLATE(ticket_text, 'fr', 'en') AS english_translation,
    submitted_at
FROM support_tickets
WHERE detected_language = 'fr'
AND submitted_at ≥ DATEADD(DAY, -7, CURRENT_TIMESTAMP())
ORDER BY submitted_at DESC;

-- Multi-language batch translation into English
SELECT
    doc_id,
    detected_language,
    SNOWFLAKE.CORTEX.TRANSLATE(doc_text, detected_language, 'en') AS english_text
FROM multilingual_documents
WHERE detected_language ≠ 'en'
AND detected_language IN ('fr', 'de', 'es', 'it', 'pt', 'nl', 'ja', 'zh', 'ko');
```

EXTRACT_ANSWER

`EXTRACT_ANSWER` performs question answering over a provided context. It returns a JSON object with `answer` and `score` (confidence) fields:

```

-- Extract specific facts from contract text
SELECT
  contract_id,
  PARSE_JSON(
    SNOWFLAKE.CORTEX.EXTRACT_ANSWER(
      contract_text,
      'What is the payment term?'
    )
  ):answer::VARCHAR AS payment_term,
  PARSE_JSON(
    SNOWFLAKE.CORTEX.EXTRACT_ANSWER(
      contract_text,
      'What is the total contract value?'
    )
  ):answer::VARCHAR AS contract_value,
  PARSE_JSON(
    SNOWFLAKE.CORTEX.EXTRACT_ANSWER(
      contract_text,
      'What is the contract effective date?'
    )
  ):answer::VARCHAR AS effective_date
FROM contracts
WHERE contract_status = 'ACTIVE';

```

CLASSIFY_TEXT

`CLASSIFY_TEXT` maps text to one of your provided candidate labels. It returns the winning label and a confidence score:

```

-- Classify support tickets into categories
SELECT
  ticket_id,
  ticket_text,
  SNOWFLAKE.CORTEX.CLASSIFY_TEXT(
    ticket_text,
    ['billing', 'technical_issue', 'feature_request', 'account_access', 'general_inquiry']
  ):label::VARCHAR AS predicted_category,
  SNOWFLAKE.CORTEX.CLASSIFY_TEXT(
    ticket_text,
    ['billing', 'technical_issue', 'feature_request', 'account_access', 'general_inquiry']
  ):score::FLOAT AS confidence_score
FROM support_tickets
WHERE processed_at IS NULL
ORDER BY confidence_score ASC; -- low-confidence tickets need human review

```

13.3 Cortex Search — Semantic Search

Cortex Search creates a managed search service over your Snowflake tables. Unlike keyword search, it uses embeddings to find semantically similar results — “noise canceling headphones” will match “audio devices that block ambient sound” even without shared keywords.

Creating a Cortex Search Service

```
-- Create a search service over the product catalog
-- The AS clause defines the query that populates the search index
CREATE OR REPLACE CORTEX SEARCH SERVICE product_search_service
  ON product_name, product_description, category
  WAREHOUSE = analytics_wh
  TARGET_LAG = '1 hour'
AS (
  SELECT
    product_id,
    sku,
    product_name,
    product_description,
    category,
    subcategory,
    price,
    is_active
  FROM marts.dim_products
  WHERE is_active = TRUE
);
```

`TARGET_LAG` controls how stale the index can be. Shorter lag means fresher results but more serverless compute cost.

Querying via Python (Snowpark)

```
from snowflake.core import Root

# session is an active Snowpark Session
root = Root(session)

search_service = (
    root
    .databases["analytics_db"]
    .schemas["public"]
    .cortex_search_services["product_search_service"]
)

results = search_service.search(
    query="wireless noise canceling headphones under $200",
    columns=["product_id", "product_name", "product_description", "price", "category"],
    filter={"@eq": {"is_active": True}},
    limit=5
)

for item in results.results:
    print(f"{item['product_name']}: ${item['price']:.2f}")
```

Querying via SQL (Preview)

```
-- Call Cortex Search from SQL using SEARCH_PREVIEW
SELECT
  r.value:product_id::VARCHAR      AS product_id,
  r.value:product_name::VARCHAR    AS product_name,
  r.value:price::FLOAT             AS price,
  r.value:category::VARCHAR        AS category
FROM TABLE(
  FLATTEN(
    INPUT => PARSE_JSON(
      SNOWFLAKE.CORTEX.SEARCH_PREVIEW(
        'analytics_db.public.product_search_service',
        '{
          "query": "wireless noise canceling headphones",
          "columns": ["product_id", "product_name", "price",
            "category"],
          "limit": 5
        }'
      )
    ):results
  )
) r;
```

13.4 Document AI — Structured Extraction from PDFs

Document AI lets you train a custom extraction model on sample documents (labeled in the Snowsight UI), then call that model to extract structured fields from new documents at scale.

Workflow

1. Create a stage and upload representative sample PDFs or images.
2. Label the documents in Snowsight: highlight text and assign field names (e.g., `invoice_number`, `vendor_name`, `total_amount`).
3. Build and publish the model.
4. Call it via SQL on new documents.

Processing Documents at Scale

```
-- Process a single document
SELECT
  SNOWFLAKE.CORTEX.EXTRACT_DOCUMENT(
    BUILD_SCOPED_FILE_URL(@invoice_stage, 'invoices/invoice_2024_001.pdf'),
    'invoice_extraction_model',
    ['invoice_number', 'vendor_name', 'total_amount', 'due_date',
     'po_number']
  ) AS extracted_data;

-- Process all PDFs in a stage — bulk extraction
CREATE OR REPLACE TABLE raw.extracted_invoices AS
SELECT
  METADATA$FILENAME                                AS source_filename,
  CURRENT_TIMESTAMP()                              AS extracted_at,
  extracted:invoice_number::VARCHAR                 AS invoice_number,
  extracted:vendor_name::VARCHAR                    AS vendor_name,
  extracted:total_amount::FLOAT                     AS total_amount,
  extracted:due_date::DATE                           AS due_date,
  extracted:po_number::VARCHAR                       AS po_number,
  extracted                                          AS
    full_extracted_payload
FROM (
  SELECT
    METADATA$FILENAME,
    SNOWFLAKE.CORTEX.EXTRACT_DOCUMENT(
      BUILD_SCOPED_FILE_URL(@invoice_stage, METADATA$FILENAME),
      'invoice_extraction_model',
      ['invoice_number', 'vendor_name', 'total_amount', 'due_date', 'po_number']
    ) AS extracted
  FROM DIRECTORY(@invoice_stage)
  WHERE METADATA$FILENAME ILIKE '%.pdf'
);
```

13.5 Cortex Analyst — Natural Language to SQL

Cortex Analyst allows business users to ask questions in plain English and receive SQL-powered answers — without writing any SQL themselves. It is governed by a semantic model (a YAML file) that you define and control.

Defining the Semantic Model

```

# semantic_model.yaml — deploy to a named stage
name: sales_analytics
description: Sales analytics data model for the commercial team
tables:
  - name: fct_orders
    description: One row per customer order transaction
    base_table:
      database: analytics_db
      schema: marts
      table: fct_orders
    dimensions:
      - name: status
        description: Order status (COMPLETED, CANCELLED, PENDING, REFUNDED)
        expr: status
        data_type: TEXT
      - name: region
        description: Geographic sales region (NORTH, SOUTH, EAST, WEST, INTERNATIONAL)
        expr: region
        data_type: TEXT
      - name: product_category
        description: High-level product grouping
        expr: product_category
        data_type: TEXT
      - name: customer_segment
        description: Customer value segment (Bronze, Silver, Gold, Platinum)
        expr: customer_segment
        data_type: TEXT
    time_dimensions:
      - name: order_date
        description: The calendar date the order was placed
        expr: order_date
        data_type: DATE
        default_granularity: DAY
    measures:
      - name: total_revenue
        description: Sum of order amounts in USD
        expr: SUM(amount)
        data_type: NUMBER
        default_aggregate: SUM
      - name: order_count
        description: Total number of orders
        expr: COUNT(order_id)
        data_type: NUMBER
        default_aggregate: COUNT
      - name: unique_customers
        description: Number of distinct customers who placed orders
        expr: COUNT(DISTINCT customer_id)
        data_type: NUMBER
        default_aggregate: COUNT_DISTINCT
      - name: avg_order_value
        description: Average order amount in USD
        expr: AVG(amount)

```

```
data_type: NUMBER  
default_aggregate: AVG
```

Querying via the REST API

```
import requests
import json

def ask_cortex_analyst(question: str, account: str, session_token: str,
                      semantic_model_path: str) → dict:
    """
    Send a natural language question to Cortex Analyst.
    Returns the response including the generated SQL.
    """
    url = f"https://{account}.snowflakecomputing.com/api/v2/cortex/analyst/
message"
    headers = {
        "Authorization": f"Snowflake Token={session_token}",
        "Content-Type": "application/json"
    }
    payload = {
        "messages": [
            {
                "role": "user",
                "content": [{"type": "text", "text": question}]
            }
        ],
        "semantic_model_file": semantic_model_path
    }

    response = requests.post(url, headers=headers, json=payload, timeout=60)
    response.raise_for_status()
    return response.json()

# Example usage
result = ask_cortex_analyst(
    question="What was total revenue last month broken down by region, and which
region grew the most compared to the prior month?",
    account="myorg-myaccount",
    session_token=session.rest.token,
    semantic_model_path="@analytics_db.public.analyst_stage/semantic_model.yaml"
)

# The response includes the generated SQL
message_content = result["message"]["content"]
for item in message_content:
    if item["type"] == "sql":
        print("Generated SQL:")
        print(item["statement"])
    elif item["type"] == "text":
        print("Answer:", item["text"])
```

13.6 Cortex ML Functions — No-Code Machine Learning

Cortex ML Functions provide forecasting, anomaly detection, and classification as first-class SQL objects. No Python, no feature engineering pipeline, no model serving infrastructure required.

Forecasting

```
-- Step 1: Train a demand forecasting model on historical data
CREATE OR REPLACE SNOWFLAKE.ML.FORECAST demand_forecast(
  INPUT_DATA => TABLE(
    SELECT
      order_date,
      product_id,
      daily_units_sold
    FROM marts.daily_product_sales
    WHERE order_date >= DATEADD(DAY, -365, CURRENT_DATE())
    AND order_date < CURRENT_DATE()
  ),
  SERIES_COLNAME => 'product_id',      -- one series per product
  TIMESTAMP_COLNAME => 'order_date',
  TARGET_COLNAME => 'daily_units_sold'
);

-- Step 2: Generate a 30-day forward forecast
CALL demand_forecast!FORECAST(
  FORECASTING_PERIODS => 30,
  CONFIG_OBJECT => {'prediction_interval': 0.95}
);

-- The result set contains: series (product_id), ts (date), forecast,
--                           lower_bound, upper_bound
-- Store it for downstream use
CREATE OR REPLACE TABLE marts.demand_forecast_output AS
SELECT * FROM TABLE(RESULT_SCAN(LAST_QUERY_ID()));

-- Step 3: Inspect model accuracy metrics
CALL demand_forecast!SHOW_EVALUATION_METRICS();
```

Anomaly Detection

```

-- Train an anomaly detector on historical transaction data
-- Unsupervised: no labels needed (learns "normal" behavior)
CREATE OR REPLACE SNOWFLAKE.ML.ANOMALY_DETECTION transaction_anomaly_detector(
  INPUT_DATA ⇒ TABLE(
    SELECT
      transaction_date,
      merchant_category,
      daily_transaction_amount
    FROM marts.daily_merchant_summary
    WHERE transaction_date ≥ DATEADD(DAY, -90, CURRENT_DATE())
  ),
  SERIES_COLNAME ⇒ 'merchant_category',
  TIMESTAMP_COLNAME ⇒ 'transaction_date',
  TARGET_COLNAME ⇒ 'daily_transaction_amount'
);

-- Detect anomalies in recent data
CALL transaction_anomaly_detector!DETECT_ANOMALIES(
  INPUT_DATA ⇒ TABLE(
    SELECT
      transaction_date,
      merchant_category,
      daily_transaction_amount
    FROM marts.daily_merchant_summary
    WHERE transaction_date ≥ DATEADD(DAY, -7, CURRENT_DATE())
  ),
  SERIES_COLNAME ⇒ 'merchant_category',
  TIMESTAMP_COLNAME ⇒ 'transaction_date',
  TARGET_COLNAME ⇒ 'daily_transaction_amount'
);

-- Result columns: series, ts, y (actual), forecast, lower_bound, upper_bound,
-- is_anomaly, percentile, distance
CREATE OR REPLACE TABLE monitoring.transaction_anomalies AS
SELECT *
FROM TABLE(RESULT_SCAN(LAST_QUERY_ID()))
WHERE is_anomaly = TRUE
ORDER BY distance DESC; -- highest distance = most anomalous

```

Classification

```

-- Train a churn prediction classifier
CREATE OR REPLACE SNOWFLAKE.ML.CLASSIFICATION churn_model(
  INPUT_DATA => TABLE(
    SELECT
      tenure_months,
      monthly_spend,
      support_ticket_count_90d,
      login_frequency_30d,
      product_count,
      last_purchase_days_ago,
      churned -- 0 or 1, this is the target
    FROM ml_features.churn_training_set
  ),
  TARGET_COLNAME => 'churned'
);

-- Score new customers: predict churn probability
CALL churn_model!PREDICT(
  INPUT_DATA => TABLE(
    SELECT
      customer_id,
      tenure_months,
      monthly_spend,
      support_ticket_count_90d,
      login_frequency_30d,
      product_count,
      last_purchase_days_ago
    FROM ml_features.active_customers
  ),
  EXPLAIN_COLUMNS => TRUE -- include feature importance scores
);

-- Store predictions
CREATE OR REPLACE TABLE marts.churn_predictions AS
SELECT
  customer_id,
  predicted_churned AS predicted_class,
  predicted_churned_probability['1']::FLOAT AS churn_probability,
  scored_at AS prediction_date
FROM (
  SELECT *, CURRENT_TIMESTAMP() AS scored_at
  FROM TABLE(RESULT_SCAN(LAST_QUERY_ID()))
)
ORDER BY churn_probability DESC;

-- Inspect model performance
CALL churn_model!SHOW_EVALUATION_METRICS();
-- Returns: precision, recall, F1, AUC-ROC for each class

```


Chapter 13 Summary

Cortex AI represents a paradigm shift: instead of moving data to AI infrastructure, you run AI directly on your data. The LLM functions (COMPLETE, SUMMARIZE, SENTIMENT, TRANSLATE, EXTRACT_ANSWER, CLASSIFY_TEXT) handle the most common NLP tasks with a single SQL call. Cortex Search enables semantic search without any vector database. Cortex ML Functions democratize time-series forecasting and anomaly detection with no data science expertise required. And Document AI bridges the gap between unstructured document archives and structured analytics.

Key Takeaways: - All Cortex functions run inside Snowflake — data never leaves your security perimeter. - Use `TRY_PARSE_JSON` when prompting LLMs for structured output — models sometimes produce malformed JSON. - Low temperature (0.1) for classification and extraction; higher temperature for creative or summarization tasks. - Cortex Search combines keyword and semantic (embedding-based) retrieval automatically. - Cortex ML forecasting requires at least 50 training data points per series; more is better.

Chapter 14: Data Governance with Snowflake Horizon

14.1 Snowflake Horizon — Unified Governance

Snowflake Horizon is the governance layer that spans the entire Data Cloud. It is not a single feature but a unified umbrella covering discovery, classification, protection, lineage, and compliance. As data estates grow to thousands of tables across multiple accounts and cloud regions, ad-hoc governance breaks down — Horizon provides the systematic framework.

The Five Pillars of Horizon

Pillar	What It Covers
Classify	Automatic PII and sensitive data detection using ML-based classifiers
Protect	Dynamic data masking, row access policies, column-level encryption, object-level policies
Discover	Universal Search across all Snowflake objects, Marketplace data catalog integration
Monitor	Access history, query history, login auditing, column-level lineage
Govern	Object tagging, tag-based policies, cross-account governance via governance account pattern

14.2 Object Tagging

Tags are key-value metadata that you attach to any Snowflake object — databases, schemas, tables, columns, warehouses, users, or roles. Tags propagate downward: a tag on a database applies to all schemas and tables within it unless overridden.

Tags are the foundation for governance automation: tag-based masking policies, cost allocation reports, compliance dashboards, and data catalog integrations all rely on tags.

```

-- Create tags at the account level
-- ALLOWED_VALUES restricts the tag to a controlled vocabulary
CREATE OR REPLACE TAG pii_category
  ALLOWED_VALUES 'email', 'phone', 'ssn', 'name', 'address', 'date_of_birth', '
    financial';

CREATE OR REPLACE TAG data_classification
  ALLOWED_VALUES 'public', 'internal', 'confidential', 'restricted';

-- Free-form tags (no ALLOWED_VALUES) accept any string
CREATE OR REPLACE TAG data_owner
  COMMENT = 'Email address of the team or person responsible for this data';

CREATE OR REPLACE TAG data_domain
  COMMENT = 'Business domain: commercial, finance, hr, operations, product';

CREATE OR REPLACE TAG cost_center
  COMMENT = 'Internal cost center code for chargeback';

-- Apply tags to columns
ALTER TABLE analytics_db.raw.customers MODIFY COLUMN email
  SET TAG pii_category = 'email';

ALTER TABLE analytics_db.raw.customers MODIFY COLUMN phone
  SET TAG pii_category = 'phone';

ALTER TABLE analytics_db.raw.customers MODIFY COLUMN ssn
  SET TAG pii_category = 'ssn';

ALTER TABLE analytics_db.raw.customers MODIFY COLUMN full_name
  SET TAG pii_category = 'name';

-- Apply tags to tables and schemas
ALTER TABLE analytics_db.raw.customers
  SET TAG data_classification = 'restricted',
    data_owner = 'data-governance@company.com';

ALTER SCHEMA analytics_db.raw
  SET TAG data_domain = 'commercial',
    cost_center = 'CC-1042';

-- Apply tags to warehouses for cost attribution
ALTER WAREHOUSE analytics_wh
  SET TAG cost_center = 'CC-1042',
    data_domain = 'commercial';

-- Query all tagged columns in the account
SELECT
  object_database,
  object_schema,
  object_name,
  column_name,
  tag_name,

```

```
    tag_value
FROM TABLE(SNOWFLAKE.ACCOUNT_USAGE.TAG_REFERENCES_ALL_COLUMNS())
WHERE tag_name = 'PII_CATEGORY'
ORDER BY object_database, object_schema, object_name, column_name;

-- Produce a full PII inventory report
SELECT
    object_database || '.' || object_schema || '.' || object_name AS
        full_table_name,
    column_name,
    tag_value AS pii_type,
    object_schema
FROM TABLE(SNOWFLAKE.ACCOUNT_USAGE.TAG_REFERENCES_ALL_COLUMNS())
WHERE tag_name = 'PII_CATEGORY'
ORDER BY full_table_name, column_name;
```

14.3 Dynamic Data Masking

Dynamic Data Masking (DDM) policies control what different roles see when they query masked columns. The actual data in storage is unchanged — the transformation happens at query time. This is critical for sharing the same table with different audiences: analysts see masked data, production applications see real data.

```

-- Create masking policies
-- Policy for email: show only domain to non-privileged roles
CREATE OR REPLACE MASKING POLICY mask_email AS (val VARCHAR)
  RETURNS VARCHAR →
  CASE
    WHEN CURRENT_ROLE() IN ('PII_DATA_ROLE', 'SYSADMIN') THEN val
    WHEN CURRENT_ROLE() = 'DATA_ANALYST_ROLE' THEN
      '***@' || SPLIT_PART(val, '@', 2) -- show domain only
    ELSE '***REDACTED***'
  END;

-- Policy for SSN: show only last 4 digits
CREATE OR REPLACE MASKING POLICY mask_ssn AS (val VARCHAR)
  RETURNS VARCHAR →
  CASE
    WHEN CURRENT_ROLE() IN ('PII_DATA_ROLE', 'SYSADMIN') THEN val
    ELSE 'XXX-XX-' || RIGHT(val, 4)
  END;

-- Policy for phone: fully redact unless privileged
CREATE OR REPLACE MASKING POLICY mask_phone AS (val VARCHAR)
  RETURNS VARCHAR →
  CASE
    WHEN CURRENT_ROLE() IN ('PII_DATA_ROLE', 'SYSADMIN') THEN val
    ELSE '(XXX) XXX-' || RIGHT(REGEXP_REPLACE(val, '[^0-9]', ''), 4)
  END;

-- Policy for financial data: show null to unauthorized roles
CREATE OR REPLACE MASKING POLICY mask_financial_amount AS (val FLOAT)
  RETURNS FLOAT →
  CASE
    WHEN CURRENT_ROLE() IN ('FINANCE_ROLE', 'SYSADMIN') THEN val
    ELSE NULL
  END;

-- Apply masking policies to columns
ALTER TABLE analytics_db.raw.customers
  MODIFY COLUMN email    SET MASKING POLICY mask_email,
  MODIFY COLUMN phone    SET MASKING POLICY mask_phone,
  MODIFY COLUMN ssn      SET MASKING POLICY mask_ssn;

-- Tag-based masking: apply a policy to all columns tagged as 'email' PII
-- This is the scalable approach – new columns tagged 'email' are automatically
  masked
ALTER TAG analytics_db.public.pii_category
  SET MASKING POLICY mask_email USING (tag_value)
  FOR STRING;

```

14.4 Row Access Policies

Row Access Policies (RAP) restrict which rows a user or role can see. Unlike masking policies (column-level), RAPs operate at the row level. They are evaluated per query and are transparent to the end user — unauthorized rows simply do not appear.

```
-- Create a mapping table: which roles can see which regions
CREATE OR REPLACE TABLE governance.row_access_mappings (
  role_name  VARCHAR,
  region     VARCHAR
);

INSERT INTO governance.row_access_mappings VALUES
  ('NORTH_REGION_ROLE', 'NORTH'),
  ('SOUTH_REGION_ROLE', 'SOUTH'),
  ('EAST_REGION_ROLE', 'EAST'),
  ('WEST_REGION_ROLE', 'WEST'),
  ('GLOBAL_SALES_ROLE', 'NORTH'),
  ('GLOBAL_SALES_ROLE', 'SOUTH'),
  ('GLOBAL_SALES_ROLE', 'EAST'),
  ('GLOBAL_SALES_ROLE', 'WEST'),
  ('SYSADMIN', 'NORTH'),
  ('SYSADMIN', 'SOUTH'),
  ('SYSADMIN', 'EAST'),
  ('SYSADMIN', 'WEST');

-- Create row access policy
CREATE OR REPLACE ROW ACCESS POLICY rap_region_filter AS (order_region VARCHAR)
  RETURNS BOOLEAN →
  EXISTS (
    SELECT 1
    FROM governance.row_access_mappings
    WHERE role_name = CURRENT_ROLE()
    AND region = order_region
  );

-- Apply the policy to the orders table
ALTER TABLE marts.fct_orders
  ADD ROW ACCESS POLICY rap_region_filter ON (region);

-- Now: DATA_ANALYST_ROLE with only NORTH_REGION_ROLE
-- SELECT * FROM marts.fct_orders → only sees rows where region = 'NORTH'
-- GLOBAL_SALES_ROLE → sees all rows
-- SYSADMIN → sees all rows
```

14.5 Access History and Column-Level Lineage

Snowflake records every access to every base object, including which columns were read. This is accessible in `SNOWFLAKE.ACCOUNT_USAGE.ACCESS_HISTORY` with up to 365 days of history.

```

-- Who accessed sensitive columns in the last 7 days?
SELECT
    ah.query_start_time,
    ah.user_name,
    ah.role_name,
    obj.value:objectName::VARCHAR AS table_accessed,
    col.value:columnName::VARCHAR AS column_accessed
FROM SNOWFLAKE.ACCOUNT_USAGE.ACCESS_HISTORY ah,
     LATERAL FLATTEN(INPUT => ah.base_objects_accessed) obj,
     LATERAL FLATTEN(INPUT => obj.value:columns) col
WHERE ah.query_start_time >= DATEADD(DAY, -7, CURRENT_TIMESTAMP())
AND   col.value:columnName::VARCHAR IN ('SSN', 'EMAIL', 'PHONE', 'CREDIT_CARD_NUMBER')
ORDER BY ah.query_start_time DESC;

-- Data lineage: what downstream objects depend on a source table?
-- "objects_modified" shows what was written; "base_objects_accessed" shows what was read
SELECT
    ah.query_start_time,
    ah.user_name,
    src.value:objectName::VARCHAR AS source_table,
    tgt.value:objectName::VARCHAR AS target_table
FROM SNOWFLAKE.ACCOUNT_USAGE.ACCESS_HISTORY ah,
     LATERAL FLATTEN(INPUT => ah.base_objects_accessed) src,
     LATERAL FLATTEN(INPUT => ah.objects_modified) tgt
WHERE ah.query_start_time >= DATEADD(DAY, -30, CURRENT_TIMESTAMP())
AND   src.value:objectName::VARCHAR ILIKE '%raw.customers%'
ORDER BY ah.query_start_time DESC;

-- Login audit: failed logins (potential brute-force detection)
SELECT
    event_timestamp,
    user_name,
    client_ip,
    reported_client_type,
    error_message,
    first_authentication_factor
FROM SNOWFLAKE.ACCOUNT_USAGE.LOGIN_HISTORY
WHERE error_message IS NOT NULL
AND   event_timestamp >= DATEADD(DAY, -7, CURRENT_TIMESTAMP())
ORDER BY event_timestamp DESC;

-- Users without MFA enabled (compliance gap)
SELECT
    name AS user_name,
    login_name,
    has_mfa,
    created_on,
    last_success_login,
    email
FROM SNOWFLAKE.ACCOUNT_USAGE.USERS
WHERE has_mfa = FALSE

```

```

AND disabled = FALSE
AND name NOT IN ('SNOWFLAKE')
ORDER BY last_success_login DESC NULLS LAST;

-- Comprehensive PII access audit report (last 30 days)
WITH pii_accesses AS (
    SELECT
        ah.query_start_time,
        ah.user_name,
        ah.role_name,
        qh.query_text,
        obj.value:objectName::VARCHAR AS table_name,
        col.value:columnName::VARCHAR AS column_name
    FROM SNOWFLAKE.ACCOUNT_USAGE.ACCESS_HISTORY ah
    JOIN SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY qh USING (query_id)
    , LATERAL FLATTEN(INPUT => ah.base_objects_accessed) obj
    , LATERAL FLATTEN(INPUT => obj.value:columns) col
    WHERE ah.query_start_time ≥ DATEADD(DAY, -30, CURRENT_TIMESTAMP())
    AND col.value:columnName::VARCHAR IN ('SSN', 'EMAIL', 'PHONE', 'CREDIT_CARD')
)
SELECT
    TO_DATE(query_start_time) AS access_date,
    user_name,
    role_name,
    table_name,
    column_name,
    COUNT(*) AS access_count
FROM pii_accesses
GROUP BY 1, 2, 3, 4, 5
ORDER BY access_date DESC, access_count DESC;

```

Chapter 14 Summary

Governance is not a one-time project — it is an ongoing operational capability. Snowflake Horizon provides the tools, but the hard work is in the consistent application: tagging every sensitive column, defining masking policies before data goes live, reviewing access history weekly, and building compliance reports into operational dashboards.

Key Takeaways: - Tags are the linchpin of scalable governance — tag-based masking policies scale automatically to new columns. - Dynamic Data Masking operates at query time; the underlying data is never modified. - Row Access Policies are invisible to users — they simply cannot see rows they are not authorized for. -

`SNOWFLAKE.ACCOUNT_USAGE.ACCESS_HISTORY` provides column-level access tracking — invaluable for GDPR, HIPAA, and SOC 2 compliance. - Run `SYSTEM$CLASSIFY_SCHEMA` periodically on new schemas to catch untagged PII.

Chapter 15: Cost Management and Optimization

15.1 The Snowflake Pricing Model

Snowflake charges on three dimensions: compute credits, storage, and data transfer egress.

Compute Credits

Credits measure compute consumption. The price per credit varies by edition (Standard, Enterprise, Business Critical) and by cloud provider and region. Credits are consumed per second of warehouse activity, billed in 60-second minimums per query.

Warehouse Size	Credits/Hour	Nodes
X-Small (XS)	1	1
Small (S)	2	2
Medium (M)	4	4
Large (L)	8	8
X-Large (XL)	16	16
2X-Large	32	32
3X-Large	64	64
4X-Large	128	128

Multi-cluster warehouses multiply credits by the number of active clusters. Serverless features (Snowpipe, Dynamic Tables, Automatic Clustering, Materialized Views, Search Optimization Service) consume credits separately from warehouses.

Storage

- ~\$23/TB/month (on-demand pricing)
- ~\$40/TB/month (capacity pre-purchase, but heavily discounted at scale)
- Time Travel adds to storage: a table with 90 days of Time Travel may consume 1.5–3x the base table storage
- Fail-Safe (7 days, non-configurable for permanent tables) adds additional storage

Data Transfer

Egress from Snowflake's cloud region to the internet or a different cloud region incurs data transfer costs, typically \$0.08–\$0.15/GB depending on provider and region. Intra-region transfers within the same cloud provider are generally free.

15.2 Resource Monitors

Resource Monitors set credit quotas and trigger automated actions (notify, suspend warehouses) when thresholds are crossed. They are essential for preventing runaway queries from generating surprise bills.

```

-- Account-level resource monitor
-- This is a safety net for the entire account
CREATE OR REPLACE RESOURCE MONITOR account_monthly_guard
  WITH CREDIT_QUOTA = 2000 -- 2,000 credits per month
  FREQUENCY = MONTHLY
  START_TIMESTAMP = IMMEDIATELY
  TRIGGERS
    ON 70 PERCENT DO NOTIFY -- email alert at 70%
    ON 90 PERCENT DO NOTIFY -- email alert at 90%
    ON 100 PERCENT DO SUSPEND -- suspend all warehouses at
    100%
    ON 110 PERCENT DO SUSPEND_IMMEDIATE; -- kill running queries at 110%

-- Assign to account (requires ACCOUNTADMIN)
ALTER ACCOUNT SET RESOURCE_MONITOR = account_monthly_guard;

-- Team-specific resource monitor
CREATE OR REPLACE RESOURCE MONITOR analytics_team_monthly
  WITH CREDIT_QUOTA = 300
  FREQUENCY = MONTHLY
  START_TIMESTAMP = IMMEDIATELY
  TRIGGERS
    ON 80 PERCENT DO NOTIFY
    ON 100 PERCENT DO SUSPEND;

-- Assign to specific warehouses
ALTER WAREHOUSE analytics_wh SET RESOURCE_MONITOR = analytics_team_monthly;
ALTER WAREHOUSE reporting_wh SET RESOURCE_MONITOR = analytics_team_monthly;

-- Weekly budget for a high-burn data science warehouse
CREATE OR REPLACE RESOURCE MONITOR ds_weekly_budget
  WITH CREDIT_QUOTA = 100
  FREQUENCY = WEEKLY
  START_TIMESTAMP = '2025-01-06 00:00:00' -- start on a Monday
  TRIGGERS
    ON 80 PERCENT DO NOTIFY
    ON 100 PERCENT DO SUSPEND;

ALTER WAREHOUSE ds_xl_wh SET RESOURCE_MONITOR = ds_weekly_budget;

```

Important: `SUSPEND` only prevents new queries from starting. `SUSPEND_IMMEDIATE` kills currently running queries. Use `SUSPEND_IMMEDIATE` sparingly and only at a high threshold (e.g., 150%) as it can interrupt active user sessions without warning.

15.3 Cost Analysis Queries

Snowflake's `ACCOUNT_USAGE` schema contains rich cost telemetry. Build these queries into a Snowsight dashboard or a dbt model for ongoing visibility.

```

-- Credit usage by warehouse (last 30 days), with estimated USD cost
SELECT
    warehouse_name,
    SUM(credits_used) AS total_credits,
    SUM(credits_used_compute) AS compute_credits,
    SUM(credits_used_cloud_services) AS cloud_svc_credits,
    ROUND(SUM(credits_used) * 3.0, 2) AS est_cost_usd -- adjust $/credit for
    your contract
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
WHERE start_time ≥ DATEADD(DAY, -30, CURRENT_TIMESTAMP())
GROUP BY warehouse_name
ORDER BY total_credits DESC;

-- Daily credit trend (spot spikes)
SELECT
    TO_DATE(start_time) AS usage_date,
    SUM(credits_used) AS daily_credits,
    ROUND(SUM(credits_used) * 3.0, 2) AS est_daily_cost_usd
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
WHERE start_time ≥ DATEADD(DAY, -60, CURRENT_TIMESTAMP())
GROUP BY usage_date
ORDER BY usage_date;

-- Top 20 most expensive individual queries (last 7 days)
SELECT
    query_id,
    user_name,
    warehouse_name,
    ROUND(total_elapsed_time / 1000 / 60, 2) AS elapsed_minutes,
    bytes_scanned / POWER(1024, 3) AS gb_scanned,
    credits_used_cloud_services,
    LEFT(query_text, 200) AS query_preview
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE start_time ≥ DATEADD(DAY, -7, CURRENT_TIMESTAMP())
ORDER BY total_elapsed_time DESC
LIMIT 20;

-- Storage cost by database (last 30 days average)
SELECT
    database_name,
    ROUND(AVG(storage_bytes) / POWER(1024, 3), 2) AS avg_table_storage_gb,
    ROUND(AVG(failsafe_bytes) / POWER(1024, 3), 2) AS avg_failsafe_gb,
    ROUND(AVG(stage_bytes) / POWER(1024, 3), 2) AS avg_stage_gb,
    ROUND(
        AVG(storage_bytes + failsafe_bytes + stage_bytes) / POWER(1024, 4) * 23,
        2
    ) AS est_monthly_cost_usd
FROM SNOWFLAKE.ACCOUNT_USAGE.DATABASE_STORAGE_USAGE_HISTORY
WHERE usage_date ≥ DATEADD(DAY, -30, CURRENT_DATE())
GROUP BY database_name
ORDER BY avg_table_storage_gb DESC;

-- Serverless feature credit usage (Snowpipe, Auto-Clustering, etc.)

```

```

SELECT
    service_type,
    SUM(credits_used) AS total_credits,
    ROUND(SUM(credits_used) * 3.0, 2) AS est_cost_usd
FROM SNOWFLAKE.ACCOUNT_USAGE.METERING_DAILY_HISTORY
WHERE start_time ≥ DATEADD(DAY, -30, CURRENT_TIMESTAMP())
AND service_type IN (
    'AUTO_CLUSTERING',
    'MATERIALIZED_VIEW',
    'SNOWPIPE',
    'DYNAMIC_TABLE',
    'SEARCH_OPTIMIZATION',
    'CORTEX'
)
GROUP BY service_type
ORDER BY total_credits DESC;

-- Monthly credit burn rate: are you on track vs. budget?
WITH daily_burn AS (
    SELECT
        TO_DATE(start_time) AS usage_date,
        SUM(credits_used) AS daily_credits
    FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
    WHERE start_time ≥ DATE_TRUNC('MONTH', CURRENT_DATE())
    GROUP BY usage_date
)
SELECT
    SUM(daily_credits) AS credits_used_mtd,
    DAY(CURRENT_DATE()) AS days_elapsed,
    DAY(LAST_DAY(CURRENT_DATE())) AS days_in_month,
    ROUND(
        SUM(daily_credits) / DAY(CURRENT_DATE())
        * DAY(LAST_DAY(CURRENT_DATE())), 0
    ) AS
        projected_month_end_credits,
    2000 AS
        monthly_budget_credits, -- adjust to your budget
    ROUND(
        (2000 - SUM(daily_credits) / DAY(CURRENT_DATE())
        * DAY(LAST_DAY(CURRENT_DATE())) * 3.0, 0
    ) AS
        projected_overunder_usd
FROM daily_burn;

```

15.4 Cost Optimization Strategies

Warehouse Rightsizing

```
-- Identify warehouses that are consistently oversized
-- Low avg execution time suggests queries don't need the warehouse size
SELECT
    warehouse_name,
    COUNT(*) AS query_count,
    ROUND(AVG(total_elapsed_time) / 1000, 1) AS avg_elapsed_sec,
    ROUND(AVG(compilation_time) / 1000, 1) AS avg_compile_sec,
    ROUND(AVG(execution_time) / 1000, 1) AS avg_execute_sec,
    MAX(warehouse_size) AS warehouse_size
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE start_time ≥ DATEADD(DAY, -14, CURRENT_TIMESTAMP())
AND warehouse_size IS NOT NULL
GROUP BY warehouse_name
HAVING AVG(execution_time) < 5000 -- avg under 5 seconds
AND COUNT(*) > 100 -- meaningful sample size
ORDER BY warehouse_name;
```

Result Cache and Warehouse Queue Analysis

```
-- Result cache effectiveness: what % of queries are served from cache?
SELECT
  ROUND(COUNT_IF(percentage_scanned_from_cache = 100) / COUNT(*) * 100, 1) AS
    result_cache_hit_pct,
  ROUND(COUNT_IF(percentage_scanned_from_cache BETWEEN 1 AND 99) / COUNT(*) * 1
    00, 1) AS partial_cache_pct,
  ROUND(COUNT_IF(percentage_scanned_from_cache = 0) / COUNT(*) * 100, 1) AS
    no_cache_pct,
  COUNT(*) AS total_queries
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE start_time ≥ DATEADD(DAY, -7, CURRENT_TIMESTAMP())
AND   execution_status = 'SUCCESS'
AND   query_type = 'SELECT';

-- Queuing analysis: are users waiting for warehouse slots?
SELECT
  warehouse_name,
  COUNT(*) AS queries_queued,
  AVG(queued_overload_time) / 1000 AS avg_queue_sec
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE start_time ≥ DATEADD(DAY, -7, CURRENT_TIMESTAMP())
AND   queued_overload_time > 0
GROUP BY warehouse_name
ORDER BY avg_queue_sec DESC;

-- High queuing → consider increasing max_cluster_count or warehouse size
```

Storage Optimization

```
-- Tables with Time Travel consuming significant storage (candidates for shorter retention)

SELECT
  table_catalog || '.' || table_schema || '.' || table_name AS full_table_name,
  ROUND(active_bytes / POWER(1024, 3), 2) AS active_gb,
  ROUND(time_travel_bytes / POWER(1024, 3), 2) AS time_travel_gb,
  ROUND(failsafe_bytes / POWER(1024, 3), 2) AS failsafe_gb,
  retention_time
FROM SNOWFLAKE.ACCOUNT_USAGE.TABLE_STORAGE_METRICS
WHERE time_travel_bytes > POWER(1024, 3) -- more than 1GB in Time Travel
ORDER BY time_travel_bytes DESC
LIMIT 20;

-- Reduce Time Travel on high-churn staging tables
-- Transient tables have 0 Fail-Safe and max 1 day Time Travel
ALTER TABLE analytics_db.staging.order_staging
  SET DATA_RETENTION_TIME_IN_DAYS = 1;

-- For volatile intermediate tables: use TRANSIENT (saves up to 50% storage)
CREATE TRANSIENT TABLE analytics_db.staging.session_events_temp (
  session_id      VARCHAR(36),
  event_type      VARCHAR(50),
  event_timestamp  TIMESTAMP_NTZ,
  properties      VARIANT
);
```

Chapter 15 Summary

Cost management is an engineering discipline, not just a finance concern. The most effective Snowflake cost controls are: resource monitors for budget guardrails, warehouse auto-suspend tuned to actual idle patterns (60–120 seconds is typical), transient tables for high-churn intermediate data, and regular query profiling to catch scans of unpartitioned large tables.

Key Takeaways: - Cloud Services credits are free up to 10% of your daily compute credits — monitor the ratio. - Auto-suspend of 60 seconds (not the default 600) saves significant idle credits. - Query result cache is automatic and free — structure repeated BI queries to benefit from it. - Transient tables eliminate Fail-Safe storage costs for data that is easily regenerable. - Resource Monitors are the single most important cost control for multi-team environments.

Chapter 16: Enterprise Architecture and DevOps

16.1 Multi-Account Strategy

Production Snowflake deployments at enterprise scale use multiple accounts for isolation, compliance, and cost attribution. A common pattern:

Snowflake Organization

└─ PROD account	(Business Critical edition, VPC, production data)
└─ STAGING account	(Enterprise edition, full dataset, pre-production)
└─ DEV account	(Enterprise edition, masked/sampled data)
└─ SECURITY account	(centralized audit log aggregation, governance)

Account isolation provides: - Independent security perimeters (separate credentials, network policies, MFA enforcement) - Clear cost attribution by environment - Independent scaling — DEV warehouse changes do not affect PROD - Regulatory compliance: PROD can be Business Critical with HIPAA BAA while DEV does not need to be

Database Replication

```
-- On the PRIMARY (PROD) account: enable replication to STAGING
ALTER DATABASE analytics_db
  ENABLE REPLICATION TO ACCOUNTS myorg.staging_account, myorg.dev_account;

-- On the REPLICA (STAGING) account: create the replica database
CREATE DATABASE analytics_db
  AS REPLICA OF myorg.prod_account.analytics_db
  DATA_RETENTION_TIME_IN_DAYS = 7;

-- Manual refresh
ALTER DATABASE analytics_db REFRESH;

-- Automate refresh via Task (runs daily at 2am UTC)
CREATE OR REPLACE TASK refresh_analytics_replica
  WAREHOUSE = admin_wh
  SCHEDULE = 'USING CRON 0 2 * * * UTC'
AS
  ALTER DATABASE analytics_db REFRESH;

ALTER TASK refresh_analytics_replica RESUME;

-- Check replication progress
SELECT
  phase_name,
  start_time,
  end_time,
  progress,
  details
FROM TABLE(INFORMATION_SCHEMA.DATABASE_REFRESH_PROGRESS('analytics_db'))
ORDER BY start_time;
```

16.2 Infrastructure as Code with Terraform

The Snowflake Terraform provider ([Snowflake-Labs/snowflake](https://github.com/Snowflake-Labs/snowflake)) allows you to manage Snowflake infrastructure declaratively. All databases, warehouses, roles, grants, and integrations can be version-controlled and deployed via CI/CD.

```
# providers.tf
terraform {
  required_providers {
    snowflake = {
      source = "Snowflake-Labs/snowflake"
      version = "~> 0.90"
    }
  }
  required_version = "≥ 1.5"

  backend "s3" {
    bucket = "mycompany-terraform-state"
    key    = "snowflake/production/terraform.tfstate"
    region = "us-east-1"
  }
}

provider "snowflake" {
  account          = var.snowflake_account
  username         = var.snowflake_user
  role             = "SYSADMIN"
  authenticator    = "jwt"
  private_key_path = var.private_key_path
}
```

```
# databases.tf
resource "snowflake_database" "analytics" {
  name                        = "ANALYTICS_DB"
  data_retention_time_in_days = 30
  comment                    = "Primary analytics database – managed by
Terraform"
}

resource "snowflake_schema" "raw" {
  database      = snowflake_database.analytics.name
  name          = "RAW"
  data_retention_time_in_days = 7
  is_transient  = false
}

resource "snowflake_schema" "staging" {
  database      = snowflake_database.analytics.name
  name          = "STAGING"
  data_retention_time_in_days = 14
}

resource "snowflake_schema" "marts" {
  database      = snowflake_database.analytics.name
  name          = "MARTS"
  data_retention_time_in_days = 30
}
```

```
# warehouses.tf
resource "snowflake_warehouse" "transform" {
  name                = "TRANSFORM_WH"
  warehouse_size      = "MEDIUM"
  auto_suspend        = 120
  auto_resume         = true
  initially_suspended = true
  statement_timeout_in_seconds = 3600
  comment              = "dbt transformation workloads"
}

resource "snowflake_warehouse" "analytics" {
  name                = "ANALYTICS_WH"
  warehouse_size      = "LARGE"
  auto_suspend        = 60
  auto_resume         = true
  initially_suspended = true
  min_cluster_count   = 1
  max_cluster_count   = 4
  scaling_policy       = "ECONOMY"
  comment              = "Concurrent BI and ad-hoc analytics – multi-cluster"
}

resource "snowflake_warehouse" "reporting" {
  name                = "REPORTING_WH"
  warehouse_size      = "SMALL"
  auto_suspend        = 300
  auto_resume         = true
  initially_suspended = true
  comment              = "Scheduled reporting and Tableau workloads"
}
```

```

# roles.tf
resource "snowflake_role" "data_engineer" {
  name      = "DATA_ENGINEER_ROLE"
  comment   = "Full read-write on RAW and STAGING schemas; full access to TRANSFORM_WH"
}

resource "snowflake_role" "data_analyst" {
  name      = "DATA_ANALYST_ROLE"
  comment   = "Read-only on MARTS schema; access to ANALYTICS_WH"
}

resource "snowflake_role" "dbt_role" {
  name      = "DBT_ROLE"
  comment   = "Service account role for dbt CI/CD pipeline"
}

# Grant warehouse usage
resource "snowflake_grant_privileges_to_role" "engineer_transform_wh" {
  privileges = ["USAGE", "OPERATE", "MONITOR"]
  role_name  = snowflake_role.data_engineer.name
  on_account_object {
    object_type = "WAREHOUSE"
    object_name = snowflake_warehouse.transform.name
  }
}

# Grant schema privileges
resource "snowflake_grant_privileges_to_role" "analyst_marts_schema" {
  privileges = ["USAGE"]
  role_name  = snowflake_role.data_analyst.name
  on_schema {
    schema_name = "${snowflake_database.analytics.name}.${snowflake_schema.marts.name}"
  }
}

resource "snowflake_grant_privileges_to_role" "analyst_marts_tables" {
  privileges = ["SELECT"]
  role_name  = snowflake_role.data_analyst.name
  on_schema_object {
    all {
      object_type_plural = "TABLES"
      in_schema          = "${snowflake_database.analytics.name}.${snowflake_schema.marts.name}"
    }
  }
}

```

16.3 dbt with Snowflake

dbt (data build tool) is the de facto standard for SQL-based data transformation. It provides modular SQL models, lineage tracking, testing, documentation, and incremental loading — all integrated with Snowflake's features.

```
# dbt_project.yml
name: 'company_analytics'
version: '1.0.0'
config-version: 2

profile: 'company_analytics'

model-paths: ["models"]
test-paths: ["tests"]
seed-paths: ["seeds"]
macro-paths: ["macros"]

target-path: "target"
clean-targets: ["target", "dbt_packages"]

models:
  company_analytics:
    staging:
      +schema:      staging
      +materialized: view
      +tags:        ['staging']
    marts:
      +schema:      marts
      +materialized: table
      +tags:        ['marts']
    core:
      +materialized: table
      +cluster_by:  ['order_date']
      +post-hook:   "GRANT SELECT ON {{ this }} TO ROLE DATA_ANALYST_ROLE"
    finance:
      +materialized: table
      +snowflake_warehouse: analytics_wh
      +grants:
        select: ['FINANCE_ROLE']
```

```
# ~/.dbt/profiles.yml
company_analytics:
  target: dev
  outputs:
    dev:
      type:          snowflake
      account:       "{{ env_var('SNOWFLAKE_ACCOUNT') }}"
      user:          "{{ env_var('SNOWFLAKE_USER') }}"
      private_key_path: "{{ env_var('SNOWFLAKE_PRIVATE_KEY_PATH') }}"
      role:          DBT_ROLE
      database:      ANALYTICS_DB
      warehouse:     TRANSFORM_WH
      schema:        STAGING
      threads:       8
      query_tag:     dbt_dev
      client_session_keep_alive: false

    prod:
      type:          snowflake
      account:       "{{ env_var('SNOWFLAKE_ACCOUNT') }}"
      user:          "{{ env_var('SNOWFLAKE_USER') }}"
      private_key_path: "{{ env_var('SNOWFLAKE_PRIVATE_KEY_PATH') }}"
      role:          DBT_ROLE
      database:      ANALYTICS_DB
      warehouse:     TRANSFORM_WH
      schema:        STAGING
      threads:       16
      query_tag:     dbt_prod
      client_session_keep_alive: false
```

```

-- models/staging/stg_orders.sql
-- Staging models clean and standardize raw data
{{ config(
    materialized='view',
    tags=['staging', 'orders']
) }}

WITH source AS (
    SELECT * FROM {{ source('raw', 'orders') }}
),
cleaned AS (
    SELECT
        order_id::VARCHAR(36) AS order_id,
        customer_id::NUMBER AS customer_id,
        TO_DATE(order_date::VARCHAR) AS order_date,
        UPPER(TRIM(status::VARCHAR)) AS status,
        ROUND(amount::FLOAT, 2) AS amount,
        product_id::VARCHAR(50) AS product_id,
        UPPER(TRIM(region::VARCHAR)) AS region,
        _loaded_at::TIMESTAMP_NTZ AS loaded_at
    FROM source
    WHERE order_id IS NOT NULL
    -- Deduplicate: keep the latest load of each order_id
    QUALIFY ROW_NUMBER() OVER (
        PARTITION BY order_id::VARCHAR
        ORDER BY _loaded_at DESC
    ) = 1
)
SELECT * FROM cleaned

```

```

-- models/marts/fct_orders.sql
-- Incremental mart: only process new/updated records
{{ config(
    materialized='incremental',
    unique_key='order_id',
    incremental_strategy='merge',
    cluster_by=['order_date'],
    tags=['marts', 'orders', 'incremental']
) }}

WITH orders AS (
    SELECT * FROM {{ ref('stg_orders') }}
    {% if is_incremental() %}
        -- Only pick up records loaded since the last dbt run
        WHERE loaded_at > (SELECT MAX(loaded_at) FROM {{ this }})
    {% endif %}
),
customers AS (
    SELECT * FROM {{ ref('stg_customers') }}
),
products AS (
    SELECT * FROM {{ ref('stg_products') }}
)
SELECT
    o.order_id,
    o.order_date,
    o.customer_id,
    c.full_name           AS customer_name,
    c.country_code,
    c.segment             AS customer_segment,
    o.product_id,
    p.product_name,
    p.category            AS product_category,
    o.amount,
    o.status,
    o.region,
    o.loaded_at,
    CURRENT_TIMESTAMP()  AS dbt_updated_at
FROM orders o
LEFT JOIN customers c USING (customer_id)
LEFT JOIN products  p USING (product_id)

```

```
# models/staging/schema.yml – dbt data tests
version: 2

models:
  - name: stg_orders
    description: Cleaned and deduplicated orders from the raw layer
    columns:
      - name: order_id
        description: Unique order identifier
        tests:
          - unique
          - not_null
      - name: status
        tests:
          - not_null
          - accepted_values:
              values: ['COMPLETED', 'CANCELLED', 'PENDING', 'REFUNDED', 'PROCESSING']
      - name: amount
        tests:
          - not_null
          - dbt_expectations.expect_column_values_to_be_between:
              min_value: 0
              max_value: 10000000
      - name: customer_id
        tests:
          - not_null
          - relationships:
              to: ref('stg_customers')
              field: customer_id
```

16.4 Schema Version Control with schemachange

schemachange is a lightweight Python tool that applies versioned SQL migration scripts to Snowflake and tracks what has been deployed. It follows the Flyway naming convention:

```
V<version>__<description>.sql
```

```
-- migrations/V1.0.0__initial_schema.sql
USE ROLE SYSADMIN;
USE DATABASE ANALYTICS_DB;

CREATE SCHEMA IF NOT EXISTS RAW;
CREATE SCHEMA IF NOT EXISTS STAGING;
CREATE SCHEMA IF NOT EXISTS MARTS;

CREATE TABLE IF NOT EXISTS RAW.ORDERS (
  ORDER_ID      VARCHAR(36),
  CUSTOMER_ID   NUMBER,
  ORDER_DATE    DATE,
  AMOUNT        FLOAT,
  STATUS        VARCHAR(50),
  PRODUCT_ID    VARCHAR(50),
  REGION        VARCHAR(50),
  RAW_PAYLOAD   VARIANT,
  _LOADED_AT    TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

CREATE TABLE IF NOT EXISTS RAW.CUSTOMERS (
  CUSTOMER_ID   NUMBER,
  EMAIL          VARCHAR(255),
  FULL_NAME      VARCHAR(200),
  PHONE          VARCHAR(50),
  COUNTRY_CODE   CHAR(2),
  SEGMENT        VARCHAR(50),
  RAW_PAYLOAD   VARIANT,
  _LOADED_AT    TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);
```

```
-- migrations/V1.1.0__add_customer_ltv.sql
USE ROLE SYSADMIN;
USE DATABASE ANALYTICS_DB;

-- Add lifetime value columns to customers
ALTER TABLE RAW.CUSTOMERS ADD COLUMN IF NOT EXISTS LIFETIME_VALUE FLOAT;
ALTER TABLE RAW.CUSTOMERS ADD COLUMN IF NOT EXISTS LTV_CALCULATED_AT
    TIMESTAMP_NTZ;

-- Create customer segment reference table
CREATE TABLE IF NOT EXISTS MARTS.CUSTOMER_SEGMENTS (
    SEGMENT_ID      NUMBER AUTOINCREMENT PRIMARY KEY,
    SEGMENT_NAME    VARCHAR(50) NOT NULL UNIQUE,
    MIN_LTV         FLOAT,
    MAX_LTV         FLOAT,
    DESCRIPTION     VARCHAR(500),
    CREATED_AT      TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

INSERT INTO MARTS.CUSTOMER_SEGMENTS (SEGMENT_NAME, MIN_LTV, MAX_LTV,
    DESCRIPTION) VALUES
    ('Bronze', 0, 999.99, 'New or low-value customers – acquisition focus'),
    ('Silver', 1000, 4999.99, 'Mid-tier customers – growth focus'),
    ('Gold', 5000, 9999.99, 'High-value customers – retention focus'),
    ('Platinum', 10000, NULL, 'Top-tier customers – white-glove service');
```

```

-- migrations/V1.2.0__add_product_catalog.sql
USE ROLE SYSADMIN;
USE DATABASE ANALYTICS_DB;

CREATE TABLE IF NOT EXISTS RAW.PRODUCTS (
  PRODUCT_ID    VARCHAR(50) PRIMARY KEY,
  SKU           VARCHAR(100) UNIQUE,
  PRODUCT_NAME  VARCHAR(500),
  CATEGORY      VARCHAR(100),
  SUBCATEGORY   VARCHAR(100),
  PRICE         FLOAT,
  IS_ACTIVE     BOOLEAN DEFAULT TRUE,
  CREATED_AT    DATE,
  _LOADED_AT    TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

CREATE TABLE IF NOT EXISTS MARTS.DIM_PRODUCTS AS
SELECT
  PRODUCT_ID,
  SKU,
  PRODUCT_NAME,
  CATEGORY,
  SUBCATEGORY,
  PRICE,
  IS_ACTIVE,
  CREATED_AT
FROM RAW.PRODUCTS
WHERE 1 = 0; -- empty table with correct schema

```

```

# schemachange.yml
config-version: 1
root-folder: ./migrations
snowflake-account: myorg-myaccount
snowflake-user: deploy_svc_user
snowflake-role: SYSADMIN
snowflake-warehouse: ADMIN_WH
snowflake-database: ANALYTICS_DB
snowflake-schema: SCHEMACHANGE
change-history-table: ANALYTICS_DB.SCHEMACHANGE.CHANGE_HISTORY
create-change-history-table: true
autocommit: true
verbose: true

```

16.5 GitHub Actions CI/CD Pipeline

```

# .github/workflows/snowflake_cicd.yml
name: Snowflake CI/CD

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

env:
  SNOWFLAKE_ACCOUNT: ${ secrets.SNOWFLAKE_ACCOUNT }
  SNOWFLAKE_USER: ${ secrets.SNOWFLAKE_USER }
  SNOWFLAKE_DATABASE: ANALYTICS_DB
  SNOWFLAKE_WAREHOUSE: TRANSFORM_WH
  SNOWFLAKE_ROLE: DBT_ROLE

jobs:
  # Job 1: Run schema migrations (main branch only)
  schema-migrations:
    name: Schema Migrations
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install schemachange
        run: pip install schemachange==3.6.1

      - name: Deploy schema migrations
        env:
          SNOWFLAKE_PASSWORD: ${ secrets.SNOWFLAKE_SERVICE_PASSWORD }
        run: |
          schemachange \
            --root-folder ./migrations \
            --sf-account $SNOWFLAKE_ACCOUNT \
            --sf-user $SNOWFLAKE_USER \
            --sf-role SYSADMIN \
            --sf-warehouse ADMIN_WH \
            --sf-database $SNOWFLAKE_DATABASE \
            --change-history-table ANALYTICS_DB.SCHEMACHANGE.CHANGE_HISTORY \
            --create-change-history-table

  # Job 2: dbt slim CI on pull requests (only changed models)
  dbt-ci:
    name: dbt CI Check (PR)
    runs-on: ubuntu-latest
    if: github.event_name == 'pull_request'
    steps:

```

```

- uses: actions/checkout@v4

- uses: actions/setup-python@v5
  with:
    python-version: '3.11'

- name: Install dbt-snowflake
  run: pip install dbt-snowflake==1.8.3

- name: Write private key file
  run: |
    echo "${{ secrets.SNOWFLAKE_PRIVATE_KEY }}" > /tmp/rsa_key.p8
    chmod 600 /tmp/rsa_key.p8

- name: dbt deps
  working-directory: ./dbt
  run: dbt deps

- name: Fetch dbt artifacts from production (for slim CI)
  working-directory: ./dbt
  run: |
    mkdir -p target-base
    # Download manifest.json from the last production run
    aws s3 cp s3://my-dbt-artifacts/production/manifest.json ./target-base/
    manifest.json
  env:
    AWS_ACCESS_KEY_ID:      ${{ secrets.AWS_ACCESS_KEY_ID }}
    AWS_SECRET_ACCESS_KEY:  ${{ secrets.AWS_SECRET_ACCESS_KEY }}

- name: dbt build (slim CI – changed models and their downstream
  dependents)
  working-directory: ./dbt
  env:
    SNOWFLAKE_PRIVATE_KEY_PATH: /tmp/rsa_key.p8
    DBT_TARGET: ci
  run: |
    dbt build \
      --select      "state:modified+" \
      --defer \
      --state       ./target-base \
      --target      ci \
      --fail-fast

# Job 3: dbt production deploy (main branch, after migrations)
dbt-deploy:
  name: dbt Production Deploy
  runs-on: ubuntu-latest
  needs: [schema-migrations]
  if: github.ref == 'refs/heads/main'
  steps:
    - uses: actions/checkout@v4

    - uses: actions/setup-python@v5
      with:

```

```

python-version: '3.11'

- name: Install dbt-snowflake
  run: pip install dbt-snowflake==1.8.3

- name: Write private key file
  run: |
    echo "${{ secrets.SNOWFLAKE_PRIVATE_KEY }}" > /tmp/rsa_key.p8
    chmod 600 /tmp/rsa_key.p8

- name: dbt deps
  working-directory: ./dbt
  run: dbt deps

- name: dbt build (full production run)
  working-directory: ./dbt
  env:
    SNOWFLAKE_PRIVATE_KEY_PATH: /tmp/rsa_key.p8
  run: dbt build --target prod

- name: Upload dbt artifacts for next slim CI
  if: success()
  run: |
    aws s3 cp ./dbt/target/manifest.json s3://my-dbt-artifacts/production/
    manifest.json
  env:
    AWS_ACCESS_KEY_ID:      ${{ secrets.AWS_ACCESS_KEY_ID }}
    AWS_SECRET_ACCESS_KEY:  ${{ secrets.AWS_SECRET_ACCESS_KEY }}

- name: Notify Slack on failure
  if: failure()
  uses: slackapi/slack-github-action@v1.26.0
  with:
    payload: |
      {
        "text": "dbt production deploy FAILED on commit ${{ github.sha }}.
        <${{ github.server_url }}/${{ github.repository }}/actions/runs/${{
        github.run_id }}|View run>"
      }
  env:
    SLACK_WEBHOOK_URL: ${{ secrets.SLACK_WEBHOOK_URL }}

```

Chapter 16 Summary

Enterprise Snowflake deployments require treating infrastructure as code, enforcing environment separation via multiple accounts, and automating every deployment via CI/CD. The combination of Terraform (infrastructure), schemachange (DDL migrations), dbt (transformations), and GitHub Actions (orchestration) provides a complete DevOps stack for Snowflake.

Key Takeaways: - Multi-account strategy provides security isolation, cost attribution, and independent scaling. - Database Replication with automated Tasks keeps STAGING synchronized with PROD automatically. - Terraform manages all Snowflake objects declaratively — no more manual `CREATE` statements in production. - dbt's `state:modified+` selector (slim CI) dramatically reduces CI pipeline runtimes. - Never store credentials in GitHub secrets as passwords — use key-pair authentication.

Chapter 17: Monitoring and Observability

17.1 The Two Monitoring Schemas

Snowflake provides monitoring data in two schemas with different trade-offs:

Schema	Latency	History	Scope	Use Case
<code>SNOWFLAKE.ACCOUNT_USAGE</code>	45 min – 3 hours	365 days	Account-wide	Historical analysis, trend reports, compliance
<code><db>.INFORMATION_SCHEMA</code>	Real-time	7 days	Per database	Operational monitoring, live dashboards, alerting

For an operational monitoring dashboard, use `INFORMATION_SCHEMA` for current status and `ACCOUNT_USAGE` for trends and cost analysis.

17.2 Essential Monitoring Queries

```
-- Currently running queries (real-time)
SELECT
  query_id,
  user_name,
  warehouse_name,
  ROUND(DATEDIFF('second', start_time, CURRENT_TIMESTAMP()), 0) AS
    running_seconds,
  LEFT(query_text, 150) AS query_preview
FROM TABLE(INFORMATION_SCHEMA.QUERY_HISTORY_BY_SESSION(
  SESSION_ID      ⇒ CURRENT_SESSION(),
  DATE_RANGE_START ⇒ DATEADD(HOUR, -1, CURRENT_TIMESTAMP())
))
WHERE execution_status = 'RUNNING'
ORDER BY running_seconds DESC;

-- Account-wide query performance dashboard (last 30 days)
SELECT
  TO_DATE(start_time)                AS query_date,
  warehouse_name,
  COUNT(*)                          AS
    total_queries,
  COUNT_IF(execution_status = 'SUCCESS') AS successful,
  COUNT_IF(execution_status = 'FAIL')   AS failed,
  ROUND(AVG(total_elapsed_time) / 1000, 2) AS
    avg_elapsed_sec,
  ROUND(PERCENTILE_CONT(0.95)
    WITHIN GROUP (ORDER BY total_elapsed_time) / 1000, 2) AS
    p95_elapsed_sec,
  ROUND(MAX(total_elapsed_time) / 1000, 2) AS
    max_elapsed_sec,
  SUM(bytes_scanned) / POWER(1024, 3) AS
    total_gb_scanned
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE start_time ≥ DATEADD(DAY, -30, CURRENT_TIMESTAMP())
GROUP BY 1, 2
ORDER BY 1 DESC, total_queries DESC;

-- Warehouse utilization heatmap: credits by hour of day and day of week
SELECT
  DAYOFWEEKISO(start_time) AS day_of_week_iso, -- 1=Mon to 7=Sun
  HOUR(start_time)         AS hour_of_day,
  COUNT(DISTINCT query_id) AS query_count,
  SUM(credits_used)        AS credits_consumed
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
WHERE start_time ≥ DATEADD(DAY, -28, CURRENT_TIMESTAMP())
GROUP BY 1, 2
ORDER BY 1, 2;

-- Use this to find low-utilization windows for maintenance tasks
```

17.3 Snowflake Alerts

Alerts are Snowflake-native scheduled checks that trigger an action (notification, stored procedure call) when a condition is met. They run on a serverless compute pool — no warehouse required.

```

-- Prerequisite: email notification integration
CREATE OR REPLACE NOTIFICATION INTEGRATION email_ops_alerts
  TYPE          = EMAIL
  ENABLED       = TRUE
  ALLOWED_RECIPIENTS = ('data-ops@company.com', 'oncall-data@company.com');

-- Alert: notify when any query has been running more than 15 minutes
CREATE OR REPLACE ALERT alert_long_running_queries
  WAREHOUSE = admin_wh
  SCHEDULE  = '5 MINUTE'
  IF (EXISTS (
    SELECT 1
    FROM TABLE(INFORMATION_SCHEMA.QUERY_HISTORY(
      DATE_RANGE_START => DATEADD(MINUTE, -20, CURRENT_TIMESTAMP()),
      DATE_RANGE_END   => CURRENT_TIMESTAMP()
    ))
    WHERE execution_status = 'RUNNING'
    AND   total_elapsed_time > 900000 -- 15 minutes in milliseconds
  ))
  THEN
    CALL SYSTEM$SEND_EMAIL(
      'email_ops_alerts',
      'oncall-data@company.com',
      'ALERT: Long-Running Query Detected',
      'A query has been running for more than 15 minutes. Log into Snowsight
      and check Query History immediately.'
    );

ALTER ALERT alert_long_running_queries RESUME;

-- Alert: weekly storage budget check
CREATE OR REPLACE ALERT alert_storage_exceeds_threshold
  WAREHOUSE = admin_wh
  SCHEDULE  = 'USING CRON 0 9 * * MON UTC' -- every Monday at 9am UTC
  IF (EXISTS (
    SELECT 1
    FROM SNOWFLAKE.ACCOUNT_USAGE.DATABASE_STORAGE_USAGE_HISTORY
    WHERE usage_date = CURRENT_DATE() - 1
    GROUP BY usage_date
    HAVING SUM(storage_bytes + failsafe_bytes + stage_bytes) / POWER(1024, 4)
      > 20 -- 20TB
  ))
  THEN
    CALL SYSTEM$SEND_EMAIL(
      'email_ops_alerts',
      'data-ops@company.com',
      'ALERT: Storage Exceeds 20TB Threshold',
      'Account storage has exceeded 20TB. Review TABLE_STORAGE_METRICS and
      consider enabling Time Travel reduction or archiving.'
    );

ALTER ALERT alert_storage_exceeds_threshold RESUME;

```

```

-- Alert: failed Snowpipe loads in the last hour
CREATE OR REPLACE ALERT alert_snowpipe_failures
  WAREHOUSE = admin_wh
  SCHEDULE = '15 MINUTE'
  IF (EXISTS (
    SELECT 1
    FROM SNOWFLAKE.ACCOUNT_USAGE.COPY_HISTORY
    WHERE last_load_time ≥ DATEADD(HOUR, -1, CURRENT_TIMESTAMP())
    AND status = 'LOAD_FAILED'
  ))
  THEN
    CALL SYSTEM$SEND_EMAIL(
      'email_ops_alerts',
      'data-ops@company.com',
      'ALERT: Snowpipe Load Failures Detected',
      'One or more Snowpipe files failed to load in the past hour. Check
      SNOWFLAKE.ACCOUNT_USAGE.COPY_HISTORY for details.'
    );

ALTER ALERT alert_snowpipe_failures RESUME;

```

17.4 Event Tables — Structured Application Logging

Event Tables capture logs, traces, and metrics emitted by Snowpark functions, procedures, and UDFs. They store structured telemetry in a queryable Snowflake table.

```

-- Create a dedicated event table
CREATE EVENT TABLE analytics_db.governance.app_event_log
  DATA_RETENTION_TIME_IN_DAYS = 30
  COMMENT = 'Application telemetry: logs and traces from Snowpark functions';

-- Set as the account-level event table
ALTER ACCOUNT SET EVENT_TABLE = analytics_db.governance.app_event_log;

-- Grant read access to the observability team
GRANT SELECT ON TABLE analytics_db.governance.app_event_log
  TO ROLE DATA_OPS_ROLE;

```

```

# In Snowpark Python: emit structured log events
import logging
from snowflake.snowpark import Session

logger = logging.getLogger("data_pipeline")
logger.setLevel(logging.DEBUG)

def process_orders_batch(session: Session, batch_date: str) → dict:
    """Process a batch of orders for the given date."""
    logger.info("Batch processing started", extra={
        "batch_date": batch_date,
        "function": "process_orders_batch"
    })

    try:
        df = session.table("raw.orders").filter(f"order_date = '{batch_date}'")
        record_count = df.count()

        logger.info("Records loaded", extra={
            "batch_date": batch_date,
            "record_count": record_count
        })

        # ... transformation logic ...

        logger.info("Batch processing complete", extra={
            "batch_date": batch_date,
            "records_processed": record_count,
            "status": "success"
        })

        return {"status": "success", "records": record_count}

    except Exception as e:
        logger.error("Batch processing failed", extra={
            "batch_date": batch_date,
            "error_type": type(e).__name__,
            "error_message": str(e)
        })
        raise

```

```

-- Query the event log for operational monitoring
SELECT
    timestamp,
    severity_text AS log_level,
    resource_attributes:snow.executable.name::VARCHAR AS function_name,
    resource_attributes:snow.executable.type::VARCHAR AS executable_type,
    value:message::VARCHAR AS log_message,
    value:batch_date::VARCHAR AS batch_date,
    value:record_count::NUMBER AS record_count,
    value:error_message::VARCHAR AS error_message
FROM analytics_db.governance.app_event_log
WHERE timestamp ≥ DATEADD(HOUR, -24, CURRENT_TIMESTAMP())
AND severity_text IN ('ERROR', 'WARN', 'INFO')
ORDER BY timestamp DESC;

-- Error rate by function (last 7 days)
SELECT
    resource_attributes:snow.executable.name::VARCHAR AS function_name,
    COUNT(*) AS total_events,
    COUNT_IF(severity_text = 'ERROR') AS error_count,
    ROUND(COUNT_IF(severity_text = 'ERROR') / COUNT(*) * 100, 2) AS
        error_rate_pct
FROM analytics_db.governance.app_event_log
WHERE timestamp ≥ DATEADD(DAY, -7, CURRENT_TIMESTAMP())
GROUP BY 1
ORDER BY error_rate_pct DESC;

```

17.5 Snowsight Monitoring Dashboards

Snowsight (the Snowflake web UI) provides built-in monitoring for warehouses, queries, and costs. Beyond the built-in views, you can build custom dashboards using Snowsight Charts that query `ACCOUNT_USAGE` views.

Recommended custom dashboard tiles:

1. **Daily credit consumption** — line chart of `WAREHOUSE_METERING_HISTORY` grouped by day
2. **Credit burn by warehouse** — bar chart for the current month
3. **Query volume and failure rate** — dual-axis line chart
4. **P95 query latency by warehouse** — line chart using `QUERY_HISTORY`
5. **Storage growth trend** — area chart from `DATABASE_STORAGE_USAGE_HISTORY`
6. **Projected monthly cost vs. budget** — single-value tile from the burn rate query in

Chapter 15

Chapter 17 Summary

Observability in Snowflake is built on three layers: `ACCOUNT_USAGE` for historical analysis and trend detection, `INFORMATION_SCHEMA` for real-time operational monitoring, and Event Tables for application-level structured logging. Alerts close the loop by converting monitoring signals into automated notifications.

Key Takeaways: - `ACCOUNT_USAGE` has a 45-minute to 3-hour latency — do not use it for real-time alerting. - Alerts run on serverless compute — set the schedule to match your SLA, not shorter. - Event Tables require you to set `ALTER ACCOUNT SET EVENT_TABLE` — without this, logs are not stored. - Build monitoring dashboards in Snowsight using `ACCOUNT_USAGE` as the data source for a living view of your environment.

Chapter 18: Advanced Topics and Enterprise Patterns

18.1 The Medallion Architecture in Snowflake

The Medallion Architecture organizes data into progressively refined layers. Snowflake's schema hierarchy maps naturally to this pattern:

Layer	Snowflake Schema	Characteristics
Bronze	RAW	Raw, unmodified ingested data; VARIANT columns; transient tables; no Fail-Safe
Silver	STAGING	Cleaned, validated, typed, deduplicated; business rules applied
Gold	MARTS	Business-ready aggregates, dimensional models, optimized for BI tools

The key architectural insight is that each layer serves a different consumer: data engineers work in Bronze and Silver, data analysts consume from Gold, and ML engineers often work in Silver.

Implementing Medallion with Dynamic Tables

Dynamic Tables automate the Silver and Gold layers — when Bronze data changes, Snowflake automatically propagates updates through the chain.

```

-- Bronze layer: raw ingestion (transient saves Fail-Safe costs for regenerable
data)
CREATE OR REPLACE TRANSIENT TABLE raw.orders (
  raw_payload  VARIANT,
  source_file  VARCHAR(500),
  _loaded_at   TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

-- Silver layer: Dynamic Table for automated cleaning
-- Refreshes within 10 minutes of Bronze changes
CREATE OR REPLACE DYNAMIC TABLE staging.silver_orders
  TARGET_LAG = '10 minutes'
  WAREHOUSE  = transform_wh
AS
SELECT
  raw_payload:order_id::VARCHAR(36)      AS order_id,
  raw_payload:customer_id::NUMBER       AS customer_id,
  TO_DATE(raw_payload:order_date::VARCHAR) AS order_date,
  UPPER(TRIM(raw_payload:status::VARCHAR)) AS status,
  ROUND(raw_payload:amount::FLOAT, 2)    AS amount,
  raw_payload:product_id::VARCHAR(50)    AS product_id,
  UPPER(TRIM(raw_payload:region::VARCHAR)) AS region,
  _loaded_at
FROM raw.orders
WHERE raw_payload:order_id IS NOT NULL
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY raw_payload:order_id::VARCHAR
  ORDER BY _loaded_at DESC
) = 1;

-- Gold layer: business aggregate – depends on silver_orders
-- Refreshes within 1 hour of silver changes (chain propagation)
CREATE OR REPLACE DYNAMIC TABLE marts.gold_daily_revenue
  TARGET_LAG = '1 hour'
  WAREHOUSE  = analytics_wh
AS
SELECT
  order_date,
  region,
  status,
  COUNT(DISTINCT order_id) AS order_count,
  COUNT(DISTINCT customer_id) AS unique_customers,
  SUM(amount) AS total_revenue,
  AVG(amount) AS avg_order_value,
  MIN(amount) AS min_order_value,
  MAX(amount) AS max_order_value
FROM staging.silver_orders
WHERE status = 'COMPLETED'
GROUP BY order_date, region, status;

-- Monitor Dynamic Table refresh lag
SELECT
  name,

```

```
target_lag,  
scheduling_state,  
last_completed_dependency_update_time,  
data_timestamp  
FROM INFORMATION_SCHEMA.DYNAMIC_TABLES  
WHERE schema_name IN ('STAGING', 'MARTS')  
ORDER BY name;
```

18.2 Iceberg Tables — Open Lakehouse Format

Apache Iceberg is an open table format for large analytic datasets. Snowflake Iceberg Tables store data in Iceberg format in your own cloud storage (S3, ADLS, GCS), while Snowflake provides the SQL query engine. This means other engines — Apache Spark, Trino, Flink, Dremio — can read and write the same data.

```

-- Step 1: Create an external volume pointing to your cloud storage
-- The IAM role must have S3 GetObject, PutObject, ListBucket, DeleteObject
  permissions
CREATE OR REPLACE EXTERNAL VOLUME iceberg_volume
  STORAGE_LOCATIONS = (
    (
      NAME                = 'prod-iceberg-s3',
      STORAGE_PROVIDER    = 'S3',
      STORAGE_BASE_URL    = 's3://mycompany-iceberg-data/snowflake/',
      STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::123456789012:role/snowflake-
iceberg-role',
      STORAGE_AWS_EXTERNAL_ID = 'iceberg_external_id_12345'
    )
  );

-- Verify Snowflake can access the volume
DESCRIBE EXTERNAL VOLUME iceberg_volume;
-- Use the STORAGE_AWS_IAM_USER_ARN to configure the trust policy on the IAM role

-- Step 2: Create an Iceberg table
-- CATALOG = 'SNOWFLAKE' means Snowflake manages the Iceberg catalog
-- Data is written to YOUR S3 in open Iceberg format
CREATE OR REPLACE ICEBERG TABLE marts.fct_orders_iceberg (
  order_id      VARCHAR,
  customer_id   LONG,
  order_date    DATE,
  amount        DOUBLE,
  status        VARCHAR,
  region        VARCHAR,
  product_id    VARCHAR,
  loaded_at     TIMESTAMP
)
CATALOG          = 'SNOWFLAKE'
EXTERNAL_VOLUME  = 'iceberg_volume'
BASE_LOCATION    = 'fct_orders/'
CLUSTER BY (order_date, region);

-- Step 3: Write data (it lands in S3 as Parquet with Iceberg metadata)
INSERT INTO marts.fct_orders_iceberg
SELECT
  order_id,
  customer_id,
  order_date,
  amount,
  status,
  region,
  product_id,
  loaded_at
FROM staging.silver_orders;

-- Step 4: Iceberg tables support full DML
UPDATE marts.fct_orders_iceberg
SET status = 'REFUNDED'

```

```
WHERE order_id = 'ord-99999'
AND status = 'COMPLETED';

DELETE FROM marts.fct_orders_iceberg
WHERE order_date < DATEADD(YEAR, -5, CURRENT_DATE());

-- Step 5: Iceberg time travel (uses Iceberg snapshots)
SELECT * FROM marts.fct_orders_iceberg
AT (TIMESTAMP => DATEADD(HOUR, -6, CURRENT_TIMESTAMP()));
```

18.3 Hybrid Tables — Operational Analytics (UNISTORE)

Hybrid Tables combine row-based storage (optimized for transactional point lookups) with columnar storage (optimized for analytics) in a single table object. They support primary keys, unique constraints, secondary indexes, and multi-statement ACID transactions.

Hybrid Tables are the foundation of Snowflake's UNISTORE workload: operational data and analytics on the same platform, eliminating the need to replicate data from an OLTP database into a data warehouse.

```

-- Hybrid Table: supports fast point lookups AND analytical queries
CREATE OR REPLACE HYBRID TABLE operations.order_fulfillment (
  order_id      VARCHAR(36) NOT NULL PRIMARY KEY,
  customer_id   NUMBER      NOT NULL,
  status        VARCHAR(50) NOT NULL DEFAULT 'PENDING',
  created_at    TIMESTAMP_NTZ NOT NULL DEFAULT CURRENT_TIMESTAMP(),
  shipped_at     TIMESTAMP_NTZ,
  delivered_at  TIMESTAMP_NTZ,
  tracking_number VARCHAR(100),
  carrier       VARCHAR(50),
  warehouse_id  NUMBER,

  -- Secondary indexes enable fast lookups without full table scans
  INDEX idx_customer (customer_id),
  INDEX idx_status   (status, created_at),
  INDEX idx_warehouse (warehouse_id, status)
);

-- Fast point lookup: row-based path — typically single-digit milliseconds
SELECT * FROM operations.order_fulfillment
WHERE order_id = 'ord-20240115-00001';

-- Analytical query: columnar path — same table, different execution path
SELECT
  status,
  carrier,
  COUNT(*)                                AS order_count,
  AVG(DATEDIFF('HOUR', created_at, shipped_at)) AS avg_hours_to_ship,
  AVG(DATEDIFF('HOUR', shipped_at, delivered_at)) AS
    avg_hours_in_transit,
  PERCENTILE_CONT(0.95) WITHIN GROUP
    (ORDER BY DATEDIFF('HOUR', created_at, shipped_at)) AS p95_hours_to_ship
FROM operations.order_fulfillment
WHERE created_at ≥ DATEADD(DAY, -30, CURRENT_TIMESTAMP())
AND status IN ('SHIPPED', 'DELIVERED')
GROUP BY status, carrier
ORDER BY avg_hours_to_ship DESC;

-- Multi-statement transaction (Snowpark Python)
def ship_order(session, order_id: str, tracking_number: str, carrier: str) →
  str:
  """Atomically update order status and record shipment event."""
  try:
    session.sql("BEGIN").collect()

    session.sql(f"""
      UPDATE operations.order_fulfillment
      SET
        status          = 'SHIPPED',
        shipped_at       = CURRENT_TIMESTAMP(),
        tracking_number = '{tracking_number}',
        carrier         = '{carrier}'
      WHERE order_id = '{order_id}'
    """).collect()
  except:
    session.sql("ROLLBACK").collect()
    raise

```

```

        AND status = 'PROCESSING'
    """).collect()

    session.sql(f"""
        INSERT INTO operations.shipment_events
            (order_id, event_type, event_at, tracking_number, carrier)
        VALUES
            ({'order_id'}, 'SHIPPED', CURRENT_TIMESTAMP(), {'tracking_number'}, {'carrier'})
    """).collect()

    session.sql("COMMIT").collect()
    return f"Order {order_id} shipped successfully"

except Exception as e:
    session.sql("ROLLBACK").collect()
    raise RuntimeError(f"Failed to ship order {order_id}: {e}") from e

```

18.4 Snowflake Native Apps

The Native Apps Framework lets you build data applications that run inside a consumer's Snowflake account. The application code runs within the consumer's security perimeter — the consumer's data never leaves their account. You distribute via the Snowflake Marketplace.

```

-- Provider: create an application package (the distributable artifact)
CREATE APPLICATION PACKAGE analytics_suite_pkg
  COMMENT = 'Advanced analytics suite for sales teams';

-- Create a stage to hold application files
CREATE STAGE analytics_suite_pkg.v1_stage;

-- setup.sql: executed when the consumer installs the app
-- This script defines everything the app creates in the consumer account
CREATE SCHEMA IF NOT EXISTS app_public;

CREATE OR REPLACE PROCEDURE app_public.analyze_revenue(
  target_table    VARCHAR,
  date_column     VARCHAR,
  amount_column   VARCHAR
)
  RETURNS TABLE (
    period         DATE,
    total_revenue  FLOAT,
    order_count    NUMBER,
    avg_order      FLOAT,
    mom_growth_pct FLOAT
  )
  LANGUAGE PYTHON
  RUNTIME_VERSION = '3.11'
  PACKAGES = ('snowflake-snowpark-python')
  HANDLER = 'analyze_revenue'
AS $$
from snowflake.snowpark import Session
import snowflake.snowpark.functions as F

def analyze_revenue(session, target_table, date_column, amount_column):
    df = session.table(target_table)
    monthly = (
        df
        .with_column('period', F.date_trunc('MONTH', F.col(date_column)))
        .group_by('period')
        .agg(
            F.sum(F.col(amount_column)).alias('total_revenue'),
            F.count('*').alias('order_count'),
            F.avg(F.col(amount_column)).alias('avg_order')
        )
        .sort('period')
    )

    from snowflake.snowpark.window import Window
    w = Window.order_by('period')
    result = monthly.with_column(
        'mom_growth_pct',
        (F.col('total_revenue') - F.lag('total_revenue', 1).over(w))
        / F.lag('total_revenue', 1).over(w) * 100
    )

```

```
        return result
    $$;

-- Add a Streamlit UI to the app package
ALTER APPLICATION PACKAGE analytics_suite_pkg
    ADD STREAMLIT app_public.main_ui
    FROM @analytics_suite_pkg.v1_stage/streamlit/
    MAIN_FILE = 'main.py';

-- Publish a version
ALTER APPLICATION PACKAGE analytics_suite_pkg
    ADD VERSION v1_0 USING @analytics_suite_pkg.v1_stage;

-- Consumer: install the app from the marketplace or directly
CREATE APPLICATION analytics_suite
    FROM APPLICATION PACKAGE analytics_suite_pkg
    USING VERSION v1_0;

-- Consumer: grant app access to their data
GRANT SELECT ON TABLE my_sales_data TO APPLICATION analytics_suite;

-- Consumer: call the app's procedure on their own data
CALL analytics_suite.app_public.analyze_revenue(
    'MY_SALES_DATA',
    'SALE_DATE',
    'SALE_AMOUNT'
);
```

18.5 Performance at Enterprise Scale

At very large data volumes (trillions of rows, petabyte-scale), several additional patterns apply.

Multi-Table Inserts

Fan out a single source scan into multiple target tables in one pass — saves compute vs. scanning the source multiple times:

```

-- Route raw events to typed target tables in one scan
FROM staging.raw_events
INSERT INTO marts.clickstream_events
    SELECT session_id, user_id, page_url, clicked_element, event_timestamp
    WHERE event_type = 'click'
INSERT INTO marts.purchase_events
    SELECT session_id, user_id, order_id, amount, event_timestamp
    WHERE event_type = 'purchase'
INSERT INTO marts.pageview_events
    SELECT session_id, user_id, page_url, referrer, duration_sec, event_timestamp
    WHERE event_type = 'pageview'
INSERT INTO marts.search_events
    SELECT session_id, user_id, search_query, result_count, event_timestamp
    WHERE event_type = 'search';

```

Optimized MERGE for Large Tables

```

-- Pre-filter the source to only changed records before the MERGE
-- This dramatically reduces the MERGE target scan
MERGE INTO marts.fct_orders t
USING (
    SELECT s.*
    FROM staging.silver_orders s
    -- Only include records newer than the most recent target record
    -- This turns a full-table MERGE into a narrow-window MERGE
    WHERE s.loaded_at > (
        SELECT COALESCE(MAX(loaded_at), '1900-01-01'::TIMESTAMP_NTZ)
        FROM marts.fct_orders
    )
) s
ON t.order_id = s.order_id
WHEN MATCHED AND t.status ≠ s.status THEN
    UPDATE SET
        t.status      = s.status,
        t.loaded_at   = s.loaded_at
WHEN NOT MATCHED THEN
    INSERT (order_id, customer_id, order_date, amount, status, region,
            product_id, loaded_at)
    VALUES (s.order_id, s.customer_id, s.order_date, s.amount, s.status,
            s.region, s.product_id, s.loaded_at);

```

Search Optimization Service

For selective lookups on large tables (finding one row among billions):

```
-- Enable Search Optimization for point-lookup columns
ALTER TABLE marts.fct_orders
  ADD SEARCH OPTIMIZATION ON EQUALITY(order_id, customer_id);

-- Check Search Optimization usage and cost
SELECT
  table_name,
  active_bytes / POWER(1024, 3) AS search_opt_storage_gb
FROM SNOWFLAKE.ACCOUNT_USAGE.TABLE_STORAGE_METRICS
WHERE search_optimization_bytes > 0
ORDER BY search_optimization_bytes DESC;
```

18.6 Enterprise Naming Conventions and Standards

Consistent naming conventions are a force multiplier at enterprise scale. When every engineer follows the same conventions, tooling (dbt, Terraform, lineage graphs, data catalogs) works without custom configuration.

```

-- =====
-- SNOWFLAKE ENTERPRISE NAMING CONVENTIONS
-- =====

-- ACCOUNTS: <org>-<environment>
-- Examples: acme-prod, acme-staging, acme-dev, acme-security

-- DATABASES: <environment>_<purpose> (UPPER_CASE)
-- Examples: PROD_ANALYTICS, STAGING_ANALYTICS, DEV_ANALYTICS, PROD_RAWDATA

-- SCHEMAS: layer name (UPPER_CASE)
-- RAW          - bronze layer, raw ingestion
-- STAGING       - silver layer, cleaned data
-- MARTS         - gold layer, business-ready models
-- ML_FEATURES   - feature store for ML models
-- GOVERNANCE    - audit logs, event tables, policy mapping tables
-- SCHEMACHANGE  - migration history (managed by schemachange)

-- TABLES by type:
-- Raw:         raw_<source>_<entity>          raw_salesforce_accounts
-- Staging:     stg_<entity>                   stg_accounts
-- Dims:        dim_<entity>                   dim_customers
-- Facts:       fct_<entity>                   fct_orders
-- Bridge:      brg_<entity1>_<entity2>        brg_orders_products
-- Dynamic:     dyn_<description>              dyn_live_inventory
-- Views:       v_<description>                v_active_customers
-- Secure:      sv_<description>              sv_customer_pii

-- WAREHOUSES: <team>_<size>_wh (UPPER_CASE)
-- DATA_ENG_M_WH, ANALYTICS_L_WH, REPORTING_S_WH, DS_XL_WH

-- ROLES: <scope>_<access_level>_role (UPPER_CASE)
-- DATA_ENGINEER_RW_ROLE, ANALYTICS_RO_ROLE, FINANCE_RO_ROLE
-- dbt service account: DBT_ROLE
-- Snowpipe service account: SNOWPIPE_ROLE

-- TAGS: <domain>_<attribute> (lower_case with underscores)
-- pii_category, data_classification, data_owner, data_domain, cost_center

-- RESOURCE MONITORS: <scope>_<frequency>_<type> (lower_case)
-- account_monthly_guard, analytics_team_monthly, ds_weekly_budget

-- TASKS: <action>_<object>_<frequency> (lower_case)
-- refresh_staging_db_daily, process_orders_hourly, archive_old_data_weekly

-- STAGES: <source_or_purpose>_stage (lower_case)
-- s3_orders_stage, azure_crm_stage, invoice_pdf_stage

-- FILE FORMATS: <format>_<purpose> (lower_case)
-- parquet_raw_ingest, csv_bulk_load, json_api_events

```

18.7 The Complete Snowflake Architecture Reference

Bringing together everything from Parts 1, 2, and 3, here is a reference architecture for a production Snowflake data platform:



GOVERNANCE: Tags → Masking Policies → Row Access Policies
Access History → Compliance Reporting

DEVOPS: Terraform → schemachange → dbt → GitHub Actions

MONITORING: Account Usage → Alerts → Event Tables → Snowsight

Chapter 18 Summary and Course Conclusion

Congratulations — you have completed the Snowflake Master Course. You now command the full surface area of one of the most capable data platforms available.

What you have learned across all 18 chapters:

- **Architecture:** Virtual warehouses, storage separation, multi-cluster design, cloud services layer
- **Data Loading:** COPY INTO, Snowpipe, Snowpipe Streaming, semi-structured data with VARIANT
- **SQL Mastery:** Window functions, recursive CTEs, PIVOT/UNPIVOT, QUALIFY, zero-copy cloning, Time Travel
- **Performance:** Micro-partitions, clustering keys, materialized views, Dynamic Tables, query profiling
- **Security:** RBAC, column masking, row access policies, network policies, key-pair authentication
- **Data Sharing:** Secure data sharing, Marketplace, Clean Rooms
- **Snowpark:** Python and Scala DataFrames, UDFs, stored procedures, vectorized UDFs
- **Streamlit in Snowflake:** Native app development without infrastructure
- **Cortex AI:** LLM functions, semantic search, Document AI, Cortex Analyst, ML functions
- **Governance:** Snowflake Horizon, object tagging, automatic classification, access history, lineage
- **Cost Management:** Resource monitors, credit analysis, storage optimization, rightsizing
- **DevOps:** Multi-account strategy, Terraform, dbt, schemachange, GitHub Actions CI/CD
- **Monitoring:** Account Usage views, Alerts, Event Tables, Snowsight dashboards
- **Enterprise Patterns:** Medallion architecture, Iceberg Tables, Hybrid Tables, Native Apps

Your Next Steps:

1. **Certifications:** Begin with SnowPro Core, then pursue SnowPro Advanced: Data Engineer, Architect, or Data Scientist depending on your role.
2. **Portfolio Project:** Build an end-to-end pipeline — ingest from a public API → dbt transformations → Streamlit dashboard → Cortex AI analysis.
3. **Community:** Join the Snowflake Community (community.snowflake.com), follow the Snowflake Blog, and attend Snowflake Summit.

4. **Stay Current:** Snowflake ships features continuously. Subscribe to the release notes at docs.snowflake.com/en/release-notes.

The data platform landscape is evolving rapidly — open formats (Iceberg), AI-native analytics (Cortex), and operational-analytic convergence (UNISTORE/Hybrid Tables) are reshaping what is possible. The foundation you have built in this course positions you to grow with these trends.

End of Part 3 — Snowflake Master Course: Zero to Hero

Part 1 covered Chapters 1–6 (foundations and core SQL). Part 2 covered Chapters 7–12 (engineering and advanced features). This document, Part 3, covers Chapters 13–18 (AI, governance, DevOps, and enterprise architecture).